PawPet

A Mobile Client-Server System for Veterinary Inventory, Scheduling, and Appointment Management

Integrated System Development Project

Group Members

[Member 1 full name]

[Member 2 full name]

[Member 3 full name]

[Member 4 full name]

Bachelor of Science in Information Technology
Third Year

Subject: Integrated Programming

Date of Submission: [Month Day, Year]

SYSTEM OVERVIEW

Purpose. PawPet is a client-server system that helps a veterinary clinic and pet shop manage its inventory, veterinarian schedules, and customer appointments in one place. Clinic staff use a desktop application, while pet owners use a mobile application. Both connect to one server and one MySQL database.

Intended users. (1) Administrators and staff, who use the desktop application (landscape) to manage customers, products, batches, services, veterinarians, appointments, and sales. (2) Pet owners (customers), who use the Ionic mobile application (portrait) to browse the shop and services, book appointments, and read notifications.

Problem addressed. Many small veterinary clinics still record appointments, stock, and sales on paper or in separate files. This causes double-booked veterinarians, forgotten appointments, running out of stock, and medicines that expire unnoticed on the shelf.

Main features.

- Customer management and pet profiles (registration and login for customers, roles for staff and admin).
- Product and inventory management with reorder levels, low-stock alerts, and batch/lot tracking with expiry dates (first-expiry-first-out deduction, expired stock blocked from sale, discard of expired batches).
- Veterinarian availability tracking (working days and hours) and automatic time-slot generation that prevents double booking.
- Appointment booking, scheduling, confirmation, completion, cancellation, no-show, and follow-up appointments.
- Service and sales management; completing a visit records the sale and deducts the products used.
- Notifications and reminders sent to customers and shown in the mobile app.
- Daily appointment list and an admin dashboard (today's appointments, pending requests, low stock, expiring medicines, sales). The desktop app refreshes automatically every 8 seconds.

SYSTEM INTEGRATION

The system has three components that share one database through one server. The desktop application (Python with the Flet UI library) runs on Laptop 1, the Flask server and MySQL database run on Laptop 2, and the Ionic mobile application runs on a smartphone or emulator. All devices are connected to the same Wi-Fi or local area network, and both client applications use the server laptop's local IP address (for example 192.168.1.10:5000), not localhost.

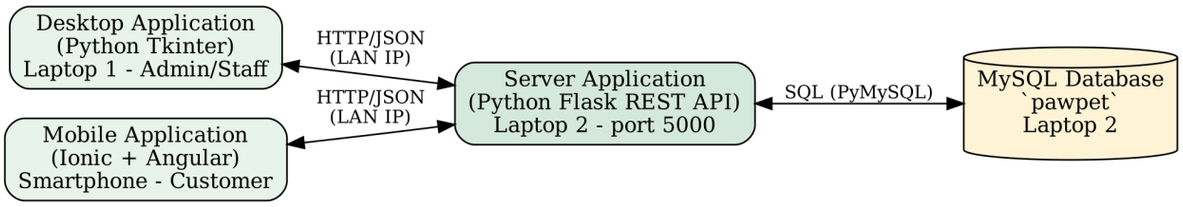


Figure 1. System architecture.

Communication. The clients send HTTP requests with JSON bodies to the REST API of the server. After login the server returns a bearer token, which the clients send in the Authorization header. The server validates the input (required fields, dates, quantities, role permissions, time-slot conflicts), runs the SQL, and returns JSON with an appropriate HTTP status code. Neither client connects directly to the database.

Shared data and consistency. A record added in the desktop application (for example a product, a veterinarian schedule, or a reminder) is stored in MySQL by the server and is retrieved by the mobile application on its next request. An appointment booked on the mobile application appears in the desktop application as a pending request and is shown automatically within seconds. Real-time push is not required; the latest saved data is retrieved through the API.

Reliability and security. Every booking and stock change runs under one server lock, so two customers can never take the same time slot or the same stock. All input is validated (types, lengths, dates, ranges) and bad input returns a clear JSON error. Passwords are stored as salted PBKDF2 hashes, repeated wrong logins are temporarily blocked, sessions expire after 30 days, and accounts created or reset by staff must choose a new password at first sign-in.

Error handling. If the server cannot be reached, the desktop application shows a "Cannot reach the server" message and the mobile application shows a connection error. Invalid requests receive a 400/401/403/404/409 response with an error message that the applications display.

DATABASE DESIGN

A. Entity-Relationship Diagram

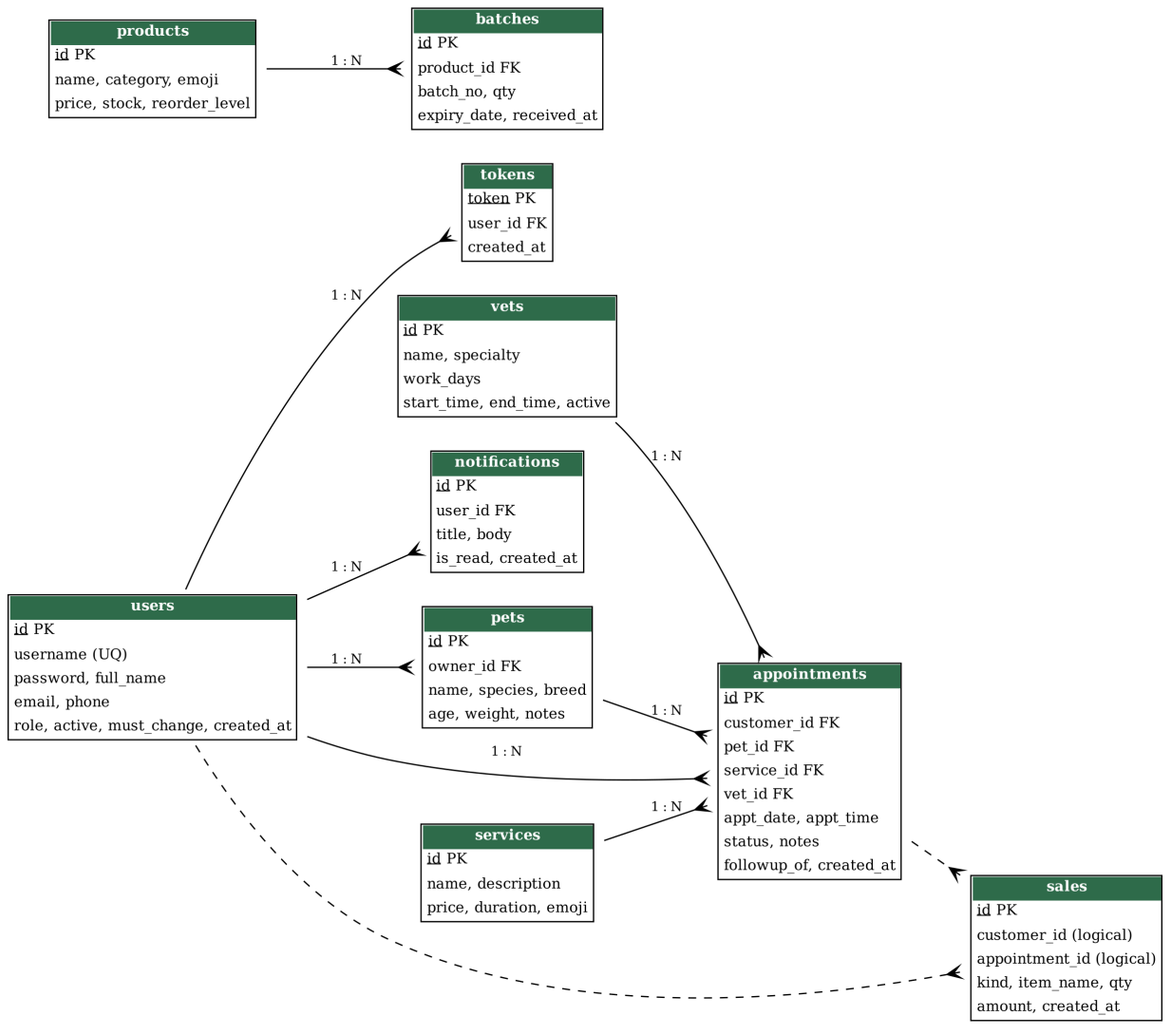


Figure 2. ERD of the pawpet database. Solid lines are foreign-key constraints; dashed lines on sales are logical references (no FK constraint) so that sales history is kept if a customer or appointment is removed.

B. Database Dictionary

Table: users

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique user ID
username	VARCHAR(50)	UQ	NOT NULL	Login name (unique)
password	VARCHAR(200)		NOT NULL	Salted PBKDF2-SHA256 hash (pbkdf2$iterations$salt$hash)
full_name	VARCHAR(100)		NOT NULL	Full name
email	VARCHAR(100)		NULL	Email address
phone	VARCHAR(30)		NULL	Contact number
role	VARCHAR(20)		NOT NULL	customer, staff, or admin
active	TINYINT		NOT NULL	1 = active, 0 = disabled
must_change	TINYINT		NOT NULL	1 = must choose a new password at next sign-in
created_at	VARCHAR(19)		NOT NULL	Date/time registered

Table: tokens

Field	Data type	Key	Null	Description
token	VARCHAR(64)	PK	NOT NULL	Session token
user_id	INT	FK	NOT NULL	Owner of the session (FK to users.id)
created_at	VARCHAR(19)		NOT NULL	Date/time issued

Table: pets

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique pet ID
owner_id	INT	FK	NOT NULL	Customer who owns the pet (FK to users.id)
name	VARCHAR(60)		NOT NULL	Pet name
species	VARCHAR(30)		NOT NULL	Dog, cat, etc.
breed	VARCHAR(60)		NULL	Breed
age	DOUBLE		NULL	Age in years
weight	DOUBLE		NULL	Weight in kg
notes	VARCHAR(255)		NULL	Medical or other notes

Table: products

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique product ID
name	VARCHAR(100)		NOT NULL	Product name
category	VARCHAR(40)		NOT NULL	Medicine, food, accessory, etc.
price	DOUBLE		NOT NULL	Unit price
stock	INT		NOT NULL	Total quantity on hand
reorder_level	INT		NOT NULL	Low-stock threshold
emoji	VARCHAR(8)		NULL	Icon shown in the apps

Table: batches

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique batch ID
product_id	INT	FK	NOT NULL	Product the batch belongs to (FK to products.id)
batch_no	VARCHAR(40)		NULL	Lot/batch number
qty	INT		NOT NULL	Quantity remaining in the batch
expiry_date	VARCHAR(10)		NULL	Expiry date (YYYY-MM-DD)
received_at	VARCHAR(19)		NOT NULL	Date/time received

Table: services

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique service ID
name	VARCHAR(100)		NOT NULL	Service name
description	VARCHAR(255)		NULL	Short description
price	DOUBLE		NOT NULL	Service fee
duration	INT		NOT NULL	Duration in minutes
emoji	VARCHAR(8)		NULL	Icon shown in the apps

Table: vets

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique veterinarian ID
name	VARCHAR(100)		NOT NULL	Veterinarian name
specialty	VARCHAR(100)		NULL	Specialty
work_days	VARCHAR(20)		NOT NULL	Working days: 0=Mon, 1=Tue ... 6=Sun (comma-separated)
start_time	VARCHAR(5)		NOT NULL	Shift start (HH:MM)
end_time	VARCHAR(5)		NOT NULL	Shift end (HH:MM)
active	TINYINT		NOT NULL	1 = available, 0 = inactive

Table: appointments

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique appointment ID
customer_id	INT	FK	NOT NULL	Customer who booked (FK to users.id)
pet_id	INT	FK	NOT NULL	Pet to be seen (FK to pets.id)
service_id	INT	FK	NOT NULL	Requested service (FK to services.id)
vet_id	INT	FK	NOT NULL	Assigned veterinarian (FK to vets.id)
appt_date	VARCHAR(10)		NOT NULL	Appointment date (YYYY-MM-DD)
appt_time	VARCHAR(5)		NOT NULL	Start time (HH:MM)
status	VARCHAR(20)		NOT NULL	pending, confirmed, completed, cancelled, or no_show
notes	VARCHAR(500)		NULL	Customer notes
followup_of	INT		NULL	ID of the earlier appointment this follows up (logical self-reference)
created_at	VARCHAR(19)		NOT NULL	Date/time booked

Table: sales

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique sale ID
customer_id	INT		NULL	Customer (logical reference to users, optional for walk-ins)
appointment_id	INT		NULL	Related appointment (logical reference, optional)
kind	VARCHAR(20)		NOT NULL	service or product
item_name	VARCHAR(100)		NOT NULL	Item sold
qty	INT		NOT NULL	Quantity
amount	DOUBLE		NOT NULL	Total amount
created_at	VARCHAR(19)		NOT NULL	Date/time of sale

Table: notifications

Field	Data type	Key	Null	Description
id	INT	PK	NOT NULL	Unique notification ID
user_id	INT	FK	NOT NULL	Recipient (FK to users.id)
title	VARCHAR(120)		NOT NULL	Title
body	VARCHAR(500)		NULL	Message
is_read	TINYINT		NOT NULL	1 = read, 0 = unread
created_at	VARCHAR(19)		NOT NULL	Date/time sent

Table: schema_version

Field	Data type	Key	Null	Description
version	INT	PK	NOT NULL	Schema version number (the server checks it at start-up)
description	VARCHAR(100)		NOT NULL	What this version is
applied_at	TIMESTAMP		NOT NULL	When it was applied

APPLICATION SCREENSHOTS

[Insert screenshot here]

Figure 3. Desktop application - Admin Dashboard.

The dashboard shows today's appointments, pending booking requests from customers (with Confirm and Decline), low-stock items, expiring medicines, and sales totals. It refreshes automatically.

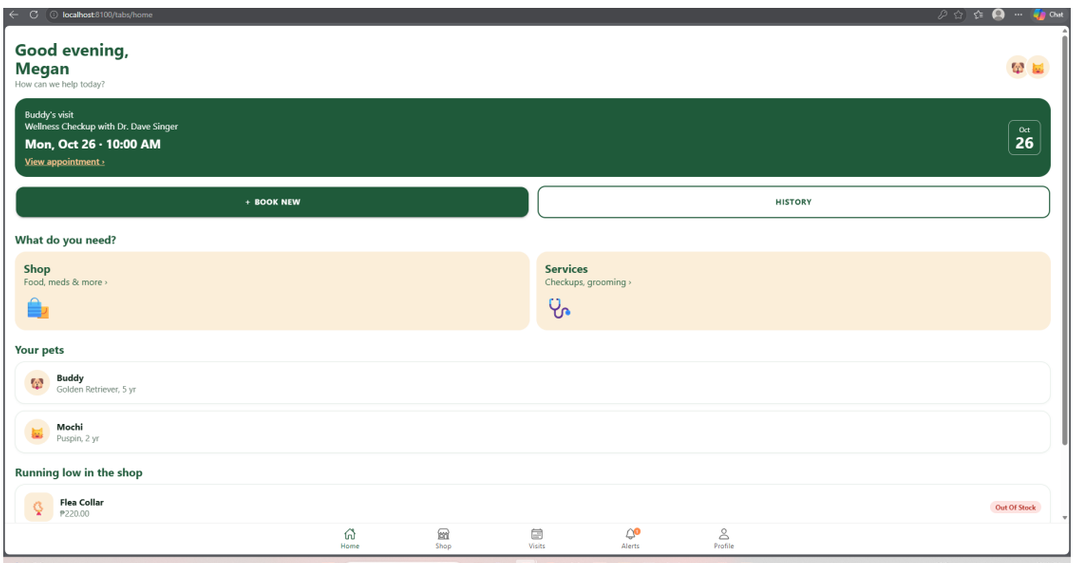


Figure 4. Mobile application - Customer Home.

The customer home screen shows the next appointment, quick links to the Shop and Services, the customer's pets, and low-stock products.

[Insert screenshot here]

Figure 5. Desktop - Appointments (daily list).

Staff see all appointments, confirm, complete (recording the sale), mark no-show, cancel, or create a follow-up.

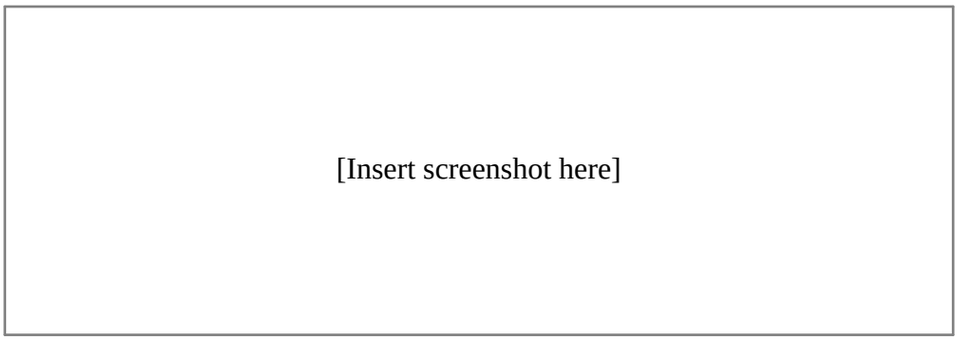


Figure 6. Desktop - Inventory and Batches & Expiry.

Staff receive new batches with expiry dates, see expired and near-expiry stock, and discard expired batches.

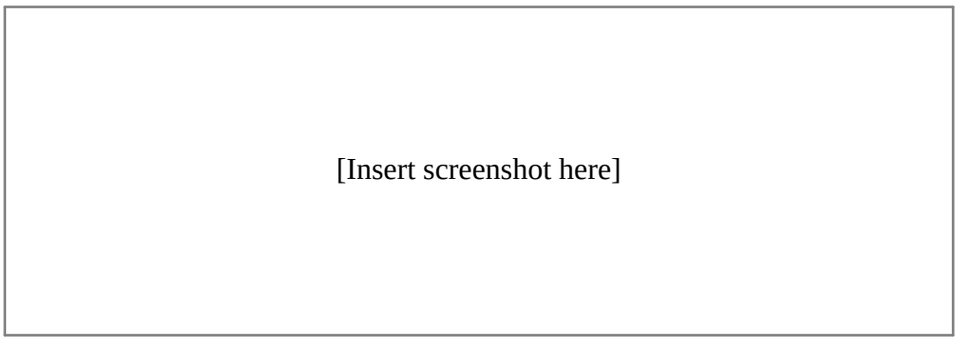


Figure 7. Mobile - Shop and Services.

Customers browse products with prices and live stock, and the list of services.

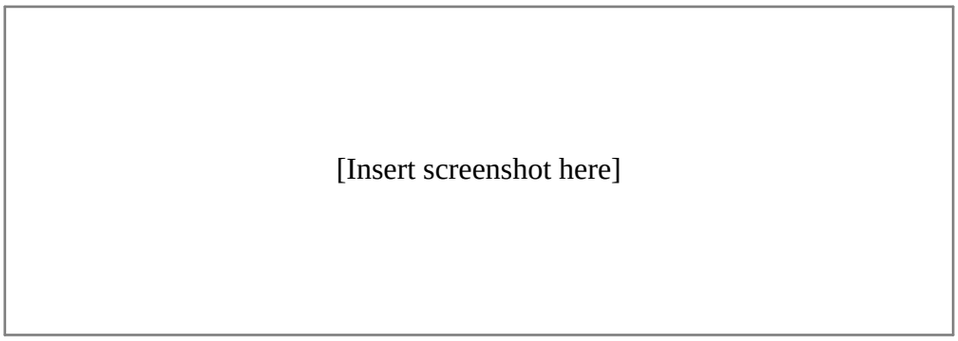


Figure 8. Mobile - Book Appointment.

Customers choose a pet, service, veterinarian, date and an available time slot.

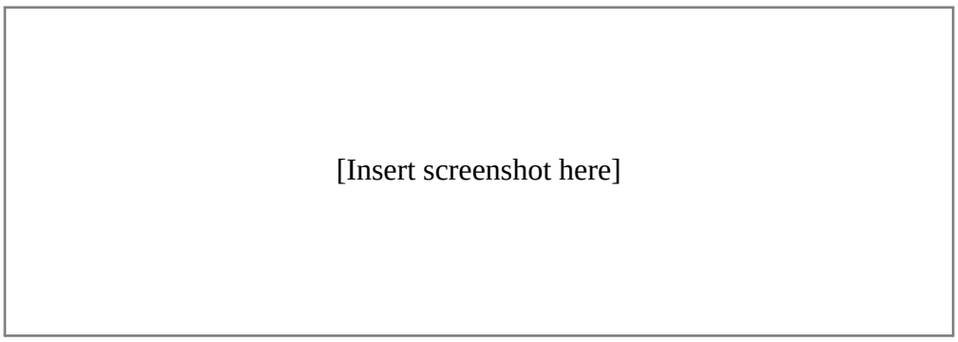


Figure 9. Mobile - Notifications (Alerts).

Reminders and appointment status updates sent from the desktop application.

SOURCE CODE AND SETUP GUIDE

A. Database setup

1. Install XAMPP and start **MySQL** (and Apache if phpMyAdmin is wanted). 2. Run the one-time database setup, `py setup_database.py` (or `setup_database.bat`). It creates the **pawpet** database, runs `database/schema.sql` (plain CREATE TABLE statements, one table definition per table, plus indexes) and adds sample data and random first-run passwords saved in `FIRST_RUN_PASSWORDS.txt`. A database from an older version is upgraded without deleting data; `py setup_database.py --reset` rebuilds everything (asks first). The server itself never creates or alters tables: it only checks the `schema_version` table and stops with a clear message if setup was not done. 3. If MySQL has a password, set `PAWPET_DB_PASSWORD` (also `PAWPET_DB_HOST`, `PAWPET_DB_USER`, `PAWPET_DB_NAME`).

B. Server (Laptop 2)

Run `cd server`, `py -m pip install -r requirements.txt`, then `py app.py`. Find the laptop IP with `ipconfig` (IPv4 Address) and allow Python through Windows Firewall (private network). Test: http://<server-ip>:5000/api/health. Sign in with the passwords in FIRST_RUN_PASSWORDS.txt; admin and staff must choose a new password at first sign-in.

C. Desktop application (Laptop 1)

Run `cd admin`, `py -m pip install -r requirements.txt`, then `py admin.py` (the requirements install Flet, the Python UI library used by the desktop app). On the login screen set Server URL to `http://<server-ip>:5000`. Staff and admin accounts must change their first-run password when they sign in.

D. Mobile application (Ionic)

Run `npm install -g @ionic/cli`, `ionic start pawpet-app blank --type=angular`, `npm install zone.js`, copy `client/src` over `pawpet-app/src`, set `API_URL` in `src/app/config.ts` to `http://<server-ip>:5000/api`, then `ionic serve` (browser/emulator) or `ionic cap run android` (Android device on the same Wi-Fi).

E. Source code

LAPTOP2_Server/db.py - Server - database layer

```
"""Database layer for PawPet.

Default engine is MySQL (PyMySQL). Set PAWPET_DB=sqlite to use a local file instead
(handy for quick tests without MySQL installed).

MySQL settings (environment variables, defaults in brackets):
    PAWPET_DB_HOST [localhost]   PAWPET_DB_PORT [3306]
    PAWPET_DB_USER [root]        PAWPET_DB_PASSWORD []
    PAWPET_DB_NAME [pawpet]
"""
import os
import re
import sqlite3

ENGINE = os.environ.get("PAWPET_DB", "mysql").lower()
SQLITE_PATH = os.environ.get("PAWPET_SQLITE_PATH", os.path.join(os.path.dirname(os.path.abspath(__file__)), "pawpet.db"

MYSQL_CFG = dict(
    host=os.environ.get("PAWPET_DB_HOST", "localhost"),
    port=int(os.environ.get("PAWPET_DB_PORT", "3306")),
    user=os.environ.get("PAWPET_DB_USER", "root"),
    password=os.environ.get("PAWPET_DB_PASSWORD", ""),
)
MYSQL_NAME = os.environ.get("PAWPET_DB_NAME", "pawpet")

class Conn:
    """Tiny wrapper so app.py is identical for MySQL and SQLite. SQL is written with ? placeholders."""

    def __init__(self, database=True):
        if ENGINE == "sqlite":
            self.raw = sqlite3.connect(SQLITE_PATH)
            self.raw.row_factory = sqlite3.Row
        else:
            import pymysql
            import pymysql.cursors
            cfg = dict(MYSQL_CFG, charset="utf8mb4", cursorclass=pymysql.cursors.DictCursor, autocommit=False,
                       connect_timeout=5, read_timeout=15, write_timeout=15,
                       init_command="SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED")
            if database:
                cfg["database"] = MYSQL_NAME
            self.raw = pymysql.connect(**cfg)

    def _sql(self, sql):
        return sql if ENGINE == "sqlite" else sql.replace("?", "%s")

    def query(self, sql, args=()):
        cur = self.raw.cursor()
        cur.execute(self._sql(sql), tuple(args))
        rs = cur.fetchall()
        cur.close()
        return [dict(r) for r in rs]

    def execute(self, sql, args=()):
        cur = self.raw.cursor()
        cur.execute(self._sql(sql), tuple(args))
        last = cur.lastrowid
        cur.close()
        self.raw.commit()
        return last

    def close(self):
        try:
            self.raw.close()
        except Exception:
            pass


# schema (the tables live in database/schema.sql)
SCHEMA_VERSION = 1        # must match the last INSERT INTO schema_version in database/schema.sql
SCHEMA_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "database", "schema.sql")
# Future changes: bump SCHEMA_VERSION, add the change to schema.sql AND list it here, e.g.
#   UPGRADES = {2: ["ALTER TABLE pets ADD COLUMN birthday VARCHAR(10)"]}
# setup_database.py applies every entry above the database's current version, in order.
UPGRADES = {}


class SetupNeeded(Exception):
    """The database is missing, empty or on another schema version: setup_database.py must be run."""


def schema_statements():
```

```
    """The statements of database/schema.sql for the current engine (without CREATE DATABASE / USE / SET:
    setup_database.py creates and selects the database itself)."""
    with open(SCHEMA_FILE, encoding="utf-8") as f:
        text = "\n".join(re.sub(r"\s*--.*$", "", line) for line in f.read().splitlines())      # drop comments
    out = []
    for stmt in (s.strip() for s in text.split(";")):
        head = stmt[:20].upper()
        if not stmt or head.startswith(("CREATE DATABASE", "USE ", "SET ")):
            continue
        if ENGINE == "sqlite":
            stmt = (stmt.replace("INT AUTO_INCREMENT PRIMARY KEY", "INTEGER PRIMARY KEY AUTOINCREMENT")
                        .replace(") ENGINE=InnoDB DEFAULT CHARSET=utf8mb4", ")")
                        .replace(") ENGINE=InnoDB", ")"))
        out.append(stmt)
    return out


def table_of(stmt):
    """'CREATE TABLE users(...' -> 'users'  (None for other statements)."""
    words = stmt.replace("(", " ( ").split()
    return words[2].lower() if len(words) > 2 and words[0].upper() == "CREATE" and words[1].upper() == "TABLE" else Nor


def existing_tables(c):
    if ENGINE == "sqlite":
        rs = c.query("SELECT name AS n FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%'")
    else:
        rs = c.query("SELECT table_name AS n FROM information_schema.tables WHERE table_schema=DATABASE()")
    return {r["n"].lower() for r in rs}


def current_version(c):
    """Schema version stored in the database, or None when there is no schema_version table (empty / old DB)."""
    if "schema_version" not in existing_tables(c):
        return None
    rs = c.query("SELECT MAX(version) AS v FROM schema_version")
    return rs[0]["v"] if rs and rs[0]["v"] is not None else None


def connect_checked():
    """Open the database for the server. Never creates or changes anything: raises SetupNeeded with a plain
    message when the database is not ready."""
    try:
        c = Conn()
    except Exception as e:
        if ENGINE == "sqlite":
            raise
        text = str(e)
        if "Unknown database" in text or "1049" in text:
            raise SetupNeeded("The database '%s' does not exist yet." % MYSQL_NAME)
        raise
    v = current_version(c)
    if v is None:
        c.close()
        raise SetupNeeded("The database '%s' has not been set up (no schema_version table)." % MYSQL_NAME)
    if v != SCHEMA_VERSION:
        c.close()
        raise SetupNeeded("The database is on schema version %s but this server needs version %s." % (v, SCHEMA_VERSION
    return c
```

LAPTOP2_Server/app.py - Server - REST API

```
"""PawPet server - Flask REST API on top of MySQL.

Run:  py -m pip install -r requirements.txt  ->  py app.py   (API at http://<your-pc-ip>:5000/api)

Safety rules built in (see PAWPET_GUIDE.md):
  * every booking / stock change runs inside WRITE_LOCK, so two requests can never take the same slot or the same stock
  * all input is validated (types, lengths, ranges, dates) and bad input returns a JSON 400, never a crash
  * passwords use PBKDF2 with a per-user salt, logins are rate limited, tokens expire after 30 days
  * accounts that still use a known default password must change it before doing anything else
"""
import hashlib
import hmac
import math
import os
import re
import secrets
import threading
import time
from collections import defaultdict, deque
from datetime import datetime, timedelta, date
from functools import wraps

from flask import Flask, g, jsonify, request
from werkzeug.exceptions import HTTPException
```

```
import db as dbmod
from db import Conn


app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 64 * 1024          # no request body above 64 KB

# One lock for every operation that checks something and then changes it (booking, stock, completing a visit).
# The server runs as ONE process, so this is enough to make those operations safe when many users act together.
WRITE_LOCK = threading.RLock()

TEST_FIXED_PASSWORDS = os.environ.get("PAWPET_TEST_FIXED_PASSWORDS") == "1"   # automated tests only
TOKEN_DAYS = 30
PW_ITERATIONS = 200_000
KNOWN_DEFAULTS = ("admin123", "staff123", "pass123")
MAX_FUTURE_DAYS = 180
MAX_ACTIVE_APPTS_PER_CUSTOMER = 10
MAX_PETS_PER_CUSTOMER = 20
STATUSES = ("pending", "confirmed", "completed", "cancelled", "no_show")
STAFF = ("staff", "admin")


# database

def db():
    if "db" not in g:
        g.db = Conn()
    return g.db


@app.teardown_appcontext
def close_db(_):
    d = g.pop("db", None)
    if d:
        d.close()


def q(sql, args=(), one=False):
    rs = db().query(sql, args)
    return (rs[0] if rs else None) if one else rs


def run(sql, args=()):
    return db().execute(sql, args)


def now():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")


def rows(rs):
    return [dict(r) for r in rs]


def notify(user_id, title, body=""):
    run("INSERT INTO notifications(user_id,title,body,created_at) VALUES(?,?,?,?)",
        (user_id, title[:120], body[:500], now()))


def fresh_read():
    """End the current database transaction so the next SELECT sees everything other requests have already saved.
    (MySQL otherwise keeps showing the data as it was when this request first touched the database.)"""
    db().raw.commit()


def locked(fn):
    @wraps(fn)
    def wrap(*a, **kw):
        with WRITE_LOCK:
            fresh_read()
            return fn(*a, **kw)
    return wrap


# input validation

class ApiError(Exception):
    def __init__(self, message, status=400):
        super().__init__(message)
        self.message, self.status = message, status


@app.errorhandler(ApiError)
def api_error(e):
    return jsonify(error=e.message), e.status
```

```
@app.errorhandler(Exception)
def any_error(e):
    if isinstance(e, HTTPException):
        msg = {413: "That request is too large.", 405: "Method not allowed.", 404: "Not found."}.get(e.code, e.descript
        return jsonify(error=msg), e.code
    app.logger.exception("Unhandled error on %s %s", request.method, request.path)
    return jsonify(error="Something went wrong on the server. Please try again."), 500


def body():
    d = request.get_json(silent=True)
    if not isinstance(d, dict):
        raise ApiError("Please send the form data as a JSON object.")
    return d


def text(d, key, label, maxlen, required=False, default=""):
    v = d.get(key)
    if v is None:
        v = ""
    if isinstance(v, bool) or not isinstance(v, (str, int, float)):
        raise ApiError(f"{label} is not valid.")
    v = str(v).strip()
    if not v:
        if required:
            raise ApiError(f"{label} is required.")
        return default
    if len(v) > maxlen:
        raise ApiError(f"{label} must be at most {maxlen} characters.")
    if any(ord(c) < 32 and c not in "\n\r\t" for c in v):
        raise ApiError(f"{label} has invalid characters.")
    return v


def num(d, key, label, lo=None, hi=None, integer=False, required=True, default=None):
    v = d.get(key)
    if v is None or (isinstance(v, str) and not v.strip()):
        if required:
            raise ApiError(f"{label} is required.")
        return default
    if isinstance(v, (bool, list, dict)):
        raise ApiError(f"{label} must be a number.")
    try:
        x = float(v)
    except (TypeError, ValueError):
        raise ApiError(f"{label} must be a number.")
    if math.isnan(x) or math.isinf(x):
        raise ApiError(f"{label} must be a number.")
    if integer:
        if x != int(x):
            raise ApiError(f"{label} must be a whole number.")
        x = int(x)
    if lo is not None and x < lo:
        raise ApiError(f"{label} must be at least {lo}.")
    if hi is not None and x > hi:
        raise ApiError(f"{label} must be at most {hi}.")
    return x


def valid_iso(s):
    try:
        datetime.strptime(s, "%Y-%m-%d")
        return True
    except (TypeError, ValueError):
        return False


def valid_hhmm(s):
    return isinstance(s, str) and re.fullmatch(r"([01]\d|2[0-3]):[0-5]\d", s) is not None


def appt_date_ok(s):
    if not valid_iso(s):
        raise ApiError("Date must look like 2026-12-31.")
    d = datetime.strptime(s, "%Y-%m-%d").date()
    if d < date.today():
        raise ApiError("You cannot book a date that has already passed.")
    if d > date.today() + timedelta(days=MAX_FUTURE_DAYS):
        raise ApiError(f"Appointments can only be booked up to {MAX_FUTURE_DAYS} days ahead.")
    return s


def work_days_ok(v):
    parts = [p.strip() for p in str(v).split(",") if p.strip() != ""]
```

```
    if not parts or any(p not in "0123456" or len(p) != 1 for p in parts):
        raise ApiError("Working days must be numbers 0-6 (0 = Monday ... 6 = Sunday), separated by commas.")
    return ",".join(sorted(set(parts)))


EMAIL_RE = re.compile(r"^[^@\s]{1,64}@[^@\s]{1,200}\.[^@\s]{2,}$")
PHONE_RE = re.compile(r"^[0-9+()\- ]{0,30}$")
USER_RE = re.compile(r"^[A-Za-z0-9_.\-]{3,50}$")


def contact_fields(d):
    email = text(d, "email", "Email", 100)
    phone = text(d, "phone", "Phone", 30)
    if email and not EMAIL_RE.match(email):
        raise ApiError("Email address is not valid.")
    if phone and not PHONE_RE.match(phone):
        raise ApiError("Phone may only contain digits, spaces and + - ( ).")
    return email, phone


# passwords, tokens, rate limits

def hash_pw(p, salt=None):
    salt = salt or secrets.token_hex(16)
    h = hashlib.pbkdf2_hmac("sha256", p.encode(), bytes.fromhex(salt), PW_ITERATIONS).hex()
    return f"pbkdf2${PW_ITERATIONS}${salt}${h}"


def legacy_hash(p):
    return hashlib.sha256(("pawpet$" + p).encode()).hexdigest()


def check_pw(p, stored):
    try:
        if stored.startswith("pbkdf2$"):
            _, it, salt, h = stored.split("$")
            calc = hashlib.pbkdf2_hmac("sha256", p.encode(), bytes.fromhex(salt), int(it)).hex()
            return hmac.compare_digest(calc, h)
        return hmac.compare_digest(legacy_hash(p), stored)
    except Exception:
        return False


DUMMY_HASH = hash_pw("not-a-real-password")


def password_rules(p, username=""):
    if not isinstance(p, str) or len(p) < 8:
        raise ApiError("Password must be at least 8 characters.")
    if len(p) > 128:
        raise ApiError("Password must be at most 128 characters.")
    if not (re.search(r"[A-Za-z]", p) and re.search(r"\d", p)):
        raise ApiError("Password needs at least one letter and one number.")
    if p in KNOWN_DEFAULTS or (username and p.lower() == username.lower()):
        raise ApiError("Choose a password that is not a common or default one.")


def temp_password():
    letters, digits = "abcdefghjkmnpqrstuvwxyz", "23456789"
    pw = [secrets.choice(letters.upper()), secrets.choice(letters), secrets.choice(digits)]
    pw += [secrets.choice(letters + digits) for _ in range(7)]
    secrets.SystemRandom().shuffle(pw)
    return "".join(pw)


_hits = defaultdict(deque)
_hits_lock = threading.Lock()


def rate_limited(key, limit, window, record=False):
    """True when `key` already used up `limit` hits in the last `window` seconds."""
    t = time.time()
    with _hits_lock:
        if len(_hits) > 5000:                                   # forget old keys so memory cannot grow without limit
            for k in [k for k, v in _hits.items() if not v or v[-1] < t - 3600]:
                del _hits[k]
        dq = _hits[key]
        while dq and dq[0] < t - window:
            dq.popleft()
        if len(dq) >= limit:
            return True
        if record:
            dq.append(t)
        return False
```

```
def clear_hits(key):
    with _hits_lock:
        _hits.pop(key, None)


def client_ip():
    return request.remote_addr or "?"


def token_cutoff():
    return (datetime.now() - timedelta(days=TOKEN_DAYS)).strftime("%Y-%m-%d %H:%M:%S")


def current_user():
    h = request.headers.get("Authorization", "")
    tok = h.replace("Bearer ", "").strip()
    if not tok or len(tok) > 64:
        return None
    return q("""SELECT u.* FROM tokens t JOIN users u ON u.id=t.user_id
                WHERE t.token=? AND u.active=1 AND t.created_at>=?""", (tok, token_cutoff()), one=True)


ALLOWED_WHEN_MUST_CHANGE = ("change_password", "logout", "me")


def auth(*roles):
    def deco(fn):
        @wraps(fn)
        def wrap(*a, **kw):
            u = current_user()
            if not u:
                return jsonify(error="Please sign in again."), 401
            if u["must_change"] and request.endpoint not in ALLOWED_WHEN_MUST_CHANGE:
                return jsonify(error="Please change your temporary password first.", must_change=True), 403
            if roles and u["role"] not in roles:
                return jsonify(error="Not allowed."), 403
            g.user = u
            return fn(*a, **kw)
        return wrap
    return deco


def is_staff_request():
    u = current_user()
    return bool(u and u["role"] in STAFF and not u["must_change"])


def user_json(u):
    d = {k: u[k] for k in ("id", "username", "full_name", "email", "phone", "role")}
    d["must_change"] = bool(u["must_change"])
    return d


# startup checks (tables are made by setup_database.py)

def flag_default_passwords(c):
    """Any account that still uses admin123 / staff123 / pass123 must change it at its next sign-in."""
    if TEST_FIXED_PASSWORDS:
        return
    for u in c.query("SELECT id,password FROM users WHERE must_change=0"):
        if any(check_pw(p, u["password"]) for p in KNOWN_DEFAULTS):
            c.execute("UPDATE users SET must_change=1 WHERE id=?", (u["id"],))


# batches / expiry helpers

EXPIRY_WARN_DAYS = 30


def today_str():
    return date.today().strftime("%Y-%m-%d")


def batch_info():
    """product_id -> {'expired': qty already expired, 'next': earliest upcoming expiry}"""
    t = today_str()
    info = {}
    for b in q("SELECT product_id, qty, expiry_date FROM batches WHERE qty>0"):
        i = info.setdefault(b["product_id"], {"expired": 0, "next": None})
        if b["expiry_date"]:
            if b["expiry_date"] < t:
                i["expired"] += b["qty"]
            elif i["next"] is None or b["expiry_date"] < i["next"]:
                i["next"] = b["expiry_date"]
    return info
```

```
def product_view(p, info, staff=True):
    """`stock` is what can really be sold (expired batches are not counted)."""
    p = dict(p)
    i = info.get(p["id"], {"expired": 0, "next": None})
    physical = p["stock"]
    p["stock"] = max(physical - i["expired"], 0)
    if staff:                                   # customers do not need the internal numbers
        p["physical_stock"] = physical
        p["expired_qty"] = i["expired"]
        p["next_expiry"] = i["next"]
    return p


def usable_stock(pid):
    p = q("SELECT stock FROM products WHERE id=?", (pid,), one=True)
    if not p:
        return 0
    return max(p["stock"] - batch_info().get(pid, {"expired": 0})["expired"], 0)


def add_batch(pid, qty, batch_no="", expiry=None):
    run("INSERT INTO batches(product_id,batch_no,qty,expiry_date,received_at) VALUES(?,?,?,?,?)",
        (pid, (batch_no or "")[:40], qty, expiry or None, now()))
    run("UPDATE products SET stock=stock+? WHERE id=?", (qty, pid))


def deduct_stock(pid, qty, include_expired=False):
    """Take stock from the batches that expire first (FEFO). Callers check the stock first, under WRITE_LOCK."""
    t = today_str()
    left = qty
    sql = "SELECT * FROM batches WHERE product_id=? AND qty>0"
    args = [pid]
    if not include_expired:
        sql += " AND (expiry_date IS NULL OR expiry_date >= ?)"
        args.append(t)
    sql += " ORDER BY CASE WHEN expiry_date IS NULL THEN 1 ELSE 0 END, expiry_date, id"
    for b in q(sql, args):
        if left <= 0:
            break
        take = min(b["qty"], left)
        run("UPDATE batches SET qty=qty-? WHERE id=?", (take, b["id"]))
        left -= take
    run("UPDATE products SET stock=stock-? WHERE id=?", (qty, pid))


def batch_status(expiry):
    if not expiry:
        return "no_expiry", None
    days = (datetime.strptime(expiry, "%Y-%m-%d").date() - date.today()).days
    if days < 0:
        return "expired", days
    if days <= EXPIRY_WARN_DAYS:
        return "expiring", days
    return "ok", days


def batch_rows(only_active=True, product_id=None):
    sql = """SELECT b.id, b.product_id, p.name AS product_name, p.category, b.batch_no, b.qty,
                    b.expiry_date, b.received_at
             FROM batches b JOIN products p ON p.id=b.product_id WHERE 1=1"""
    args = []
    if only_active:
        sql += " AND b.qty>0"
    if product_id:
        sql += " AND b.product_id=?"
        args.append(product_id)
    out = []
    for b in q(sql + " ORDER BY CASE WHEN b.expiry_date IS NULL THEN 1 ELSE 0 END, b.expiry_date, b.id", args):
        b = dict(b)
        b["status"], b["days_left"] = batch_status(b["expiry_date"])
        out.append(b)
    return out


# auth endpoints

@app.post("/api/register")
def register():
    ip = "reg:" + client_ip()
    if rate_limited(ip, 20, 3600, record=True):
        raise ApiError("Too many sign-ups from this device. Please try again later.", 429)
    d = body()
    username = text(d, "username", "Username", 50, required=True)
    if not USER_RE.match(username):
        raise ApiError("Username must be 3-50 letters, numbers, dot, dash or underscore.")
```

```
    full_name = text(d, "full_name", "Full name", 100, required=True)
    email, phone = contact_fields(d)
    pw = d.get("password")
    password_rules(pw, username)
    if q("SELECT 1 FROM users WHERE LOWER(username)=?", (username.lower(),), one=True):
        raise ApiError("Username is already taken.", 409)
    with WRITE_LOCK:
        fresh_read()
        if q("SELECT 1 FROM users WHERE LOWER(username)=?", (username.lower(),), one=True):
            raise ApiError("Username is already taken.", 409)
        uid = run("INSERT INTO users(username,password,full_name,email,phone,role,created_at) VALUES(?,?,?,?,?,'custome
                  (username, hash_pw(pw), full_name, email, phone, now()))
    notify(uid, "Welcome to PawPet", "Add your pet and book your first visit.")
    return jsonify(ok=True), 201


@app.post("/api/login")
def login():
    d = body()
    username = str(d.get("username", "")).strip()[:50]
    password = d.get("password", "")
    password = password if isinstance(password, str) else ""
    ukey, ikey = "login-u:" + username.lower(), "login-ip:" + client_ip()
    if rate_limited(ukey, 5, 600) or rate_limited(ikey, 60, 600):
        return jsonify(error="Too many failed attempts. Please wait 10 minutes and try again."), 429
    u = q("SELECT * FROM users WHERE LOWER(username)=? AND active=1", (username.lower(),), one=True)
    ok = check_pw(password[:128], u["password"] if u else DUMMY_HASH) and u is not None
    if not ok:
        rate_limited(ukey, 5, 600, record=True)
        rate_limited(ikey, 60, 600, record=True)
        return jsonify(error="Wrong username or password."), 401
    clear_hits(ukey)
    role_wanted = d.get("portal")  # 'admin' portal only for staff/admin
    if role_wanted == "admin" and u["role"] not in STAFF:
        return jsonify(error="This portal is for clinic staff only."), 403
    if not u["password"].startswith("pbkdf2$"):                  # upgrade an old SHA-256 hash
        run("UPDATE users SET password=? WHERE id=?", (hash_pw(password), u["id"]))
    if not TEST_FIXED_PASSWORDS and password in KNOWN_DEFAULTS and not u["must_change"]:
        run("UPDATE users SET must_change=1 WHERE id=?", (u["id"],))
    u = q("SELECT * FROM users WHERE id=?", (u["id"],), one=True)
    run("DELETE FROM tokens WHERE created_at<?", (token_cutoff(),))
    tok = secrets.token_hex(24)
    run("INSERT INTO tokens(token,user_id,created_at) VALUES(?,?,?)", (tok, u["id"], now()))
    return jsonify(token=tok, user=user_json(u))


@app.post("/api/logout")
@auth()
def logout():
    tok = request.headers.get("Authorization", "").replace("Bearer ", "").strip()
    run("DELETE FROM tokens WHERE token=?", (tok,))
    return jsonify(ok=True)


@app.get("/api/me")
@auth()
def me():
    return jsonify(user_json(g.user))


@app.post("/api/change-password")
@auth()
def change_password():
    d = body()
    old, new = d.get("old_password", ""), d.get("new_password", "")
    if not isinstance(old, str) or not check_pw(old[:128], g.user["password"]):
        if rate_limited("chg:" + str(g.user["id"]), 5, 600, record=True):
            raise ApiError("Too many attempts. Please wait 10 minutes.", 429)
        raise ApiError("Your current password is not correct.", 403)
    password_rules(new, g.user["username"])
    if new == old:
        raise ApiError("The new password must be different from the old one.")
    tok = request.headers.get("Authorization", "").replace("Bearer ", "").strip()
    run("UPDATE users SET password=?, must_change=0 WHERE id=?", (hash_pw(new), g.user["id"]))
    run("DELETE FROM tokens WHERE user_id=? AND token<>?", (g.user["id"], tok))     # sign out other devices
    return jsonify(ok=True)


@app.post("/api/users/<int:uid>/reset-password")
@auth(*STAFF)
def reset_password(uid):
    """Staff can reset a customer's password; an admin can reset anybody's. Returns a one-time temporary password."""
    u = q("SELECT * FROM users WHERE id=?", (uid,), one=True)
    if not u:
        raise ApiError("User not found.", 404)
    if u["role"] != "customer" and g.user["role"] != "admin":
```

```
        raise ApiError("Only an admin can reset staff accounts.", 403)
    if u["id"] == g.user["id"]:
        raise ApiError("Use Change password for your own account.")
    tmp = temp_password()
    run("UPDATE users SET password=?, must_change=1 WHERE id=?", (hash_pw(tmp), uid))
    run("DELETE FROM tokens WHERE user_id=?", (uid,))
    return jsonify(temp_password=tmp, username=u["username"])


# catalog (public-ish)

@app.get("/api/products")
def products():
    cat = text(request.args, "category", "Category", 40)
    s = text(request.args, "q", "Search", 100)
    sql, args = "SELECT * FROM products WHERE 1=1", []
    if cat and cat != "All":
        sql += " AND category=?"
        args.append(cat)
    if s:
        sql += " AND name LIKE ?"
        args.append(f"%{s}%")
    info = batch_info()
    staff = is_staff_request()
    return jsonify([product_view(p, info, staff) for p in q(sql + " ORDER BY name", args)])


@app.get("/api/services")
def services():
    return jsonify(rows(q("SELECT * FROM services ORDER BY name")))


@app.get("/api/vets")
def vets():
    return jsonify(rows(q("SELECT * FROM vets WHERE active=1 ORDER BY name")))


def slot_window(vet):
    """(start, end) datetimes of a vet's day, or None when the vet record is unusable."""
    try:
        start = datetime.strptime(vet["start_time"], "%H:%M")
        end = datetime.strptime(vet["end_time"], "%H:%M")
    except (ValueError, TypeError):
        return None
    return (start, end) if end > start else None


def busy_blocks(vet_id, the_date, exclude_id=None):
    out = []
    for b in q("""SELECT a.id, a.appt_time, s.duration FROM appointments a JOIN services s ON s.id=a.service_id
                  WHERE a.vet_id=? AND a.appt_date=? AND a.status IN ('pending','confirmed')""", (vet_id, the_date)):
        if exclude_id and b["id"] == exclude_id:
            continue
        try:
            bs = datetime.strptime(b["appt_time"], "%H:%M")
        except ValueError:
            continue
        out.append((b["id"], bs, bs + timedelta(minutes=max(b["duration"], 30))))
    return out


def slots_for(vet, service, the_date):
    """Return list of {time, available} for a vet on a date. Past dates and non-working days give no slots."""
    try:
        d = datetime.strptime(the_date, "%Y-%m-%d").date()
    except (ValueError, TypeError):
        return []
    win = slot_window(vet)
    if win is None or d < date.today():
        return []
    if str(d.weekday()) not in [x.strip() for x in str(vet["work_days"]).split(",")]:
        return []
    start, end = win
    step = 30
    dur = max(service["duration"], 30) if service else 30
    busy = [(bs, be) for _, bs, be in busy_blocks(vet["id"], the_date)]
    out = []
    cur = start
    while cur + timedelta(minutes=dur) <= end:
        ce = cur + timedelta(minutes=dur)
        free = all(ce <= bs or cur >= be for bs, be in busy)
        if d == date.today() and cur <= datetime.now().replace(year=1900, month=1, day=1):
            free = False
        out.append({"time": cur.strftime("%H:%M"), "available": free})
        cur += timedelta(minutes=step)
    return out
```

```
def overlapping(vet_id, the_date, the_time, duration, exclude_id=None, only_older_than=None):
    """ids of active appointments that overlap [the_time, the_time+duration) for this vet."""
    cs = datetime.strptime(the_time, "%H:%M")
    ce = cs + timedelta(minutes=max(duration, 30))
    hits = []
    for bid, bs, be in busy_blocks(vet_id, the_date, exclude_id):
        if only_older_than and bid >= only_older_than:
            continue
        if not (ce <= bs or cs >= be):
            hits.append(bid)
    return hits


@app.get("/api/availability")
def availability():
    vid = num(request.args, "vet_id", "Veterinarian", 1, integer=True)
    the_date = text(request.args, "date", "Date", 10, required=True)
    if not valid_iso(the_date):
        raise ApiError("Date must look like 2026-12-31.")
    vet = q("SELECT * FROM vets WHERE id=?", (vid,), one=True)
    if not vet:
        raise ApiError("Vet not found.", 404)
    sid = num(request.args, "service_id", "Service", 1, integer=True, required=False)
    svc = q("SELECT * FROM services WHERE id=?", (sid,), one=True) if sid else None
    return jsonify(slots_for(vet, svc, the_date))


# customer: pets & appointments

@app.get("/api/pets")
@auth()
def my_pets():
    if g.user["role"] in STAFF and request.args.get("all"):
        return jsonify(rows(q("""SELECT p.*, u.full_name AS owner_name FROM pets p
                                 JOIN users u ON u.id=p.owner_id ORDER BY p.name""")))
    return jsonify(rows(q("SELECT * FROM pets WHERE owner_id=? ORDER BY name", (g.user["id"],))))


@app.post("/api/pets")
@auth()
@locked
def add_pet():
    d = body()
    name = text(d, "name", "Pet name", 60, required=True)
    species = text(d, "species", "Species", 30, required=True)
    breed = text(d, "breed", "Breed", 60)
    age = num(d, "age", "Age", 0, 60, required=False)
    weight = num(d, "weight", "Weight", 0, 500, required=False)
    notes = text(d, "notes", "Notes", 255)
    owner = g.user["id"]
    if g.user["role"] in STAFF and d.get("owner_id"):
        owner = num(d, "owner_id", "Owner", 1, integer=True)
        if not q("SELECT 1 FROM users WHERE id=? AND role='customer'", (owner,), one=True):
            raise ApiError("Customer not found.", 404)
    if q("SELECT COUNT(*) AS n FROM pets WHERE owner_id=?", (owner,), one=True)["n"] >= MAX_PETS_PER_CUSTOMER:
        raise ApiError(f"A customer can have at most {MAX_PETS_PER_CUSTOMER} pets.", 409)
    pid = run("INSERT INTO pets(owner_id,name,species,breed,age,weight,notes) VALUES(?,?,?,?,?,?,?)",
              (owner, name, species, breed, age, weight, notes))
    return jsonify(id=pid), 201


@app.delete("/api/pets/<int:pid>")
@auth()
@locked
def del_pet(pid):
    p = q("SELECT * FROM pets WHERE id=?", (pid,), one=True)
    if not p or (p["owner_id"] != g.user["id"] and g.user["role"] not in STAFF):
        return jsonify(error="Pet not found."), 404
    if q("SELECT 1 FROM appointments WHERE pet_id=? AND status IN ('pending','confirmed')", (pid,), one=True):
        return jsonify(error="This pet has an active appointment."), 409
    if q("SELECT 1 FROM appointments WHERE pet_id=?", (pid,), one=True):
        return jsonify(error="This pet has visit history, so it cannot be removed."), 409
    run("DELETE FROM pets WHERE id=?", (pid,))
    return jsonify(ok=True)


APPT_SQL = """
SELECT a.*, u.full_name AS customer_name, u.phone AS customer_phone,
       p.name AS pet_name, p.species, p.breed, p.age AS pet_age, p.notes AS pet_notes,
       s.name AS service_name, s.price AS service_price, s.duration, s.emoji AS service_emoji,
       v.name AS vet_name
FROM appointments a
JOIN users u ON u.id=a.customer_id
JOIN pets p ON p.id=a.pet_id
```

```
JOIN services s ON s.id=a.service_id
JOIN vets v ON v.id=a.vet_id
"""


@app.get("/api/appointments")
@auth()
def list_appts():
    sql, args = APPT_SQL + " WHERE 1=1", []
    if g.user["role"] not in STAFF:
        sql += " AND a.customer_id=?"
        args.append(g.user["id"])
    the_date = text(request.args, "date", "Date", 10)
    if the_date:
        if not valid_iso(the_date):
            raise ApiError("Date must look like 2026-12-31.")
        sql += " AND a.appt_date=?"
        args.append(the_date)
    st = text(request.args, "status", "Status", 20)
    if st:
        if st not in STATUSES:
            raise ApiError("Unknown status.")
        sql += " AND a.status=?"
        args.append(st)
    vid = num(request.args, "vet_id", "Veterinarian", 1, integer=True, required=False)
    if vid:
        sql += " AND a.vet_id=?"
        args.append(vid)
    return jsonify(rows(q(sql + " ORDER BY a.appt_date, a.appt_time", args)))


def book_slot(cust, pet, vet, svc, the_date, the_time, notes, status, followup_of=None):
    """Check the slot and insert, under WRITE_LOCK. A second check after the insert protects even if two
    server processes were ever started by mistake (the newer booking gives way)."""
    with WRITE_LOCK:
        fresh_read()
        ok = [s for s in slots_for(vet, svc, the_date) if s["time"] == the_time and s["available"]]
        if not ok or overlapping(vet["id"], the_date, the_time, svc["duration"]):
            raise ApiError("That time is no longer available. Please pick another slot.", 409)
        aid = run("""INSERT INTO appointments(customer_id,pet_id,service_id,vet_id,appt_date,appt_time,status,notes,fol
                     VALUES(?,?,?,?,?,?,?,?,?,?)""",
                  (cust, pet, svc["id"], vet["id"], the_date, the_time, status, notes, followup_of, now()))
        if overlapping(vet["id"], the_date, the_time, svc["duration"], exclude_id=aid, only_older_than=aid):
            run("DELETE FROM appointments WHERE id=?", (aid,))
            raise ApiError("That time was just taken by someone else. Please pick another slot.", 409)
        return aid


@app.post("/api/appointments")
@auth()
def book():
    d = body()
    pet_id = num(d, "pet_id", "Pet", 1, integer=True)
    service_id = num(d, "service_id", "Service", 1, integer=True)
    vet_id = num(d, "vet_id", "Veterinarian", 1, integer=True)
    the_date = appt_date_ok(text(d, "appt_date", "Date", 10, required=True))
    the_time = text(d, "appt_time", "Time", 5, required=True)
    if not valid_hhmm(the_time):
        raise ApiError("Time must look like 09:30.")
    notes = text(d, "notes", "Notes", 500)
    cust = g.user["id"]
    if g.user["role"] in STAFF and d.get("customer_id"):
        cust = num(d, "customer_id", "Customer", 1, integer=True)
        if not q("SELECT 1 FROM users WHERE id=? AND role='customer' AND active=1", (cust,), one=True):
            raise ApiError("Customer not found.", 404)
    pet = q("SELECT * FROM pets WHERE id=? AND owner_id=?", (pet_id, cust), one=True)
    if not pet:
        return jsonify(error="Pet not found for this customer."), 404
    vet = q("SELECT * FROM vets WHERE id=? AND active=1", (vet_id,), one=True)
    svc = q("SELECT * FROM services WHERE id=?", (service_id,), one=True)
    if not vet or not svc:
        return jsonify(error="Vet or service not found."), 404
    if g.user["role"] not in STAFF:
        n = q("""SELECT COUNT(*) AS n FROM appointments WHERE customer_id=? AND status IN ('pending','confirmed')
                 AND appt_date>=?""", (cust, today_str()), one=True)["n"]
        if n >= MAX_ACTIVE_APPTS_PER_CUSTOMER:
            raise ApiError(f"You already have {n} upcoming appointments. Please cancel one or wait for the visit.", 409
    aid = book_slot(cust, pet["id"], vet, svc, the_date, the_time, notes,
                    "confirmed" if g.user["role"] in STAFF else "pending")
    notify(cust, "Appointment booked",
           f"{pet['name']} - {svc['name']} with {vet['name']} on {the_date} at {the_time}.")
    return jsonify(id=aid), 201


@app.post("/api/appointments/<int:aid>/cancel")
@auth()
```

```
@locked
def cancel(aid):
    a = q("SELECT * FROM appointments WHERE id=?", (aid,), one=True)
    if not a or (a["customer_id"] != g.user["id"] and g.user["role"] not in STAFF):
        return jsonify(error="Appointment not found."), 404
    if a["status"] not in ("pending", "confirmed"):
        return jsonify(error="This appointment can no longer be cancelled."), 409
    run("UPDATE appointments SET status='cancelled' WHERE id=?", (aid,))
    notify(a["customer_id"], "Appointment cancelled", f"Appointment #{aid} was cancelled.")
    return jsonify(ok=True)


# staff: appointment workflow

# what a staff member may change an appointment to (finished visits use /complete so the sale is recorded)
TRANSITIONS = {"pending": ("confirmed", "cancelled", "no_show"),
               "confirmed": ("cancelled", "no_show")}


@app.post("/api/appointments/<int:aid>/status")
@auth(*STAFF)
@locked
def set_status(aid):
    d = body()
    st = d.get("status")
    if st not in STATUSES:
        return jsonify(error="Invalid status."), 400
    notes = text(d, "notes", "Notes", 500) if d.get("notes") is not None else None
    a = q("SELECT * FROM appointments WHERE id=?", (aid,), one=True)
    if not a:
        return jsonify(error="Appointment not found."), 404
    if st == "completed":
        return jsonify(error="Use Complete visit so the sale is recorded."), 400
    if st not in TRANSITIONS.get(a["status"], ()):
        return jsonify(error=f"A {a['status'].replace('_', ' ')} appointment cannot be changed to {st.replace('_', ' ')
    run("UPDATE appointments SET status=?, notes=COALESCE(?,notes) WHERE id=?", (st, notes, aid))
    msg = {"confirmed": "Your appointment is confirmed.",
           "cancelled": "Your appointment was cancelled by the clinic.",
           "no_show": "You missed your appointment. Please rebook."}.get(st)
    if msg:
        notify(a["customer_id"], f"Appointment {st.replace('_', ' ')}", msg)
    return jsonify(ok=True)


@app.post("/api/appointments/<int:aid>/complete")
@auth(*STAFF)
@locked
def complete(aid):
    """Record the completed service (+ optional products used) as sales. Everything is checked BEFORE anything is saved
    d = request.get_json(silent=True) or {}
    if not isinstance(d, dict):
        raise ApiError("Please send the form data as a JSON object.")
    a = q(APPT_SQL + " WHERE a.id=?", (aid,), one=True)
    if not a:
        return jsonify(error="Appointment not found."), 404
    if a["status"] == "completed":
        return jsonify(error="Already completed."), 409
    if a["status"] not in ("pending", "confirmed"):
        return jsonify(error=f"A {a['status'].replace('_', ' ')} appointment cannot be completed."), 409
    notes = text(d, "notes", "Notes", 500) if d.get("notes") is not None else None
    items = d.get("products") or []
    if not isinstance(items, list) or len(items) > 50:
        raise ApiError("Products must be a list of up to 50 items.")
    wanted = {}
    for it in items:
        if not isinstance(it, dict):
            raise ApiError("Each product needs a product_id and qty.")
        pid = num(it, "product_id", "Product", 1, integer=True)
        qty = num(it, "qty", "Quantity", 1, 10000, integer=True, required=False, default=1)
        wanted[pid] = wanted.get(pid, 0) + qty
    prods = {}
    for pid, qty in wanted.items():
        p = q("SELECT * FROM products WHERE id=?", (pid,), one=True)
        if not p:
            raise ApiError(f"Product #{pid} was not found.", 404)
        if usable_stock(pid) < qty:
            return jsonify(error=f"Not enough usable stock for {p['name']} (expired items are not counted). "
                                 "Nothing was saved."), 409
        prods[pid] = p
    # all checks passed: now save
    run("UPDATE appointments SET status='completed', notes=COALESCE(?,notes) WHERE id=?", (notes, aid))
    run("INSERT INTO sales(customer_id,appointment_id,kind,item_name,qty,amount,created_at) VALUES(?,?,?,?,?,?,?)",
        (a["customer_id"], aid, "service", a["service_name"], 1, a["service_price"], now()))
    total = a["service_price"]
    for pid, qty in wanted.items():
        p = prods[pid]
```

```
        deduct_stock(pid, qty)
        amount = round(p["price"] * qty, 2)
        run("INSERT INTO sales(customer_id,appointment_id,kind,item_name,qty,amount,created_at) VALUES(?,?,?,?,?,?,?)",
            (a["customer_id"], aid, "product", p["name"], qty, amount, now()))
        total += amount
    notify(a["customer_id"], "Visit completed", f"{a['pet_name']}'s visit is done. Total: PHP {total:,.2f}.")
    return jsonify(ok=True, total=round(total, 2))


@app.post("/api/appointments/<int:aid>/followup")
@auth(*STAFF)
def followup(aid):
    d = body()
    the_date = appt_date_ok(text(d, "appt_date", "Date", 10, required=True))
    the_time = text(d, "appt_time", "Time", 5, required=True)
    if not valid_hhmm(the_time):
        raise ApiError("Time must look like 09:30.")
    notes = text(d, "notes", "Notes", 500, default="Follow-up visit")
    a = q("SELECT * FROM appointments WHERE id=?", (aid,), one=True)
    if not a:
        return jsonify(error="Appointment not found."), 404
    if a["status"] != "completed":
        return jsonify(error="Follow-ups are scheduled after a visit is completed."), 409
    vet = q("SELECT * FROM vets WHERE id=?", (a["vet_id"],), one=True)
    svc = q("SELECT * FROM services WHERE id=?", (a["service_id"],), one=True)
    try:
        nid = book_slot(a["customer_id"], a["pet_id"], vet, svc, the_date, the_time, notes, "confirmed", followup_of=ai
    except ApiError as e:
        if e.status == 409:
            raise ApiError("That follow-up slot is not available.", 409)
        raise
    notify(a["customer_id"], "Follow-up scheduled", f"Follow-up on {the_date} at {the_time}.")
    return jsonify(id=nid), 201


@app.post("/api/reminders/send")
@auth(*STAFF)
def send_reminders():
    tomorrow = (date.today() + timedelta(days=1)).strftime("%Y-%m-%d")
    today = today_str()
    n = 0
    for a in q(APPT_SQL + " WHERE a.appt_date IN (?,?) AND a.status IN ('pending','confirmed')", (today, tomorrow)):
        when = "today" if a["appt_date"] == today else "tomorrow"
        msg = f"{a['pet_name']} has {a['service_name']} {when} at {a['appt_time']} with {a['vet_name']}."
        if q("""SELECT 1 FROM notifications WHERE user_id=? AND title='Appointment reminder' AND body=?
                AND substr(created_at,1,10)=?""", (a["customer_id"], msg, today), one=True):
            continue                          # already reminded today: no duplicates
        notify(a["customer_id"], "Appointment reminder", msg)
        n += 1
    return jsonify(sent=n)


# staff: customers, inventory, vets

@app.get("/api/customers")
@auth(*STAFF)
def customers():
    s = text(request.args, "q", "Search", 100)
    sql = """SELECT u.id,u.username,u.full_name,u.email,u.phone,u.active,u.created_at,
             (SELECT COUNT(*) FROM pets p WHERE p.owner_id=u.id) AS pet_count,
             (SELECT COUNT(*) FROM appointments a WHERE a.customer_id=u.id) AS appt_count
             FROM users u WHERE u.role='customer'"""
    args = []
    if s:
        sql += " AND (u.full_name LIKE ? OR u.username LIKE ? OR u.phone LIKE ?)"
        args = [f"%{s}%"] * 3
    return jsonify(rows(q(sql + " ORDER BY u.full_name", args)))


@app.get("/api/customers/<int:cid>")
@auth(*STAFF)
def customer_detail(cid):
    u = q("SELECT id,username,full_name,email,phone,active,created_at FROM users WHERE id=? AND role='customer'",
          (cid,), one=True)
    if not u:
        return jsonify(error="Customer not found."), 404
    return jsonify(customer=dict(u),
                   pets=rows(q("SELECT * FROM pets WHERE owner_id=?", (cid,))),
                   appointments=rows(q(APPT_SQL + " WHERE a.customer_id=? ORDER BY a.appt_date DESC", (cid,))))


@app.post("/api/customers")
@auth(*STAFF)
@locked
def add_customer():
    d = body()
```

```
    username = text(d, "username", "Username", 50, required=True)
    if not USER_RE.match(username):
        raise ApiError("Username must be 3-50 letters, numbers, dot, dash or underscore.")
    full_name = text(d, "full_name", "Full name", 100, required=True)
    email, phone = contact_fields(d)
    if q("SELECT 1 FROM users WHERE LOWER(username)=?", (username.lower(),), one=True):
        raise ApiError("Username is already taken.", 409)
    pw = d.get("password")
    given = isinstance(pw, str) and pw != ""
    if given:
        password_rules(pw, username)
    else:
        pw = temp_password()
    uid = run("""INSERT INTO users(username,password,full_name,email,phone,role,must_change,created_at)
                 VALUES(?,?,?,?,?,'customer',1,?)""", (username, hash_pw(pw), full_name, email, phone, now()))
    out = {"id": uid}
    if not given:
        out["temp_password"] = pw          # shown once to the staff member; the customer must change it
    return jsonify(out), 201


@app.post("/api/customers/<int:cid>/toggle")
@auth("admin")
def toggle_customer(cid):
    run("UPDATE users SET active=1-active WHERE id=? AND role='customer'", (cid,))
    run("DELETE FROM tokens WHERE user_id=?", (cid,))
    return jsonify(ok=True)


@app.post("/api/products")
@auth(*STAFF)
@locked
def add_product():
    d = body()
    name = text(d, "name", "Product name", 100, required=True)
    category = text(d, "category", "Category", 40, default="Other")
    price = round(num(d, "price", "Price", 0, 1_000_000), 2)
    stock = num(d, "stock", "Stock", 0, 100_000, integer=True, required=False, default=0)
    reorder = num(d, "reorder_level", "Reorder level", 0, 100_000, integer=True, required=False, default=5)
    emoji = text(d, "emoji", "Emoji", 8, default="?")
    batch_no = text(d, "batch_no", "Batch number", 40, default="INITIAL")
    expiry = text(d, "expiry_date", "Expiry date", 10) or None
    if expiry and not valid_iso(expiry):
        raise ApiError("Expiry date must look like YYYY-MM-DD.")
    pid = run("INSERT INTO products(name,category,price,stock,reorder_level,emoji) VALUES(?,?,?,?,?,?)",
              (name, category, price, 0, reorder, emoji))
    if stock > 0:
        add_batch(pid, stock, batch_no, expiry)
    return jsonify(id=pid), 201


@app.put("/api/products/<int:pid>")
@auth(*STAFF)
@locked
def edit_product(pid):
    d = body()
    p = q("SELECT * FROM products WHERE id=?", (pid,), one=True)
    if not p:
        return jsonify(error="Product not found."), 404
    name = text(d, "name", "Product name", 100, default=p["name"]) if "name" in d else p["name"]
    category = text(d, "category", "Category", 40, default=p["category"]) if "category" in d else p["category"]
    price = round(num(d, "price", "Price", 0, 1_000_000, required=False, default=p["price"]), 2)
    reorder = num(d, "reorder_level", "Reorder level", 0, 100_000, integer=True, required=False, default=p["reorder_lev
    usable = usable_stock(pid)
    stock = num(d, "stock", "Stock", 0, 100_000, integer=True, required=False, default=usable)   # shown to staff = usa
    run("UPDATE products SET name=?,category=?,price=?,reorder_level=? WHERE id=?", (name, category, price, reorder, pi
    diff = stock - usable              # stock edits keep the batch list in step
    if diff > 0:
        add_batch(pid, diff, "ADJUST", None)
    elif diff < 0:
        deduct_stock(pid, -diff)
    return jsonify(ok=True)


@app.post("/api/products/<int:pid>/adjust")
@auth(*STAFF)
@locked
def adjust_stock(pid):
    d = body()
    p = q("SELECT * FROM products WHERE id=?", (pid,), one=True)
    if not p:
        return jsonify(error="Product not found."), 404
    delta = num(d, "delta", "Change", -100_000, 100_000, integer=True, required=False, default=0)
    if delta < 0 and usable_stock(pid) + delta < 0:
        return jsonify(error="Stock cannot go below zero."), 409
    if delta > 0:
```

```
        add_batch(pid, delta, "ADJUST", None)
    elif delta < 0:
        deduct_stock(pid, -delta)
    return jsonify(stock=usable_stock(pid))


@app.post("/api/products/<int:pid>/restock")
@auth(*STAFF)
@locked
def restock(pid):
    """Receive a new batch (quantity, batch number, expiry date)."""
    d = body()
    if not q("SELECT 1 AS x FROM products WHERE id=?", (pid,), one=True):
        return jsonify(error="Product not found."), 404
    qty = num(d, "qty", "Quantity", 1, 100_000, integer=True)
    expiry = text(d, "expiry_date", "Expiry date", 10) or None
    if expiry and not valid_iso(expiry):
        raise ApiError("Expiry date must look like YYYY-MM-DD.")
    if expiry and expiry < today_str():
        raise ApiError("That expiry date is already in the past.")
    add_batch(pid, qty, text(d, "batch_no", "Batch number", 40) or "BATCH", expiry)
    return jsonify(stock=usable_stock(pid)), 201


@app.get("/api/batches")
@auth(*STAFF)
def batches():
    return jsonify(batch_rows(only_active=not request.args.get("all"),
                              product_id=request.args.get("product_id", type=int)))


@app.post("/api/batches/<int:bid>/discard")
@auth(*STAFF)
@locked
def discard_batch(bid):
    """Write off what is left of a batch (for example expired medicine thrown away)."""
    b = q("SELECT * FROM batches WHERE id=?", (bid,), one=True)
    if not b:
        return jsonify(error="Batch not found."), 404
    if b["qty"] <= 0:
        return jsonify(error="This batch is already empty."), 409
    run("UPDATE batches SET qty=0 WHERE id=?", (bid,))
    run("UPDATE products SET stock=stock-? WHERE id=?", (b["qty"], b["product_id"]))
    return jsonify(ok=True, discarded=b["qty"])


@app.delete("/api/products/<int:pid>")
@auth("admin")
@locked
def del_product(pid):
    run("DELETE FROM products WHERE id=?", (pid,))
    return jsonify(ok=True)


@app.post("/api/products/<int:pid>/sell")
@auth(*STAFF)
@locked
def sell_product(pid):
    d = body()
    p = q("SELECT * FROM products WHERE id=?", (pid,), one=True)
    if not p:
        return jsonify(error="Product not found."), 404
    qty = num(d, "qty", "Quantity", 1, 10_000, integer=True, required=False, default=1)
    cust = None
    if d.get("customer_id"):
        cust = num(d, "customer_id", "Customer", 1, integer=True)
        if not q("SELECT 1 FROM users WHERE id=? AND role='customer'", (cust,), one=True):
            raise ApiError("Customer not found.", 404)
    if usable_stock(pid) < qty:
        return jsonify(error="Not enough usable stock (expired items are not counted)."), 409
    deduct_stock(pid, qty)
    amount = round(p["price"] * qty, 2)
    run("INSERT INTO sales(customer_id,kind,item_name,qty,amount,created_at) VALUES(?,?,?,?,?,?)",
        (cust, "product", p["name"], qty, amount, now()))
    return jsonify(ok=True, amount=amount)


def service_fields(d, cur=None):
    name = text(d, "name", "Service name", 100, required=cur is None, default=cur["name"] if cur else "")
    desc = text(d, "description", "Description", 255, default=cur["description"] if cur else "") if (cur is None or "de
    price = round(num(d, "price", "Price", 0, 1_000_000, required=cur is None, default=cur["price"] if cur else None),
    dur = num(d, "duration", "Duration", 5, 480, integer=True, required=False, default=cur["duration"] if cur else 30)
    return name, desc, price, dur


@app.post("/api/services")
```

```
@auth("admin")
def add_service():
    d = body()
    name, desc, price, dur = service_fields(d)
    emoji = text(d, "emoji", "Emoji", 8, default="?")
    sid = run("INSERT INTO services(name,description,price,duration,emoji) VALUES(?,?,?,?,?)",
              (name, desc, price, dur, emoji))
    return jsonify(id=sid), 201


@app.put("/api/services/<int:sid>")
@auth("admin")
def edit_service(sid):
    d = body()
    s = q("SELECT * FROM services WHERE id=?", (sid,), one=True)
    if not s:
        return jsonify(error="Service not found."), 404
    name, desc, price, dur = service_fields(d, s)
    run("UPDATE services SET name=?,description=?,price=?,duration=? WHERE id=?", (name, desc, price, dur, sid))
    return jsonify(ok=True)


def vet_fields(d, cur=None):
    name = text(d, "name", "Vet name", 100, required=cur is None, default=cur["name"] if cur else "")
    spec = text(d, "specialty", "Specialty", 100, default=cur["specialty"] if cur else "") if (cur is None or "specialt
    days = work_days_ok(d.get("work_days", cur["work_days"] if cur else "0,1,2,3,4"))
    st = text(d, "start_time", "Start time", 5, default=cur["start_time"] if cur else "09:00")
    en = text(d, "end_time", "End time", 5, default=cur["end_time"] if cur else "17:00")
    if not (valid_hhmm(st) and valid_hhmm(en)):
        raise ApiError("Times must look like 09:00.")
    if en <= st:
        raise ApiError("End time must be later than start time.")
    return name, spec, days, st, en


@app.post("/api/vets")
@auth("admin")
def add_vet():
    d = body()
    name, spec, days, st, en = vet_fields(d)
    vid = run("INSERT INTO vets(name,specialty,work_days,start_time,end_time) VALUES(?,?,?,?,?)", (name, spec, days, st
    return jsonify(id=vid), 201


@app.put("/api/vets/<int:vid>")
@auth("admin")
def edit_vet(vid):
    d = body()
    v = q("SELECT * FROM vets WHERE id=?", (vid,), one=True)
    if not v:
        return jsonify(error="Vet not found."), 404
    name, spec, days, st, en = vet_fields(d, v)
    active = num(d, "active", "Active", 0, 1, integer=True, required=False, default=v["active"])
    run("UPDATE vets SET name=?,specialty=?,work_days=?,start_time=?,end_time=?,active=? WHERE id=?",
        (name, spec, days, st, en, active, vid))
    return jsonify(ok=True)


# sales + dashboard

@app.get("/api/sales")
@auth(*STAFF)
def sales():
    days = num(request.args, "days", "Days", 1, 3650, integer=True, required=False, default=30)
    since = (datetime.now() - timedelta(days=days)).strftime("%Y-%m-%d")
    rs = q("""SELECT s.*, u.full_name AS customer_name FROM sales s
              LEFT JOIN users u ON u.id=s.customer_id
              WHERE s.created_at >= ? ORDER BY s.created_at DESC""", (since,))
    return jsonify(rows(rs))


@app.get("/api/dashboard")
@auth(*STAFF)
def dashboard():
    today = date.today().strftime("%Y-%m-%d")
    since = (date.today() - timedelta(days=6)).strftime("%Y-%m-%d")
    appts_today = q("SELECT COUNT(*) c FROM appointments WHERE appt_date=? AND status!='cancelled'", (today,), one=True
    pending = q("SELECT COUNT(*) c FROM appointments WHERE status='pending'", one=True)["c"]
    sales_today = q("SELECT COALESCE(SUM(amount),0) s FROM sales WHERE substr(created_at,1,10)=?", (today,), one=True)[
    sales_30 = q("SELECT COALESCE(SUM(amount),0) s FROM sales WHERE created_at >= ?",
                 ((date.today() - timedelta(days=30)).strftime("%Y-%m-%d"),), one=True)["s"]
    customers_n = q("SELECT COUNT(*) c FROM users WHERE role='customer'", one=True)["c"]
    info = batch_info()
    views = [product_view(p, info) for p in q("SELECT * FROM products")]
    low = sorted([{"id": v["id"], "name": v["name"], "stock": v["stock"], "reorder_level": v["reorder_level"]}
                  for v in views if v["stock"] <= v["reorder_level"]], key=lambda x: x["stock"])
```

```
    expiring = [b for b in batch_rows() if b["status"] in ("expired", "expiring")]
    pending_list = rows(q(APPT_SQL + " WHERE a.status='pending' ORDER BY a.appt_date, a.appt_time LIMIT 30"))
    by_status = rows(q("SELECT status, COUNT(*) c FROM appointments GROUP BY status"))
    daily = {r["d"]: r["s"] for r in q(
        "SELECT substr(created_at,1,10) d, SUM(amount) s FROM sales WHERE substr(created_at,1,10)>=? GROUP BY d", (sinc
    series = []
    for i in range(7):
        dd = (date.today() - timedelta(days=6 - i)).strftime("%Y-%m-%d")
        series.append({"date": dd, "amount": daily.get(dd, 0)})
    top = rows(q("""SELECT item_name, CAST(SUM(qty) AS SIGNED) qty, SUM(amount) amount FROM sales
                    GROUP BY item_name ORDER BY amount DESC LIMIT 5"""))
    return jsonify(appointments_today=appts_today, pending=pending, sales_today=sales_today, sales_30=sales_30,
                   customers=customers_n, low_stock=low, by_status=by_status, sales_series=series, top_items=top,
                   expiring=expiring, expired_count=sum(1 for b in expiring if b['status'] == 'expired'),
                   pending_list=pending_list)


# notifications

@app.get("/api/notifications")
@auth()
def notifs():
    return jsonify(rows(q("SELECT * FROM notifications WHERE user_id=? ORDER BY id DESC LIMIT 50", (g.user["id"],))))


@app.post("/api/notifications/read")
@auth()
def notifs_read():
    run("UPDATE notifications SET is_read=1 WHERE user_id=?", (g.user["id"],))
    return jsonify(ok=True)


# misc

@app.after_request
def cors(r):
    r.headers["Access-Control-Allow-Origin"] = "*"           # tokens travel in a header, not a cookie
    r.headers["Access-Control-Allow-Headers"] = "Authorization, Content-Type"
    r.headers["Access-Control-Allow-Methods"] = "GET, POST, PUT, DELETE, OPTIONS"
    r.headers["X-Content-Type-Options"] = "nosniff"
    if request.path.startswith("/api/"):
        r.headers["Cache-Control"] = "no-store"
    return r


@app.get("/api/health")
def health():
    return jsonify(ok=True, app="PawPet", time=now())


def check_database():
    """Run once when the server starts. READ-ONLY for the schema: it never creates or alters tables.
    If the database is not ready it says exactly what to do and stops."""
    try:
        c = dbmod.connect_checked()
    except dbmod.SetupNeeded as e:
        if dbmod.ENGINE == "sqlite":                      # test mode only: a throw-away SQLite file sets itself up
            import setup_database
            setup_database.run()
            c = dbmod.connect_checked()
        else:
            print("\n  %s\n  Run the one-time setup first:   py setup_database.py   (or double-click setup_database.bat
            raise SystemExit(1)
    except Exception as e:
        print("\n  Cannot open the database: %s\n  Is MySQL started in XAMPP? Is PAWPET_DB_PASSWORD right?\n" % e)
        raise SystemExit(1)
    flag_default_passwords(c)
    c.close()
    if dbmod.ENGINE != "sqlite" and dbmod.MYSQL_CFG["user"] == "root" and not dbmod.MYSQL_CFG["password"]:
        print("\n  WARNING: the server connects to MySQL as 'root' with NO password.\n"
              "  Before real use, run database\\secure_mysql.sql and set PAWPET_DB_USER / PAWPET_DB_PASSWORD.\n")


def lan_addresses():
    """IPv4 addresses of this laptop (what the other devices must type). Best effort; never raises."""
    found = set()
    try:
        import socket
        for info in socket.getaddrinfo(socket.gethostname(), None, socket.AF_INET):
            found.add(info[4][0])
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            s.connect(("10.255.255.255", 1))             # no packet is sent; just asks the OS which address it would u
            found.add(s.getsockname()[0])
        finally:
            s.close()
```

```
    except Exception:
        pass
    return sorted(a for a in found if not a.startswith("127."))


def print_banner(port, scheme="http"):
    print("=" * 66)
    print(" PawPet server is running.  On the OTHER devices use ONE of:")
    for a in lan_addresses() or ["(run ipconfig to find this laptop's IPv4 address)"]:
        print("     %s://%s:%d" % (scheme, a, port))
    print(" On THIS laptop:  %s://localhost:%d" % (scheme, port))
    print(" If another device cannot connect: run open_firewall.bat as Administrator.")
    print("=" * 66, flush=True)


if __name__ == "__main__":
    host, port = "0.0.0.0", int(os.environ.get("PAWPET_PORT", "5000"))
    check_database()
    cert, key = os.environ.get("PAWPET_SSL_CERT"), os.environ.get("PAWPET_SSL_KEY")
    print_banner(port, "https" if (cert and key) else "http")
    if cert and key:                                          # optional HTTPS (see PAWPET_GUIDE.md)
        app.run(host=host, port=port, debug=False, threaded=True, ssl_context=(cert, key))
    elif os.environ.get("PAWPET_WAITRESS") == "1":            # optional production server: py -m pip install waitress
        from waitress import serve
        print(f"PawPet server (waitress) on http://{host}:{port}")
        serve(app, host=host, port=port, threads=8)
    else:
        app.run(host=host, port=port, debug=False, threaded=True)
```

LAPTOP2_Server/setup_database.py - Server - one-time database setup

```
"""PawPet - one-time database setup.      Run on the SERVER laptop with MySQL started:

    py setup_database.py              set up (or upgrade) the database, nothing is deleted
    py setup_database.py --reset      DELETE every table and start again with fresh sample data (asks first)

What it does, in order:
  1. creates the database if it is missing (MySQL),
  2. empty database            -> runs database/schema.sql, then adds the sample data and the first-run accounts,
  3. database from an OLD PawPet (tables but no schema_version) -> adds only what is missing, keeps all data,
  4. already on the current version -> says so and does nothing,
  5. older numbered version    -> applies the entries of db.UPGRADES in order.
The server (app.py) never creates or changes tables; it only checks that this step was done.
"""
import os
import sys
from datetime import date, timedelta

import db as dbmod
from db import Conn, SCHEMA_VERSION, UPGRADES
from app import TEST_FIXED_PASSWORDS, hash_pw, now, temp_password


def say(msg=""):
    print(msg, flush=True)


# sample data (fresh databases only)
def seed(c):
    t = now()
    ins = c.execute
    accounts = [("admin", "Clinic Admin", "admin@pawpet.ph", "09170000001", "admin", 1),
                ("staff", "Nurse Joy Reyes", "joy@pawpet.ph", "09170000002", "staff", 1),
                ("megan", "Megan Cruz", "megan@mail.com", "09171234567", "customer", 0),
                ("lester", "Lester John Jatico", "lester@mail.com", "09181234567", "customer", 0)]
    fixed = {"admin": "admin123", "staff": "staff123", "megan": "pass123", "lester": "pass123"}
    lines = []
    for username, full, email, phone, role, must in accounts:
        pw = fixed[username] if TEST_FIXED_PASSWORDS else temp_password()
        ins("INSERT INTO users(username,password,full_name,email,phone,role,must_change,created_at) VALUES(?,?,?,?,?,?,
            (username, hash_pw(pw), full, email, phone, role, 0 if TEST_FIXED_PASSWORDS else must, t))
        lines.append(f"{role:9} {username:8} {pw}")
    if not TEST_FIXED_PASSWORDS:
        path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "FIRST_RUN_PASSWORDS.txt")
        with open(path, "w", encoding="utf-8") as f:
            f.write("PawPet first-run passwords (random). admin and staff must change theirs at first sign-in.\n"
                    "Keep this file private and delete it after you have noted the passwords.\n\n"
                    + "\n".join(lines) + "\n")
        print("\n  First run: sample accounts were created with RANDOM passwords.\n  They are saved in " + path + "\n\r
              + "\n  ".join(lines) + "\n")
    for p in [(3, "Buddy", "Dog", "Golden Retriever", 5, 28, "Allergic to chicken"),
              (3, "Mochi", "Cat", "Puspin", 2, 4, ""),
              (4, "Brock", "Dog", "Aspin", 3, 15, "")]:
        ins("INSERT INTO pets(owner_id,name,species,breed,age,weight,notes) VALUES(?,?,?,?,?,?,?)", p)

    def days(n):
```

```
        return (date.today() + timedelta(days=n)).strftime("%Y-%m-%d")
    # name, category, price, stock, reorder level, emoji, [(batch no, qty, expiry)]
    for name, cat, price, stock, rl, emoji, batches in [
            ("Premium Dog Food 5kg", "Food", 1250, 24, 8, "?", [("PF-2610", 24, days(300))]),
            ("Cat Dry Food 3kg", "Food", 890, 18, 8, "?", [("CF-2609", 18, days(260))]),
            ("Dewormer Syrup", "Medicine", 180, 40, 10, "?", [("DW-001", 25, days(240)), ("DW-002", 15, days(25))]),
            ("Anti-Tick Shampoo", "Grooming", 320, 12, 5, "?", [("AT-001", 12, days(500))]),
            ("Vitamin Drops", "Medicine", 260, 4, 6, "?", [("VD-001", 4, days(-10))]),
            ("Chew Toy Bone", "Toys", 150, 30, 8, "?", [("TY-001", 30, None)]),
            ("Cat Litter 10L", "Hygiene", 410, 15, 5, "?", [("CL-001", 15, None)]),
            ("Pet Carrier Small", "Accessories", 1450, 6, 3, "?", [("PC-001", 6, None)]),
            ("Flea Collar", "Accessories", 220, 0, 5, "?", []),
            ("Rabies Vaccine Dose", "Medicine", 450, 20, 6, "?", [("RV-001", 20, days(90))])]:
        pid = ins("INSERT INTO products(name,category,price,stock,reorder_level,emoji) VALUES(?,?,?,?,?,?)",
                  (name, cat, price, stock, rl, emoji))
        for bno, bq, bexp in batches:
            ins("INSERT INTO batches(product_id,batch_no,qty,expiry_date,received_at) VALUES(?,?,?,?,?)",
                (pid, bno, bq, bexp, t))
    for s in [("Wellness Checkup", "Full physical exam and weight check", 350, 30, "?"),
              ("Vaccination", "Core vaccines with record card", 500, 30, "?"),
              ("Deworming", "Safe deworming and advice", 250, 30, "?"),
              ("Grooming", "Bath, trim and nail clipping", 600, 60, "?"),
              ("Dental Cleaning", "Scaling and polish", 1200, 60, "?"),
              ("Minor Surgery", "Consultation required first", 3500, 90, "?")]:
        ins("INSERT INTO services(name,description,price,duration,emoji) VALUES(?,?,?,?,?)", s)
    # work_days: 0 = Monday ... 6 = Sunday
    for v in [("Dr. Dave Singer", "General Practice", "0,1,2,3,4", "09:00", "17:00"),
              ("Dr. Anna Thomas", "Surgery", "1,2,3,4,5", "10:00", "18:00"),
              ("Dr. Paolo Lim", "Dermatology", "0,2,4,5", "09:00", "15:00")]:
        ins("INSERT INTO vets(name,specialty,work_days,start_time,end_time) VALUES(?,?,?,?,?)", v)
    today = date.today()

    def day(n):
        return (today + timedelta(days=n)).strftime("%Y-%m-%d")

    def at(n, hhmm):
        return day(n) + " " + hhmm + ":00"
    vets = {v["id"]: v["work_days"].split(",") for v in c.query("SELECT id, work_days FROM vets")}

    def slot(n, vet, step=1):
        """Day n from today for this vet; moves (forward, or back with step=-1) to a day the vet works (0 = Monday)."""
        while str((today + timedelta(days=n)).weekday()) not in vets[vet]:
            n += step
        return day(n)
    # (customer, pet, service, vet, date, time, status, notes, created)
    appts = [(3, 1, 2, 1, slot(-1, 1, -1), "10:00", "completed", "Annual vaccine", at(-3, "09:15")),
             (3, 1, 1, 1, slot(0, 1), "10:00", "confirmed", "", at(-1, "14:30")),
             (4, 3, 2, 2, slot(0, 2), "11:00", "pending", "", at(0, "08:05")),
             (3, 2, 3, 1, slot(1, 1), "09:30", "pending", "", at(0, "08:40")),
             (4, 3, 1, 3, slot(2, 3), "13:00", "confirmed", "", at(0, "09:10"))]
    ids = []
    for a in appts:
        ids.append(ins("""INSERT INTO appointments(customer_id,pet_id,service_id,vet_id,appt_date,appt_time,status,note
                          VALUES(?,?,?,?,?,?,?,?,?)""", (a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7], a[8])))
    # (customer, appointment, kind, item, qty, amount, when)
    for s in [(3, ids[0], "service", "Vaccination", 1, 500, at(-1, "10:40")),
              (3, None, "product", "Premium Dog Food 5kg", 1, 1250, at(-1, "10:45")),
              (4, None, "product", "Dewormer Syrup", 2, 360, at(-2, "15:20")),
              (None, None, "product", "Chew Toy Bone", 3, 450, at(-3, "11:05")),
              (3, None, "service", "Wellness Checkup", 1, 350, at(-5, "09:50"))]:
        ins("INSERT INTO sales(customer_id,appointment_id,kind,item_name,qty,amount,created_at) VALUES(?,?,?,?,?,?,?)",
    ins("INSERT INTO notifications(user_id,title,body,created_at) VALUES(3,'Welcome to PawPet','Book your first visit a


def migrate_batches(c):
    """Older databases have stock but no batches yet: give every stocked product one starting batch.
    The sample medicines get demo expiry dates so the expiry alerts can be seen right away."""
    demo = {"Dewormer Syrup": 240, "Vitamin Drops": 20, "Rabies Vaccine Dose": 90}
    t = now()
    for p in c.query("SELECT id,name,category,stock FROM products WHERE stock>0"):
        if c.query("SELECT 1 AS x FROM batches WHERE product_id=? LIMIT 1", (p["id"],)):
            continue
        exp = None
        if p["name"] in demo:
            exp = (date.today() + timedelta(days=demo[p["name"]])).strftime("%Y-%m-%d")
        c.execute("INSERT INTO batches(product_id,batch_no,qty,expiry_date,received_at) VALUES(?,?,?,?,?)",
                  (p["id"], "INITIAL", p["stock"], exp, t))


# the steps
def create_database_if_missing():
    """MySQL only: connects without a database and creates it when it is not there yet."""
    if dbmod.ENGINE == "sqlite":
        return
    root = Conn(database=False)
    cur = root.raw.cursor()
```

```
    cur.execute("SHOW DATABASES LIKE %s", (dbmod.MYSQL_NAME,))
    if cur.fetchone():
        say("Database '%s' already exists." % dbmod.MYSQL_NAME)
    else:
        cur.execute("CREATE DATABASE `%s` CHARACTER SET utf8mb4" % dbmod.MYSQL_NAME)
        root.raw.commit()
        say("Created database '%s'." % dbmod.MYSQL_NAME)
    cur.close()
    root.close()


def run_statements(c, statements):
    for stmt in statements:
        c.execute(stmt)


def fresh_install(c):
    say("Creating the tables from database/schema.sql ...")
    run_statements(c, dbmod.schema_statements())
    say("Adding sample data and the first-run accounts ...")
    seed(c)
    migrate_batches(c)          # gives the sample products their starting batches
    say("Done: fresh database at schema version %d." % SCHEMA_VERSION)


def upgrade_old_database(c):
    """A database made by an earlier PawPet (before schema_version existed): add what is missing, keep the data."""
    say("This database has tables from an older PawPet. Adding only what is missing (no data is deleted) ...")
    have = dbmod.existing_tables(c)
    stmts = dbmod.schema_statements()
    for stmt in stmts:                                          # 1) tables that do not exist yet
        t = dbmod.table_of(stmt)
        if t and t not in have:
            c.execute(stmt)
            say("  + table %s" % t)
    cols = {r["n"].lower() for r in (c.query("SELECT name AS n FROM pragma_table_info('users')") if dbmod.ENGINE == "sq
            else c.query("SELECT column_name AS n FROM information_schema.columns WHERE table_schema=DATABASE() AND tab
    if "must_change" not in cols:                               # 2) columns added by later versions
        c.execute("ALTER TABLE users ADD COLUMN must_change %s NOT NULL DEFAULT 0" % ("INTEGER" if dbmod.ENGINE == "sql
        say("  + column users.must_change")
    if dbmod.ENGINE != "sqlite":
        c.execute("ALTER TABLE users MODIFY password VARCHAR(200) NOT NULL")   # room for the new password hashes
    for stmt in stmts:                                          # 3) indexes that are missing
        if stmt.upper().startswith("CREATE INDEX"):
            try:
                c.execute(stmt)
                say("  + index %s" % stmt.split()[2])
            except Exception as e:
                if "exists" not in str(e).lower() and "duplicate key name" not in str(e).lower():
                    raise
                try:
                    c.raw.rollback()
                except Exception:
                    pass
    migrate_batches(c)                                          # 4) old stock gets a starting batch
    if not c.query("SELECT 1 AS x FROM schema_version"):
        c.execute("INSERT INTO schema_version(version, description) VALUES (?, ?)", (SCHEMA_VERSION, "Upgraded from an
    say("Done: database is now at schema version %d." % SCHEMA_VERSION)


def apply_upgrades(c, version):
    for v in sorted(UPGRADES):
        if v > version:
            say("Applying upgrade to version %d ..." % v)
            for stmt in UPGRADES[v]:
                c.execute(stmt)
            c.execute("INSERT INTO schema_version(version, description) VALUES (?, ?)", (v, "Upgrade"))
    say("Done: database is now at schema version %d." % max(UPGRADES or {SCHEMA_VERSION: 0}))


def drop_everything(c):
    say("Deleting every PawPet table ...")
    if dbmod.ENGINE == "sqlite":
        for t in dbmod.existing_tables(c):
            c.raw.execute("DROP TABLE %s" % t)
        c.raw.commit()
    else:
        cur = c.raw.cursor()
        cur.execute("SET FOREIGN_KEY_CHECKS=0")
        for t in dbmod.existing_tables(c):
            cur.execute("DROP TABLE `%s`" % t)
        cur.execute("SET FOREIGN_KEY_CHECKS=1")
        c.raw.commit()
        cur.close()
```

```
def run(reset=False, assume_yes=False):
    create_database_if_missing()
    c = Conn()
    try:
        if reset and dbmod.existing_tables(c):
            if not assume_yes and input("This DELETES ALL PawPet data (users, pets, appointments, sales...). Type YES t
                say("Cancelled. Nothing was changed.")
                return 1
            drop_everything(c)
        tables = dbmod.existing_tables(c)
        version = dbmod.current_version(c)
        if not tables:
            fresh_install(c)
        elif version is None:
            upgrade_old_database(c)
        elif version == SCHEMA_VERSION:
            say("The database is already set up (schema version %d). Nothing to do." % version)
        elif version < SCHEMA_VERSION:
            apply_upgrades(c, version)
        else:
            say("The database is on version %d, newer than this server (%d). Use the newer PawPet files." % (version, S
            return 1
    finally:
        c.close()
    return 0


if __name__ == "__main__":
    try:
        sys.exit(run(reset="--reset" in sys.argv, assume_yes="--yes" in sys.argv))
    except Exception as e:                      # a short, readable reason instead of a long traceback
        say("\nSetup could not finish: %s" % e)
        say("Is MySQL started in XAMPP? Is the password right (PAWPET_DB_PASSWORD)?")
        sys.exit(1)
```

LAPTOP2_Server/send_reminders.py - Server - reminder script for Task Scheduler

```
"""Sends today's/tomorrow's appointment reminders through the PawPet server.
Meant to be run by Windows Task Scheduler (see send_reminders.bat). Uses only the standard library."""
import json, os, sys, urllib.request, datetime

BASE = os.environ.get("PAWPET_URL", "http://localhost:5000")
HERE = os.path.dirname(os.path.abspath(__file__))
LOG = os.path.join(HERE, "reminders.log")


def credentials():
    """Staff login used for the reminders. Either set PAWPET_STAFF_USER / PAWPET_STAFF_PASS, or put the
    username on line 1 and the password on line 2 of reminders_login.txt (next to this file; keep it private)."""
    u, p = os.environ.get("PAWPET_STAFF_USER"), os.environ.get("PAWPET_STAFF_PASS")
    if u and p:
        return u, p
    path = os.path.join(HERE, "reminders_login.txt")
    if os.path.exists(path):
        lines = [x.strip() for x in open(path, encoding="utf-8").read().splitlines() if x.strip()]
        if len(lines) >= 2:
            return lines[0], lines[1]
    raise RuntimeError("No staff login for reminders. Create reminders_login.txt (line 1 = staff username, line 2 = pas


def call(path, body=None, token=None):
    req = urllib.request.Request(BASE + path, data=json.dumps(body or {}).encode(), method="POST",
                                 headers={"Content-Type": "application/json"})
    if token:
        req.add_header("Authorization", "Bearer " + token)
    with urllib.request.urlopen(req, timeout=15) as r:
        return json.load(r)


def main():
    try:
        user, password = credentials()
        token = call("/api/login", {"username": user, "password": password, "portal": "admin"})["token"]
        sent = call("/api/reminders/send", token=token)["sent"]
        msg = f"OK - {sent} reminder(s) sent"
        code = 0
    except Exception as e:
        msg, code = f"FAILED - {e}", 1
    line = f"{datetime.datetime.now():%Y-%m-%d %H:%M:%S} {msg}"
    print(line)
    with open(LOG, "a", encoding="utf-8") as f:
        f.write(line + "\n")
    sys.exit(code)


main()
```

LAPTOP2_Server/database/schema.sql - Database schema

```
-- ======================================================================
-- PawPet database schema  (MySQL / MariaDB, XAMPP)           version 1
-- This file is the ONE source of truth for the tables.
--   * Normal use : py setup_database.py   (reads this file, run once)
--   * Manual use : import this whole file in phpMyAdmin (Import tab) on an empty server
-- Plain CREATE TABLE is used on purpose: the tables are created once, by the setup step.
-- The server never creates or alters tables; it only checks that schema_version matches.
-- Dates/times are stored as text (YYYY-MM-DD, HH:MM, YYYY-MM-DD HH:MM:SS) and money as DOUBLE,
-- so the JSON sent to the apps is simple and identical on every engine.
-- ======================================================================
CREATE DATABASE pawpet CHARACTER SET utf8mb4;
USE pawpet;
SET NAMES utf8mb4;

-- Accounts of every role. role = customer | staff | admin
CREATE TABLE users(
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(200) NOT NULL,              -- pbkdf2$iterations$salt$hash
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(30),
    role VARCHAR(20) NOT NULL DEFAULT 'customer',
    active TINYINT NOT NULL DEFAULT 1,
    must_change TINYINT NOT NULL DEFAULT 0,      -- 1 = must choose a new password at next sign-in
    created_at VARCHAR(19) NOT NULL
) ENGINE=InnoDB;

-- Sign-in sessions (bearer tokens)
CREATE TABLE tokens(
    token VARCHAR(64) PRIMARY KEY,
    user_id INT NOT NULL,
    created_at VARCHAR(19) NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE pets(
    id INT AUTO_INCREMENT PRIMARY KEY,
    owner_id INT NOT NULL,
    name VARCHAR(60) NOT NULL,
    species VARCHAR(30) NOT NULL,
    breed VARCHAR(60),
    age DOUBLE,
    weight DOUBLE,
    notes VARCHAR(255),
    FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE products(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(40) NOT NULL,
    price DOUBLE NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    reorder_level INT NOT NULL DEFAULT 5,
    emoji VARCHAR(8) DEFAULT '?'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Every delivery of a product, with its own expiry date
CREATE TABLE batches(
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT NOT NULL,
    batch_no VARCHAR(40),
    qty INT NOT NULL DEFAULT 0,
    expiry_date VARCHAR(10),
    received_at VARCHAR(19) NOT NULL,
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE services(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    price DOUBLE NOT NULL,
    duration INT NOT NULL DEFAULT 30,            -- minutes
    emoji VARCHAR(8) DEFAULT '?'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE vets(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    specialty VARCHAR(100),
    work_days VARCHAR(20) NOT NULL DEFAULT '0,1,2,3,4',   -- 0 = Monday ... 6 = Sunday
    start_time VARCHAR(5) NOT NULL DEFAULT '09:00',
```

```
    end_time VARCHAR(5) NOT NULL DEFAULT '17:00',
    active TINYINT NOT NULL DEFAULT 1
) ENGINE=InnoDB;

-- status = pending | confirmed | completed | cancelled | no_show
CREATE TABLE appointments(
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    pet_id INT NOT NULL,
    service_id INT NOT NULL,
    vet_id INT NOT NULL,
    appt_date VARCHAR(10) NOT NULL,
    appt_time VARCHAR(5) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    notes VARCHAR(500),
    followup_of INT,
    created_at VARCHAR(19) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES users(id),
    FOREIGN KEY (pet_id) REFERENCES pets(id),
    FOREIGN KEY (service_id) REFERENCES services(id),
    FOREIGN KEY (vet_id) REFERENCES vets(id),
    FOREIGN KEY (followup_of) REFERENCES appointments(id)
) ENGINE=InnoDB;

-- kind = product | service
CREATE TABLE sales(
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    appointment_id INT,
    kind VARCHAR(20) NOT NULL,
    item_name VARCHAR(100) NOT NULL,
    qty INT NOT NULL DEFAULT 1,
    amount DOUBLE NOT NULL,
    created_at VARCHAR(19) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE notifications(
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    title VARCHAR(120) NOT NULL,
    body VARCHAR(500),
    is_read TINYINT NOT NULL DEFAULT 0,
    created_at VARCHAR(19) NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Which version of this schema the database is on (the server refuses to start on a different version)
CREATE TABLE schema_version(
    version INT NOT NULL PRIMARY KEY,
    description VARCHAR(100) NOT NULL,
    applied_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Indexes for the lookups the apps do all the time
CREATE INDEX idx_vet_day ON appointments(vet_id, appt_date);
CREATE INDEX idx_appt_date_status ON appointments(appt_date, status);
CREATE INDEX idx_appt_customer ON appointments(customer_id);
CREATE INDEX idx_pets_owner ON pets(owner_id);
CREATE INDEX idx_batches_product ON batches(product_id);
CREATE INDEX idx_sales_created ON sales(created_at);
CREATE INDEX idx_notif_user ON notifications(user_id, is_read);
CREATE INDEX idx_tokens_user ON tokens(user_id);

INSERT INTO schema_version(version, description) VALUES (1, 'Initial schema');
```

LAPTOP1_Desktop/api_client.py - Desktop - API client

```
"""Thin HTTP client the desktop admin uses to talk to the PawPet server."""
import requests

CONNECT_TIMEOUT = 3     # seconds to reach the server
READ_TIMEOUT = 8        # seconds to wait for its answer


class ApiError(Exception):
    def __init__(self, message, status=0, must_change=False):
        super().__init__(message)
        self.status = status
        self.must_change = must_change


class Api:
    def __init__(self, base="http://localhost:5000"):
        self.base = base.rstrip("/")
        self.token = ""
        self.user = None
```

```
    def _call(self, method, path, body=None, params=None, timeout=(CONNECT_TIMEOUT, READ_TIMEOUT)):
        headers = {"Content-Type": "application/json"}
        if self.token:
            headers["Authorization"] = "Bearer " + self.token
        try:
            r = requests.request(method, self.base + "/api" + path, json=body, params=params,
                                 headers=headers, timeout=timeout)
        except requests.exceptions.ReadTimeout:
            raise ApiError("The server is taking too long to answer. Check the Wi-Fi/network and try again.")
        except requests.exceptions.InvalidURL:
            raise ApiError("The server address is not valid. It should look like http://192.168.1.10:5000")
        except requests.exceptions.RequestException:
            hint = ""
            if "localhost" in self.base or "127.0.0.1" in self.base:
                hint = " 'localhost' means THIS computer - on another laptop type the server laptop's IP address instea
            raise ApiError("Cannot reach the server at %s. Check that it is running, that both devices are on the same
                           "Wi-Fi, and that the address is right.%s" % (self.base, hint))
        try:
            data = r.json()
        except ValueError:
            data = {}
        if not isinstance(data, (dict, list)):
            data = {}
        if r.status_code >= 400:
            msg = data.get("error") if isinstance(data, dict) else None
            if r.status_code == 401 and self.token and path != "/login":
                msg = "Your session expired. Please sign in again."
            raise ApiError(msg or "Server error (%d). Please try again." % r.status_code, r.status_code,
                           bool(isinstance(data, dict) and data.get("must_change")))
        return data

    def get(self, path, **params):
        return self._call("GET", path, params={k: v for k, v in params.items() if v not in (None, "")})

    def post(self, path, body=None):
        return self._call("POST", path, body or {})

    def put(self, path, body):
        return self._call("PUT", path, body)

    def delete(self, path):
        return self._call("DELETE", path)

    def health(self):
        """Quick 'is the server there?' check (1.5 s at most) used before every automatic refresh."""
        try:
            requests.get(self.base + "/api/health", timeout=1.5)
            return True
        except requests.exceptions.RequestException:
            return False

    def login(self, username, password):
        r = self._call("POST", "/login", {"username": username, "password": password, "portal": "admin"})
        self.token, self.user = r["token"], r["user"]
        return self.user

    def change_password(self, old, new):
        return self._call("POST", "/change-password", {"old_password": old, "new_password": new})

    def logout(self):
        try:
            self._call("POST", "/logout", timeout=(2, 3))
        except ApiError:
            pass
        self.token, self.user = "", None
```

LAPTOP1_Desktop/admin.py - Desktop - application shell (Flet)

```
"""PawPet desktop app for admin and staff (Python + Flet).
Run: pip install -r requirements.txt, then python admin.py. The server must be running."""
import asyncio
import os
import traceback
from datetime import datetime

import flet as ft

from api_client import Api, ApiError
from pages import ALL_PAGES, Appointments, Dashboard
from ui import (AMBER, BG, GREEN, GREEN_D, LINE, MUTED, PANEL, PANEL2, RED, SIDEBAR, TEXT, card, danger, dropdown, fiel
                label, plain, primary, t12)

REFRESH_S = 8          # auto-refresh interval (seconds)
DEFAULT_URL = "http://localhost:5000"
ADDR_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "server_address.txt")
```

```
def saved_url():
    """Server address from the last successful login."""
    try:
        with open(ADDR_FILE, encoding="utf-8") as f:
            return f.read().strip() or DEFAULT_URL
    except OSError:
        return DEFAULT_URL


def remember_url(url):
    try:
        with open(ADDR_FILE, "w", encoding="utf-8") as f:
            f.write(url)
    except OSError:
        pass


class App:
    def __init__(self, page: ft.Page):
        self.page = page
        self.api = Api()
        self.session = 0              # bumped on every login/logout so old refresh loops stop
        self.open = []                # futures of the dialogs that are currently open
        self._dlgs = {}
        self.pages, self.nav, self.nav_text = {}, {}, {}
        self.current = None
        self.offline = False
        self.seen_pending = None
        page.title = "PawPet Admin"
        page.padding = 0
        page.spacing = 0
        page.bgcolor = BG
        page.theme_mode = ft.ThemeMode.DARK
        page.theme = ft.Theme(color_scheme_seed=GREEN)
        page.dark_theme = ft.Theme(color_scheme_seed=GREEN)
        page.window.width = 1280
        page.window.height = 760
        page.window.min_width = 1200
        page.window.min_height = 640

    # ================================================================ plumbing
    async def call(self, fn, *args, **kwargs):
        """Run the (slow) API call in a thread so the window does not freeze."""
        return await asyncio.to_thread(fn, *args, **kwargs)

    def show(self, *controls):
        self.page.controls = list(controls)
        self.page.update()

    async def fail(self, e, quiet=False):
        """Decide what to do with an API error."""
        if e.status == 401 and self.api.token:
            return await self.session_expired()
        if e.must_change:
            if not getattr(self, "_forcing", False):
                await self.force_change()
            return
        if quiet:                       # background refresh: no pop-up, just say so in the header
            self.offline = True
            return
        await self.info("PawPet", str(e))

    # ================================================================ dialogs
    def _hide(self, dlg):
        """Close this dialog (not just the top one, a toast may be on top)."""
        dlg.open = False
        try:
            dlg.update()
        except Exception:
            self.page.update()

    def _close(self, dlg, fut, result):
        if dlg.open:
            self._hide(dlg)
        if fut in self.open:
            self.open.remove(fut)
        if not fut.done():
            fut.set_result(result)

    def close_all(self):
        for fut in list(self.open):
            self.open.remove(fut)
            dlg = self._dlgs.pop(fut, None)
            if dlg is not None and dlg.open:
```

```
            self._hide(dlg)
        if not fut.done():
            fut.set_result(False)

@staticmethod
def _click(handler, close):
    async def click(e=None):            # a real coroutine function, so Flet awaits it
        await handler(close)
    return click

async def _show(self, title, body, buttons, modal=True):
    """buttons: list of (text, kind, handler(close)). Returns what the dialog is closed with."""
    fut = asyncio.get_running_loop().create_future()
    dlg = ft.AlertDialog(modal=modal, title=label(title, 17, TEXT, True), bgcolor=PANEL, content=body, actions=[])

    def close(result=None):
        self._close(dlg, fut, result)
    acts = []
    for text, kind, handler in buttons:
        mk = {"primary": primary, "danger": danger, "plain": plain}.get(kind, plain)
        acts.append(mk(text, self._click(handler, close)))
    dlg.actions = acts
    dlg.on_dismiss = lambda e: (fut.set_result(False) if not fut.done() else None)
    self.open.append(fut)
    self._dlgs[fut] = dlg
    self.page.show_dialog(dlg)
    return await fut

async def info(self, title, message):
    body = ft.Container(label(message, 13, TEXT, selectable=True), width=440)

    async def ok(close):
        close(True)
    await self._show(title, body, [("OK", "primary", ok)])

async def confirm(self, message, yes="Yes"):
    body = ft.Container(label(message, 13, TEXT), width=420)

    async def y(close):
        close(True)

    async def n(close):
        close(False)
    return bool(await self._show("PawPet", body, [("No", "plain", n), (yes, "primary", y)]))

def toast(self, message):
    self.page.show_dialog(ft.SnackBar(content=label(message, 13, "#ffffff"), bgcolor=GREEN_D, duration=3500))

async def custom_form(self, title, controls, on_save, save_text="Save", cancel_text="Cancel"):
    """Dialog with ready-made controls. on_save() is async; raise ValueError/ApiError to show a message
    inside the dialog, return False to keep it open. Returns True when saved."""
    err = label("", 12, RED)
    body = ft.Container(ft.Column(list(controls) + [err], tight=True, spacing=10, scroll=ft.ScrollMode.AUTO),
                        width=470)
    busy = {"on": False}

    async def save(close):
        if busy["on"]:
            return
        busy["on"] = True
        try:
            r = await on_save()
        except (ApiError, ValueError) as ex:
            if isinstance(ex, ApiError) and ex.status == 401 and self.api.token:
                close(False)
                return await self.session_expired()
            err.value = str(ex)
            self.page.update()
            return
        finally:
            busy["on"] = False
        if r is not False:
            close(True)

    async def cancel(close):
        close(False)
    return bool(await self._show(title, body, [(cancel_text, "plain", cancel), (save_text, "primary", save)]))

async def form(self, title, fields, on_save, save_text="Save"):
    """fields: (key, label, kind, default, options); kind = entry | text | password | combo | check.
    on_save(values) is async. Combo values are the option key (a string)."""
    widgets = {}
    for key, text, kind, default, opts in fields:
        if kind == "combo":
            w = dropdown(text, opts, default, width=420)
        elif kind == "check":
```

```
            w = ft.Checkbox(label=text, value=bool(int(default or 0)))
        elif kind == "password":
            w = field(text, "", width=420, password=True)
        else:
            w = field(text, default, width=420)
        widgets[key] = w

    def values():
        out = {}
        for k, w in widgets.items():
            v = w.value
            out[k] = v if isinstance(w, ft.Checkbox) else ("" if v is None else v)
        return out

    async def save():
        return await on_save(values())
    return await self.custom_form(title, list(widgets.values()), save, save_text)

# ============================================================== login
async def start(self):
    self.login_view()

def login_view(self, message=""):
    self.close_all()
    self.session += 1
    self.pages, self.nav, self.nav_text, self.current = {}, {}, {}, None
    self.url = field("Server address (server laptop IP)", saved_url(), width=360)
    self.user = field("Username", "", width=360, autofocus=True)
    self.pw = field("Password", "", width=360, password=True, on_submit=self.do_login)
    self.login_err = label(message, 12, RED)
    self.login_btn = primary("Sign in", self.do_login, ft.Icons.LOGIN)
    self.login_btn.width = 360
    self.spin = ft.ProgressRing(width=18, height=18, visible=False)
    hero = ft.Container(
        expand=True, bgcolor="#0f5132", alignment=ft.Alignment.CENTER,
        content=ft.Column([
            label("?  PawPet", 30, "#ffffff", True),
            ft.Container(height=18),
            label("Run the clinic.\nNot the paperwork.", 34, "#ffffff", True),
            ft.Container(height=8),
            label("Inventory ? vet schedules ? appointments ? sales", 13, "#bfe3cd")],
            tight=True, horizontal_alignment=ft.CrossAxisAlignment.START))
    form = ft.Container(
        expand=True, alignment=ft.Alignment.CENTER,
        content=card(ft.Column([
            label("Staff sign in", 22, TEXT, True), label("PawPet Admin Dashboard", 12, MUTED),
            ft.Container(height=6), self.url, self.user, self.pw, self.login_err,
            ft.Row([self.login_btn, self.spin], spacing=10)], tight=True, spacing=10), padding=30))
    self.show(ft.Row([hero, form], expand=True, spacing=0))

async def do_login(self, e=None):
    self.login_err.value = ""
    self.login_btn.disabled = True
    self.spin.visible = True
    self.page.update()
    self.api = Api((self.url.value or "").strip() or DEFAULT_URL)
    try:
        await self.call(self.api.login, (self.user.value or "").strip(), self.pw.value or "")
    except ApiError as ex:
        self.login_err.value = str(ex)
        self.login_btn.disabled = False
        self.spin.visible = False
        self.page.update()
        return
    remember_url(self.api.base)
    if self.api.user.get("must_change"):
        self.login_btn.disabled = False
        self.spin.visible = False
        self.page.update()
        if not await self.change_pw(forced=True):        # must choose a new password before anything else
            await self.call(self.api.logout)
            return self.login_view("You must change your temporary password to continue.")
    self.main_view()

async def force_change(self):
    """A request came back 'must change password' (e.g. an admin reset it while we were signed in)."""
    self._forcing = True
    try:
        if not await self.change_pw(forced=True):
            await self.logout()
    finally:
        self._forcing = False

async def change_pw(self, forced=False):
    async def save(v):
        if v["new"] != v["again"]:
```

```
            raise ValueError("The two new passwords do not match.")
        await self.call(self.api.change_password, v["old"], v["new"])
        self.api.user["must_change"] = False
        self.toast("Password changed.")
    await self.form("Choose a new password" if forced else "Change password", [
        ("old", "Current password", "password", "", None),
        ("new", "New password (8+ chars, letters + numbers)", "password", "", None),
        ("again", "Repeat new password", "password", "", None)], save, "Change password")
    return not self.api.user.get("must_change")

async def session_expired(self):
    self.api.token = ""
    self.login_view("Your session expired. Please sign in again.")

# ============================================================ main window
def main_view(self):
    u = self.api.user
    self.session += 1
    gen = self.session
    classes = [c for c in ALL_PAGES if not (c.admin_only and u["role"] != "admin")]
    nav = []
    for cls in classes:
        txt = label(cls.title, 13, MUTED)
        item = ft.Container(
            content=ft.Row([ft.Icon(cls.icon, size=18, color=MUTED), txt], spacing=12),
            padding=ft.Padding.symmetric(vertical=11, horizontal=14), border_radius=ft.BorderRadius.all(10),
            ink=True, on_click=(lambda e, c=cls: self.page.run_task(self.goto, c)))
        self.nav[cls], self.nav_text[cls] = item, txt
        nav.append(item)
    side = ft.Container(
        width=232, bgcolor=SIDEBAR, padding=ft.Padding.all(12),
        content=ft.Column([
            ft.Container(label("?  PawPet", 20, "#ffffff", True), padding=ft.Padding.only(left=8, top=10, bottom=18
            ft.Column(nav, spacing=3, expand=True, scroll=ft.ScrollMode.AUTO),
            plain("Log out", lambda e: self.page.run_task(self.logout), ft.Icons.LOGOUT)], expand=True))
    self.title_lbl = label("", 24, TEXT, True)
    self.status_lbl = label("", 11, MUTED)
    head = ft.Row([self.title_lbl, ft.Row([
        self.status_lbl,
        plain("Password", lambda e: self.page.run_task(self.change_pw), ft.Icons.KEY),
        plain("Send reminders", lambda e: self.page.run_task(self.reminders), ft.Icons.NOTIFICATIONS),
        label("%s  (%s)" % (u["full_name"], u["role"]), 12, MUTED)], spacing=12)],
        alignment=ft.MainAxisAlignment.SPACE_BETWEEN)
    self.alert_txt = label("", 13, "#06210f", True)
    self.alert = ft.Container(content=self.alert_txt, bgcolor=AMBER, visible=False, padding=ft.Padding.all(10),
                              border_radius=ft.BorderRadius.all(10), ink=True, on_click=self.dismiss_alert)
    self.body = ft.Container(expand=True)
    content = ft.Container(expand=True, padding=ft.Padding.all(22),
                           content=ft.Column([head, self.alert, self.body], expand=True, spacing=12))
    self.seen_pending = None
    self.offline = False
    self.show(ft.Row([side, content], expand=True, spacing=0))
    for cls in classes:
        self.pages[cls] = cls(self)
    self.page.run_task(self.goto, Dashboard)
    self.page.run_task(self.ticker, gen)

async def goto(self, cls):
    if cls not in self.pages:
        return
    self.current = self.pages[cls]
    for c, item in self.nav.items():
        on = c is cls
        item.bgcolor = GREEN_D if on else None
        self.nav_text[c].color = "#ffffff" if on else MUTED
    self.title_lbl.value = cls.title
    self.body.content = self.current.root
    self.page.update()
    await self.current.refresh()

def dismiss_alert(self, e=None):
    self.alert.visible = False
    self.page.update()

def note_pending(self, pending):
    """Badge on the Appointments menu item and a banner when a customer books something new."""
    ids = {a["id"] for a in pending}
    new = [a for a in pending if self.seen_pending is not None and a["id"] not in self.seen_pending]
    self.seen_pending = ids
    txt = self.nav_text.get(Appointments)
    if txt is not None:
        txt.value = "Appointments (%d pending)" % len(ids) if ids else "Appointments"
    if new:
        a = new[-1]
        self.alert_txt.value = "? New booking request: %s (%s) · %s on %s %s   [click to dismiss]" % (
            a["pet_name"], a["customer_name"], a["service_name"], a["appt_date"], t12(a["appt_time"]))
```

```
            self.alert.visible = True
        self.page.update()

    async def reminders(self):
        try:
            r = await self.call(self.api.post, "/reminders/send")
        except ApiError as e:
            return await self.fail(e)
        await self.info("PawPet", "Reminders sent: %d" % r["sent"])

    async def logout(self):
        self.session += 1                       # stops the refresh loop
        self.current = None
        await self.call(self.api.logout)
        self.login_view()

    # ================================================================ live updates
    async def ticker(self, gen):
        """Every few seconds: refresh the visible page and look for new booking requests.
        A 1.5 s health check comes first, so a missing/slow server never freezes the window for long."""
        delay = REFRESH_S
        while gen == self.session:
            await asyncio.sleep(delay)
            if gen != self.session:
                return
            delay = REFRESH_S
            try:
                page = self.current
                if page is None or self.open:           # do not disturb an open dialog
                    continue
                if not await self.call(self.api.health):
                    self.offline = True
                    delay = REFRESH_S * 2
                else:
                    self.offline = False
                    if page.auto:
                        await page.refresh(quiet=True)
                    if gen == self.session and not self.offline and not isinstance(page, Dashboard):
                        try:
                            self.note_pending((await self.call(self.api.get, "/dashboard"))["pending_list"])
                        except ApiError as e:
                            await self.fail(e, quiet=True)
                if gen == self.session:
                    self.status_lbl.value = ("Server not reachable, retrying?" if self.offline else
                                             "Auto-refresh on · updated " + datetime.now().strftime("%H:%M:%S"))
                    self.page.update()
            except Exception:                           # keep the loop running
                traceback.print_exc()


async def main(page: ft.Page):
    await App(page).start()


if __name__ == "__main__":
    ft.run(main)
```

LAPTOP1_Desktop/pages.py - Desktop - pages

```
"""PawPet desktop - the pages (Dashboard, Appointments, Customers, Inventory, Batches, Vets, Services, Sales).

Every page has:  build() -> a Flet control,  async refresh(quiet=False)  and a few async button handlers.
Network calls always go through `self.call(...)`, which runs the blocking HTTP request in a worker thread so the
window never freezes."""
from datetime import date, timedelta

import flet as ft

from api_client import ApiError
from ui import (AMBER, BLUE, DAYS, GREEN, LINE, MUTED, PANEL, RED, STATUS_COLOR, TEXT, Table, card, danger, days_until,
                dropdown, field, label, peso, plain, primary, stat_card, t12, today, valid_date)

STATUSES = ["All statuses", "pending", "confirmed", "completed", "cancelled", "no_show"]


def nice(status):
    return status.replace("_", " ")


class Page:
    title = ""
    icon = ft.Icons.CIRCLE
    auto = False            # True = refreshed automatically every few seconds
    admin_only = False

    def __init__(self, app):
```

```
        self.app = app
        self.api = app.api
        self.root = self.build()

    # -- to override
    def build(self):
        return ft.Column([])

    async def refresh(self, quiet=False):
        pass

    # -- helpers
    @property
    def page(self):
        return self.app.page

    async def call(self, fn, *a, **k):
        return await self.app.call(fn, *a, **k)

    def act(self, fn):
        """Wrap an async button handler: API/validation problems become a message instead of a crash."""
        async def handler(e=None):
            try:
                await fn()
            except ApiError as ex:
                await self.app.fail(ex)
            except ValueError as ex:
                await self.app.info("PawPet", str(ex))
        return handler

    def table(self, headings, weights=None):
        return Table(headings, weights).attach(self.page)

    def toolbar(self, *left, right=()):
        return ft.Row([*left, ft.Container(expand=True), *right], spacing=8,
                      vertical_alignment=ft.CrossAxisAlignment.CENTER)

    def buttons(self, *items):
        return ft.Row(list(items), spacing=8, vertical_alignment=ft.CrossAxisAlignment.CENTER)


# Dashboard
class Dashboard(Page):
    title = "Dashboard"
    icon = ft.Icons.DASHBOARD
    auto = True

    def build(self):
        self.cards = {}
        row = []
        for key, title in [("a", "APPOINTMENTS TODAY"), ("p", "PENDING REQUESTS"), ("s", "SALES TODAY"),
                           ("m", "SALES (30 DAYS)"), ("e", "EXPIRED / EXPIRING")]:
            box, val, sub = stat_card(title)
            self.cards[key] = (val, sub)
            row.append(box)
        self.pend = self.table(["Date", "Time", "Pet", "Owner", "Service", "Vet"], [2, 2, 2, 3, 3, 3])
        self.appts = self.table(["Time", "Pet", "Owner", "Service", "Vet", "Status"], [2, 2, 3, 3, 3, 2])
        self.exp = self.table(["Product", "Batch", "Qty", "Expiry", "Status"], [4, 2, 1, 3, 2])
        self.low = self.table(["Product", "Usable stock"], [3, 2])
        self.top = self.table(["Item", "Qty", "Amount"], [4, 1, 3])
        self.pend_rows = {}
        pend_head = self.toolbar(label("Booking requests waiting for confirmation (all dates)", 14, TEXT, True),
                                 right=[primary("Confirm", self.act(lambda: self.decide("confirmed"))),
                                        danger("Decline", self.act(lambda: self.decide("cancelled")))])
        left = ft.Column([
            card(ft.Column([pend_head, ft.Container(self.pend.root, height=190)], spacing=8)),
            card(ft.Column([label("Today's appointments", 14, TEXT, True), ft.Container(self.appts.root, height=190)],
                           spacing=8)),
        ], expand=3, spacing=12)
        right = ft.Column([
            card(ft.Column([label("Medicine expiry alerts", 14, TEXT, True), ft.Container(self.exp.root, height=150)],
                           spacing=8)),
            card(ft.Column([label("Low stock", 14, TEXT, True), ft.Container(self.low.root, height=130)], spacing=8)),
            card(ft.Column([label("Top sellers", 14, TEXT, True), ft.Container(self.top.root, height=130)], spacing=8)),
        ], expand=2, spacing=12)
        return ft.Column([ft.Row(row, spacing=12),
                          ft.Row([left, right], spacing=12, vertical_alignment=ft.CrossAxisAlignment.START)],
                         spacing=12, scroll=ft.ScrollMode.AUTO, expand=True)

    async def decide(self, status):
        aid = self.pend.selected
        if aid is None:
            return await self.app.info("PawPet", "Select a booking request first.")
        a = self.pend_rows.get(aid, {})
        if status == "cancelled" and not await self.app.confirm(
                "Decline the booking of %s (%s)?" % (a.get("pet_name", "?"), a.get("service_name", "?"))):
```

```
            return
        await self.call(self.api.post, "/appointments/%d/status" % aid, {"status": status})
        await self.refresh()

    async def refresh(self, quiet=False):
        try:
            d = await self.call(self.api.get, "/dashboard")
            ap = await self.call(self.api.get, "/appointments", date=today())
        except ApiError as e:
            return await self.app.fail(e, quiet)
        pend = d.get("pending_list", [])
        self.app.note_pending(pend)
        val = self.cards
        val["a"][0].value, val["a"][1].value = str(d["appointments_today"]), "booked for today"
        val["p"][0].value, val["p"][1].value = str(d["pending"]), "need your confirmation"
        val["s"][0].value, val["s"][1].value = peso(d["sales_today"]), "products and services"
        val["m"][0].value, val["m"][1].value = peso(d["sales_30"]), "all recorded sales"
        exp = d.get("expiring", [])
        n_exp = d.get("expired_count", 0)
        val["e"][0].value = "%d / %d" % (n_exp, len(exp) - n_exp)
        val["e"][1].value = "%d expired, %d expiring soon" % (n_exp, len(exp) - n_exp)
        self.pend_rows = {a["id"]: a for a in pend}
        self.pend.set([(a["id"], [a["appt_date"], t12(a["appt_time"]), a["pet_name"], a["customer_name"],
                                  a["service_name"], a["vet_name"]], AMBER) for a in pend])
        self.appts.set([(a["id"], [t12(a["appt_time"]), a["pet_name"], a["customer_name"], a["service_name"],
                                   a["vet_name"], nice(a["status"])], STATUS_COLOR.get(a["status"])) for a in ap])
        self.exp.set([(b["id"], [b["product_name"], b["batch_no"] or "-", str(b["qty"]), b["expiry_date"],
                                 "EXPIRED" if b["status"] == "expired" else "in %d d" % b["days_left"]],
                       RED if b["status"] == "expired" else AMBER) for b in exp])
        self.low.set([(p["id"], [p["name"], "OUT" if p["stock"] == 0 else str(p["stock"])],
                       RED if p["stock"] == 0 else AMBER) for p in d["low_stock"]])
        self.top.set([(i, [t["item_name"], str(t["qty"]), peso(t["amount"])], None)
                      for i, t in enumerate(d["top_items"])])
        self.page.update()


# Appointments
class Appointments(Page):
    title = "Appointments"
    icon = ft.Icons.EVENT
    auto = True

    def build(self):
        self.date = field("Date (blank = all)", "", width=170, on_submit=self.act(self.refresh))
        self.status = dropdown("Status", STATUSES, "All statuses", width=170, on_select=self.act(self.refresh))
        self.tree = self.table(["Date", "Time", "Pet", "Owner", "Service", "Vet", "Status"], [2, 2, 2, 3, 3, 3, 2])
        self.rows = {}
        bar = self.toolbar(
            ft.IconButton(icon=ft.Icons.CHEVRON_LEFT, on_click=self.act(lambda: self.shift(-1)), tooltip="Previous day"
            self.date,
            ft.IconButton(icon=ft.Icons.CHEVRON_RIGHT, on_click=self.act(lambda: self.shift(1)), tooltip="Next day"),
            plain("Today", self.act(self.go_today)), plain("All dates", self.act(self.all_dates)), self.status,
            right=[primary("New appointment", self.act(self.new), ft.Icons.ADD)])
        acts = self.buttons(primary("Confirm", self.act(self.confirm)), primary("Complete visit", self.act(self.complet
                            plain("Schedule follow-up", self.act(self.followup)), plain("No-show", self.act(self.no_sho
                            danger("Cancel", self.act(self.cancel)), plain("Details", self.act(self.details)))
        return ft.Column([bar, ft.Container(self.tree.root, expand=True), acts], expand=True, spacing=10)

    async def shift(self, n):
        try:
            y, m, d = [int(x) for x in self.date.value.split("-")]
            self.date.value = (date(y, m, d) + timedelta(days=n)).strftime("%Y-%m-%d")
        except Exception:
            self.date.value = today()
        await self.refresh()

    async def go_today(self):
        self.date.value = today()
        await self.refresh()

    async def all_dates(self):
        self.date.value = ""
        await self.refresh()

    async def refresh(self, quiet=False):
        d = (self.date.value or "").strip()
        if d and not valid_date(d):
            if quiet:
                return
            return await self.app.info("PawPet", "Date must look like 2026-10-25.")
        st = self.status.value or "All statuses"
        try:
            rows = await self.call(self.api.get, "/appointments", date=d, status="" if st == "All statuses" else st)
        except ApiError as e:
            return await self.app.fail(e, quiet)
        self.rows = {a["id"]: a for a in rows}
```

```
        self.tree.set([(a["id"], [a["appt_date"], t12(a["appt_time"]), a["pet_name"], a["customer_name"],
                                  a["service_name"] + (" (follow-up)" if a["followup_of"] else ""), a["vet_name"],
                                  nice(a["status"])], STATUS_COLOR.get(a["status"])) for a in rows])

    async def pick(self, allowed=None, need=""):
        aid = self.tree.selected
        if aid is None or aid not in self.rows:
            await self.app.info("PawPet", "Select an appointment first.")
            return None
        a = self.rows[aid]
        if allowed and a["status"] not in allowed:
            await self.app.info("PawPet", need or "That action is not available for this status.")
            return None
        return a

    async def _set(self, a, st):
        await self.call(self.api.post, "/appointments/%d/status" % a["id"], {"status": st})
        await self.refresh()

    async def confirm(self):
        a = await self.pick(("pending",), "Only pending appointments can be confirmed.")
        if a:
            await self._set(a, "confirmed")

    async def no_show(self):
        a = await self.pick(("confirmed",), "Only confirmed appointments can be marked no-show.")
        if a:
            await self._set(a, "no_show")

    async def cancel(self):
        a = await self.pick(("pending", "confirmed"), "Only pending or confirmed appointments can be cancelled.")
        if a and await self.app.confirm("Cancel %s's appointment?" % a["pet_name"]):
            await self._set(a, "cancelled")

    async def details(self):
        a = await self.pick()
        if not a:
            return
        await self.app.info("Appointment #%d" % a["id"],
                            "Pet: %s (%s %s)\nOwner: %s  %s\nService: %s - %s\nVet: %s\nWhen: %s %s\nStatus: %s\n\n"
                            "Pet notes: %s\nVisit notes: %s" % (
                                a["pet_name"], a["species"], a["breed"] or "", a["customer_name"],
                                a["customer_phone"] or "", a["service_name"], peso(a["service_price"]), a["vet_name"],
                                a["appt_date"], t12(a["appt_time"]), nice(a["status"]), a["pet_notes"] or "-",
                                a["notes"] or "-"))

    async def complete(self):
        a = await self.pick(("confirmed",), "Confirm the appointment before completing it.")
        if not a:
            return
        products = [p for p in await self.call(self.api.get, "/products") if p["stock"] > 0]
        names = [("", "(none)")] + [(p["id"], "%s (%d left)" % (p["name"], p["stock"])) for p in products]

        async def save(v):
            items = []
            for k in ("p1", "p2"):
                if v[k]:
                    try:
                        qty = int(v[k + "q"] or 1)
                    except ValueError:
                        raise ValueError("Quantity must be a whole number.")
                    if qty < 1:
                        raise ValueError("Quantity must be at least 1.")
                    items.append({"product_id": int(v[k]), "qty": qty})
            r = await self.call(self.api.post, "/appointments/%d/complete" % a["id"],
                                {"notes": v["notes"] or None, "products": items})
            await self.refresh()
            self.app.toast("Visit completed. Total: %s" % peso(r["total"]))
        await self.app.form("Complete visit - %s" % a["pet_name"], [
            ("notes", "Visit notes", "text", "", None),
            ("p1", "Product 1", "combo", "", names), ("p1q", "Qty", "entry", "1", None),
            ("p2", "Product 2", "combo", "", names), ("p2q", "Qty", "entry", "1", None)],
            save, "Complete & record sale")

    async def followup(self):
        a = await self.pick(("completed",), "Follow-ups are scheduled after a visit is completed.")
        if not a:
            return

        async def book(d, t, n):
            await self.call(self.api.post, "/appointments/%d/followup" % a["id"],
                            {"appt_date": d, "appt_time": t, "notes": n or "Follow-up visit"})
            await self.refresh()
        await self.slot_dialog("Schedule follow-up - %s" % a["pet_name"], a["vet_id"], a["service_id"], book)

    # ---- dialog with a date box + free-time dropdown (used by follow-up)
```

```
    async def slot_dialog(self, title, vet_id, service_id, on_book):
        d = field("Date (YYYY-MM-DD)", (date.today() + timedelta(days=1)).strftime("%Y-%m-%d"), width=200)
        t = dropdown("Time", [], None, width=200)
        n = field("Notes", "", width=420)
        err = label("", 12, RED)

        async def show(e=None):
            try:
                if not valid_date(d.value.strip()):
                    raise ValueError("Date must look like 2026-10-25.")
                slots = await self.call(self.api.get, "/availability", vet_id=vet_id, service_id=service_id,
                                        date=d.value.strip())
            except (ApiError, ValueError) as ex:
                err.value = str(ex)
                self.page.update()
                return
            free = [s["time"] for s in slots if s["available"]]
            t.options = [ft.DropdownOption(key=x, text=t12(x)) for x in free]
            t.value = free[0] if free else None
            err.value = "" if free else "No free times for this vet on that date."
            self.page.update()

        async def save():
            if not t.value:
                raise ValueError("Press 'Show times' and pick a time.")
            await on_book(d.value.strip(), t.value, n.value)
        await self.app.custom_form(title, [ft.Row([d, plain("Show times", show)], spacing=8), t, n, err], save, "Book")

    async def new(self):
        customers = [c for c in await self.call(self.api.get, "/customers") if c["active"]]
        pets = await self.call(self.api.get, "/pets", all=1)
        services = await self.call(self.api.get, "/services")
        vets = [v for v in await self.call(self.api.get, "/vets") if v["active"]]
        if not customers or not services or not vets:
            return await self.app.info("PawPet", "You need at least one customer, one service and one active vet first.
        cust = dropdown("Customer", [(c["id"], c["full_name"]) for c in customers], None, width=420)
        pet = dropdown("Pet", [], None, width=420)
        svc = dropdown("Service", [(s["id"], "%s - %s" % (s["name"], peso(s["price"]))) for s in services],
                       services[0]["id"], width=420)
        vet = dropdown("Veterinarian", [(v["id"], v["name"]) for v in vets], vets[0]["id"], width=420)
        d = field("Date (YYYY-MM-DD)", (date.today() + timedelta(days=1)).strftime("%Y-%m-%d"), width=200)
        t = dropdown("Time", [], None, width=200)
        n = field("Notes", "", width=420)
        err = label("", 12, RED)

        def on_customer(e=None):
            mine = [p for p in pets if str(p["owner_id"]) == str(cust.value)]
            pet.options = [ft.DropdownOption(key=str(p["id"]), text="%s (%s)" % (p["name"], p["species"])) for p in mir
            pet.value = str(mine[0]["id"]) if mine else None
            err.value = "" if mine else "This customer has no pets yet."
            self.page.update()
        cust.on_select = on_customer

        async def show(e=None):
            try:
                if not valid_date(d.value.strip()):
                    raise ValueError("Date must look like 2026-10-25.")
                slots = await self.call(self.api.get, "/availability", vet_id=vet.value, service_id=svc.value,
                                        date=d.value.strip())
            except (ApiError, ValueError) as ex:
                err.value = str(ex)
                self.page.update()
                return
            free = [s["time"] for s in slots if s["available"]]
            t.options = [ft.DropdownOption(key=x, text=t12(x)) for x in free]
            t.value = free[0] if free else None
            err.value = "" if free else "No free times for this vet on that date."
            self.page.update()

        async def save():
            if not cust.value or not pet.value:
                raise ValueError("Choose a customer who has a pet.")
            if not t.value:
                raise ValueError("Press 'Show times' and pick a time.")
            await self.call(self.api.post, "/appointments", {
                "customer_id": int(cust.value), "pet_id": int(pet.value), "service_id": int(svc.value),
                "vet_id": int(vet.value), "appt_date": d.value.strip(), "appt_time": t.value, "notes": n.value})
            await self.refresh()
        await self.app.custom_form("New appointment (auto-confirmed)", [
            cust, pet, svc, vet, ft.Row([d, plain("Show times", show)], spacing=8), t, n, err], save, "Book")


# Customers
class Customers(Page):
    title = "Customers"
    icon = ft.Icons.PEOPLE
```

```
    def build(self):
        self.q = field("Search name, username or phone", "", width=340, on_submit=self.act(self.refresh))
        self.tree = self.table(["Name", "Username", "Phone", "Pets", "Visits", "Status"], [4, 3, 3, 1, 1, 2])
        bar = self.toolbar(self.q, plain("Search", self.act(self.refresh), ft.Icons.SEARCH),
                           right=[primary("Add customer", self.act(self.add), ft.Icons.PERSON_ADD)])
        acts = self.buttons(plain("View pets & history", self.act(self.view)),
                            plain("Enable / disable account", self.act(self.toggle)),
                            plain("Reset password", self.act(self.reset_pw)))
        return ft.Column([bar, ft.Container(self.tree.root, expand=True), acts], expand=True, spacing=10)

    async def refresh(self, quiet=False):
        try:
            rows = await self.call(self.api.get, "/customers", q=(self.q.value or "").strip())
        except ApiError as e:
            return await self.app.fail(e, quiet)
        self.names = {c["id"]: c["full_name"] for c in rows}
        self.tree.set([(c["id"], [c["full_name"], c["username"], c["phone"] or "-", str(c["pet_count"]),
                                  str(c["appt_count"]), "Active" if c["active"] else "Disabled"],
                        None if c["active"] else RED) for c in rows])

    async def add(self):
        made = {}

        async def save(v):
            if not v["full_name"].strip() or not v["username"].strip():
                raise ValueError("Name and username are required.")
            made["r"] = await self.call(self.api.post, "/customers", v)
            made["u"] = v["username"].strip()
            await self.refresh()
        await self.app.form("Add customer", [
            ("full_name", "Full name", "entry", "", None), ("username", "Username", "entry", "", None),
            ("phone", "Phone", "entry", "", None), ("email", "Email", "entry", "", None),
            ("password", "Password (blank = make a temporary one)", "entry", "", None)], save)
        if made.get("r", {}).get("temp_password"):          # shown after the form has closed
            await self.app.info("PawPet", "Customer created.\n\nUsername: %s\nTemporary password: %s\n\n"
                                "Give this to the customer. They must choose a new password at first sign-in."
                                % (made["u"], made["r"]["temp_password"]))

    def _need(self):
        cid = self.tree.selected
        return cid

    async def reset_pw(self):
        cid = self._need()
        if cid is None:
            return await self.app.info("PawPet", "Select a customer first.")
        name = self.names.get(cid, "this customer")
        if not await self.app.confirm("Reset the password of %s?\nThey will be signed out everywhere." % name):
            return
        r = await self.call(self.api.post, "/users/%d/reset-password" % cid)
        await self.app.info("PawPet", "Username: %s\nTemporary password: %s\n\nGive this to the customer. "
                            "They must choose a new password at first sign-in." % (r["username"], r["temp_password"]))

    async def view(self):
        cid = self._need()
        if cid is None:
            return await self.app.info("PawPet", "Select a customer first.")
        r = await self.call(self.api.get, "/customers/%d" % cid)
        pets = "\n".join("- %s: %s %s, %s%s" % (p["name"], p["species"], p["breed"] or "",
                                                ("%s yr" % p["age"]) if p["age"] else "age ?",
                                                (" [%s]" % p["notes"]) if p["notes"] else "") for p in r["pets"]) or "No
        hist = "\n".join("- %s %s  %s / %s  [%s]" % (a["appt_date"], t12(a["appt_time"]), a["pet_name"],
                                                     a["service_name"], nice(a["status"]))
                         for a in r["appointments"][:10]) or "None"
        c = r["customer"]
        await self.app.info(c["full_name"], "%s  %s\n\nPETS\n%s\n\nRECENT APPOINTMENTS\n%s" % (
            c["email"] or "", c["phone"] or "", pets, hist))

    async def toggle(self):
        cid = self._need()
        if cid is None:
            return await self.app.info("PawPet", "Select a customer first.")
        await self.call(self.api.post, "/customers/%d/toggle" % cid)
        await self.refresh()


# Inventory
class Inventory(Page):
    title = "Inventory"
    icon = ft.Icons.INVENTORY_2
    auto = True

    def build(self):
        self.q = field("Search products", "", width=300, on_submit=self.act(self.refresh))
        self.tree = self.table(["Product", "Category", "Price", "Usable stock", "Expired", "Next expiry",
```

```
                            "Reorder at", "Status"], [4, 2, 2, 2, 1, 2, 2, 3])
        self.rows = {}
        bar = self.toolbar(self.q, plain("Search", self.act(self.refresh), ft.Icons.SEARCH),
                           right=[primary("Add product", self.act(lambda: self.form(None)), ft.Icons.ADD)])
        acts = self.buttons(plain("Receive new batch", self.act(self.restock)), plain("Walk-in sale", self.act(self.sel
                            plain("Edit", self.act(self.edit)), danger("Delete (admin)", self.act(self.delete)),
                            label("Expired stock is not counted and cannot be sold.", 11, MUTED))
        return ft.Column([bar, ft.Container(self.tree.root, expand=True), acts], expand=True, spacing=10)

    async def refresh(self, quiet=False):
        try:
            rows = await self.call(self.api.get, "/products", q=(self.q.value or "").strip())
        except ApiError as e:
            return await self.app.fail(e, quiet)
        self.rows = {p["id"]: p for p in rows}
        out = []
        for p in rows:
            left = days_until(p.get("next_expiry"))
            if p.get("expired_qty", 0) > 0:
                color, text = RED, "Has expired stock"
            elif p["stock"] == 0:
                color, text = RED, "Out of stock"
            elif left is not None and left <= 30:
                color, text = AMBER, "Expiring soon"
            elif p["stock"] <= p["reorder_level"]:
                color, text = AMBER, "Low stock"
            else:
                color, text = None, "In stock"
            out.append((p["id"], [p["name"], p["category"], peso(p["price"]), str(p["stock"]),
                                  str(p.get("expired_qty", 0) or "-"), p.get("next_expiry") or "-",
                                  str(p["reorder_level"]), text], color))
        self.tree.set(out)

    async def pick(self):
        pid = self.tree.selected
        if pid is None or pid not in self.rows:
            await self.app.info("PawPet", "Select a product first.")
            return None
        return self.rows[pid]

    async def restock(self):
        p = await self.pick()
        if not p:
            return

        async def save(v):
            try:
                qty = int(v["qty"])
            except ValueError:
                raise ValueError("Units must be a whole number.")
            if qty < 1:
                raise ValueError("Units must be at least 1.")
            exp = v["expiry_date"].strip()
            if exp and not valid_date(exp):
                raise ValueError("Expiry date must look like 2027-01-31.")
            await self.call(self.api.post, "/products/%d/restock" % p["id"],
                            {"qty": qty, "batch_no": v["batch_no"].strip(), "expiry_date": exp})
            await self.refresh()
        await self.app.form("Receive new batch: %s" % p["name"], [
            ("qty", "Units received", "entry", "10", None), ("batch_no", "Batch / lot no.", "entry", "", None),
            ("expiry_date", "Expiry date (YYYY-MM-DD)", "entry", "", None)], save, "Add batch")

    async def sell(self):
        p = await self.pick()
        if not p:
            return

        async def save(v):
            try:
                n = int(v["n"])
            except ValueError:
                raise ValueError("Quantity must be a whole number.")
            r = await self.call(self.api.post, "/products/%d/sell" % p["id"], {"qty": n})
            await self.refresh()
            self.app.toast("Sold. Amount: %s" % peso(r["amount"]))
        await self.app.form("Sell %s" % p["name"], [("n", "Quantity", "entry", "1", None)], save, "Sell")

    async def edit(self):
        p = await self.pick()
        if p:
            await self.form(p)

    async def delete(self):
        p = await self.pick()
        if p and await self.app.confirm("Delete %s?" % p["name"]):
            await self.call(self.api.delete, "/products/%d" % p["id"])
```

```
            await self.refresh()

        async def form(self, p):
            async def save(v):
                try:
                    body = {"name": v["name"], "category": v["category"] or "Other", "price": float(v["price"]),
                            "stock": int(v["stock"]), "reorder_level": int(v["reorder_level"] or 5)}
                except ValueError:
                    raise ValueError("Price must be a number; stock and reorder level must be whole numbers.")
                if p:
                    await self.call(self.api.put, "/products/%d" % p["id"], body)
                else:
                    exp = v["expiry_date"].strip()
                    if exp and not valid_date(exp):
                        raise ValueError("Expiry date must look like 2027-01-31.")
                    body["batch_no"] = v["batch_no"].strip()
                    body["expiry_date"] = exp
                    await self.call(self.api.post, "/products", body)
                await self.refresh()
            q = p or {}
            fields = [("name", "Name", "entry", q.get("name", ""), None),
                      ("category", "Category", "entry", q.get("category", "Food"), None),
                      ("price", "Price", "entry", q.get("price", ""), None),
                      ("stock", "Stock", "entry", q.get("stock", 0), None),
                      ("reorder_level", "Reorder level", "entry", q.get("reorder_level", 5), None)]
            if not p:
                fields += [("batch_no", "Batch / lot no. (optional)", "entry", "", None),
                           ("expiry_date", "Expiry date (YYYY-MM-DD, optional)", "entry", "", None)]
            await self.app.form("Edit product" if p else "Add product", fields, save)


    # Batches & Expiry
    class Batches(Page):
        title = "Batches & Expiry"
        icon = ft.Icons.WARNING_AMBER
        auto = True

        FILTERS = ["All batches", "Expired", "Expiring soon (30 days)", "OK"]

        def build(self):
            self.flt = dropdown("Show", self.FILTERS, "All batches", width=240, on_select=self.act(self.refresh))
            self.sum = label("", 12, MUTED)
            self.tree = self.table(["Product", "Batch / lot", "Qty left", "Expiry date", "When", "Status"], [4, 2, 1, 2, 2,
            self.rows = {}
            bar = self.toolbar(self.flt, self.sum)
            acts = ft.Row([danger("Discard selected batch", self.act(self.discard)),
                           label("Discard removes what is left of the batch from stock (for example expired medicine thrown
                                 "away). New batches are added from the Inventory page.", 11, MUTED)],
                          spacing=12, vertical_alignment=ft.CrossAxisAlignment.CENTER)
            return ft.Column([bar, ft.Container(self.tree.root, expand=True), acts], expand=True, spacing=10)

        async def refresh(self, quiet=False):
            try:
                rows = await self.call(self.api.get, "/batches")
            except ApiError as e:
                return await self.app.fail(e, quiet)
            want = {"Expired": ("expired",), "Expiring soon (30 days)": ("expiring",),
                    "OK": ("ok", "no_expiry")}.get(self.flt.value)
            n_exp = sum(1 for b in rows if b["status"] == "expired")
            n_soon = sum(1 for b in rows if b["status"] == "expiring")
            self.sum.value = "%d expired · %d expiring within 30 days · %d batches" % (n_exp, n_soon, len(rows))
            self.rows = {b["id"]: b for b in rows}
            out = []
            for b in rows:
                if want and b["status"] not in want:
                    continue
                if b["status"] == "expired":
                    when, text, color = "%d day(s) ago" % abs(b["days_left"]), "EXPIRED", RED
                elif b["status"] == "expiring":
                    when, text, color = "in %d day(s)" % b["days_left"], "Expiring soon", AMBER
                elif b["status"] == "ok":
                    when, text, color = "in %d day(s)" % b["days_left"], "OK", None
                else:
                    when, text, color = "-", "No expiry", None
                out.append((b["id"], [b["product_name"], b["batch_no"] or "-", str(b["qty"]), b["expiry_date"] or "-",
                                      when, text], color))
            self.tree.set(out)

        async def discard(self):
            bid = self.tree.selected
            if bid is None or bid not in self.rows:
                return await self.app.info("PawPet", "Select a batch first.")
            b = self.rows[bid]
            if not await self.app.confirm("Discard %d unit(s) of %s (batch %s)?\nThey will be removed from stock."
                                          % (b["qty"], b["product_name"], b["batch_no"] or "-")):
                return
```

```
        await self.call(self.api.post, "/batches/%d/discard" % bid)
        await self.refresh()


# Veterinarians
class Vets(Page):
    title = "Veterinarians"
    icon = ft.Icons.MEDICAL_SERVICES

    def build(self):
        self.d = field("Availability on (YYYY-MM-DD)", today(), width=240, on_submit=self.act(self.refresh))
        self.tree = self.table(["Veterinarian", "Specialty", "Working days", "Hours", "Free slots", "Active"], [3, 3, 4
        self.tree.on_change = self.show
        self.detail = label("Select a vet to see free times.", 12, MUTED)
        self.vets, self.slots = {}, {}
        bar = self.toolbar(self.d, plain("Check", self.act(self.refresh)),
                           right=[primary("Add veterinarian", self.act(lambda: self.form(None)), ft.Icons.ADD)])
        acts = ft.Row([plain("Edit selected (admin)", self.act(self.edit))])
        return ft.Column([bar, ft.Container(self.tree.root, expand=True), card(self.detail), acts],
                         expand=True, spacing=10)

    async def refresh(self, quiet=False):
        d = (self.d.value or "").strip()
        if not valid_date(d):
            return await self.app.info("PawPet", "Date must look like 2026-10-25.")
        try:
            vets = await self.call(self.api.get, "/vets")
            slots = {}
            for v in vets:
                slots[v["id"]] = await self.call(self.api.get, "/availability", vet_id=v["id"], date=d)
        except ApiError as e:
            return await self.app.fail(e, quiet)
        self.vets, self.slots = {v["id"]: v for v in vets}, slots
        rows = []
        for v in vets:
            free = len([s for s in slots[v["id"]] if s["available"]])
            rows.append((v["id"], [v["name"], v["specialty"] or "", ", ".join(DAYS[int(x)] for x in v["work_days"].spli
                                   "%s - %s" % (t12(v["start_time"]), t12(v["end_time"])),
                                   str(free) if slots[v["id"]] else "off", "Yes" if v["active"] else "No"], None))
        self.tree.set(rows)
        self.detail.value = "Select a vet to see free times."
        self.page.update()

    def show(self, vid):
        s = self.slots.get(vid, [])
        free = [t12(x["time"]) for x in s if x["available"]]
        busy = [t12(x["time"]) for x in s if not x["available"]]
        if not s:
            self.detail.value = "%s is not working on this date." % self.vets[vid]["name"]
        else:
            self.detail.value = "FREE: %s\nTAKEN: %s" % (", ".join(free) or "none", ", ".join(busy) or "none")

    async def edit(self):
        vid = self.tree.selected
        if vid is None or vid not in self.vets:
            return await self.app.info("PawPet", "Select a veterinarian first.")
        await self.form(self.vets[vid])

    async def form(self, v):
        async def save(x):
            picked = [s.strip().capitalize() for s in x["days"].split(",")]
            days = [str(i) for i, name in enumerate(DAYS) if name in picked]
            if not days:
                raise ValueError("Enter working days like: Mon,Tue,Wed")
            if x["end"] <= x["start"]:
                raise ValueError("End time must be after start time.")
            body = {"name": x["name"], "specialty": x["specialty"], "work_days": ",".join(days),
                    "start_time": x["start"], "end_time": x["end"]}
            if v:
                body["active"] = int(x["active"])
                await self.call(self.api.put, "/vets/%d" % v["id"], body)
            else:
                await self.call(self.api.post, "/vets", body)
            await self.refresh()
        fields = [("name", "Name", "entry", v["name"] if v else "", None),
                  ("specialty", "Specialty", "entry", (v["specialty"] or "") if v else "", None),
                  ("days", "Working days (Mon,Tue,..)", "entry",
                   ",".join(DAYS[int(i)] for i in v["work_days"].split(",")) if v else "Mon,Tue,Wed,Thu,Fri", None),
                  ("start", "Start (HH:MM)", "entry", v["start_time"] if v else "09:00", None),
                  ("end", "End (HH:MM)", "entry", v["end_time"] if v else "17:00", None)]
        if v:
            fields.append(("active", "Active", "check", v["active"], None))
        await self.app.form("Edit veterinarian" if v else "Add veterinarian", fields, save)


# Services (admin only)
```

```
class Services(Page):
    title = "Services"
    icon = ft.Icons.MISCELLANEOUS_SERVICES
    admin_only = True

    def build(self):
        self.tree = self.table(["Service", "Description", "Price", "Duration"], [3, 5, 2, 2])
        self.rows = {}
        bar = self.toolbar(plain("Edit selected", self.act(self.edit)),
                           right=[primary("Add service", self.act(lambda: self.form(None)), ft.Icons.ADD)])
        return ft.Column([bar, ft.Container(self.tree.root, expand=True)], expand=True, spacing=10)

    async def refresh(self, quiet=False):
        try:
            rows = await self.call(self.api.get, "/services")
        except ApiError as e:
            return await self.app.fail(e, quiet)
        self.rows = {s["id"]: s for s in rows}
        self.tree.set([(s["id"], [s["name"], s["description"] or "", peso(s["price"]), "%d min" % s["duration"]], None)
                       for s in rows])

    async def edit(self):
        sid = self.tree.selected
        if sid is None or sid not in self.rows:
            return await self.app.info("PawPet", "Select a service first.")
        await self.form(self.rows[sid])

    async def form(self, s):
        async def save(v):
            try:
                body = {"name": v["name"], "description": v["description"], "price": float(v["price"]),
                        "duration": int(v["duration"])}
            except ValueError:
                raise ValueError("Price must be a number and duration a whole number of minutes.")
            if s:
                await self.call(self.api.put, "/services/%d" % s["id"], body)
            else:
                await self.call(self.api.post, "/services", body)
            await self.refresh()
        q = s or {}
        await self.app.form("Edit service" if s else "Add service", [
            ("name", "Name", "entry", q.get("name", ""), None),
            ("description", "Description", "entry", q.get("description", ""), None),
            ("price", "Price", "entry", q.get("price", ""), None),
            ("duration", "Duration (min)", "entry", q.get("duration", 30), None)], save)


# Sales
class Sales(Page):
    title = "Sales"
    icon = ft.Icons.RECEIPT_LONG

    def build(self):
        self.days = dropdown("Period (days)", ["7", "30", "90"], "30", width=170, on_select=self.act(self.refresh))
        self.total = label("", 14, TEXT, True)
        self.tree = self.table(["Date", "Item", "Type", "Customer", "Qty", "Amount"], [3, 4, 2, 3, 1, 2])
        bar = self.toolbar(self.days, right=[self.total])
        return ft.Column([bar, ft.Container(self.tree.root, expand=True)], expand=True, spacing=10)

    async def refresh(self, quiet=False):
        try:
            rows = await self.call(self.api.get, "/sales", days=self.days.value or "30")
        except ApiError as e:
            return await self.app.fail(e, quiet)
        self.tree.set([(i, [s["created_at"][:16], s["item_name"], s["kind"], s["customer_name"] or "Walk-in",
                            str(s["qty"]), peso(s["amount"])], None) for i, s in enumerate(rows)])
        prod = sum(s["amount"] for s in rows if s["kind"] == "product")
        tot = sum(s["amount"] for s in rows)
        self.total.value = "Total %s   (products %s, services %s)" % (peso(tot), peso(prod), peso(tot - prod))
        self.page.update()


ALL_PAGES = [Dashboard, Appointments, Customers, Inventory, Batches, Vets, Services, Sales]
```

LAPTOP1_Desktop/ui.py - Desktop - UI helpers

```
"""Small UI helpers for the desktop app: colors, formatting, cards, buttons and the table."""
from datetime import date

import flet as ft

# theme
BG, SIDEBAR, PANEL, PANEL2, LINE = "#0f1a15", "#0b140f", "#16241d", "#1b2d24", "#27402f"
TEXT, MUTED = "#e8f3ec", "#8fa89a"
GREEN, GREEN_D, AMBER, RED, BLUE = "#2fb36d", "#1f8a52", "#f0a94b", "#e5645a", "#4aa8d8"
DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
```

```
STATUS_COLOR = {"pending": AMBER, "confirmed": BLUE, "completed": GREEN, "cancelled": RED, "no_show": RED,
                "low": AMBER, "out": RED}


# formatting helpers
def peso(n):
    return "PHP {:,.2f}".format(float(n or 0))


def t12(t):
    h, m = [int(x) for x in t.split(":")]
    return "%d:%02d %s" % ((h + 11) % 12 + 1, m, "PM" if h >= 12 else "AM")


def today():
    return date.today().strftime("%Y-%m-%d")


def valid_date(s):
    try:
        y, m, d = [int(x) for x in s.split("-")]
        date(y, m, d)
        return True
    except Exception:
        return False


def days_until(s):
    """Days until a date (negative = already past), None if blank."""
    if not s:
        return None
    try:
        y, m, d = [int(x) for x in s.split("-")]
        return (date(y, m, d) - date.today()).days
    except Exception:
        return None


# small widgets
def label(text, size=13, color=TEXT, bold=False, **kw):
    return ft.Text(str(text), size=size, color=color, weight=ft.FontWeight.BOLD if bold else None, **kw)


def card(content, padding=16, expand=False, width=None):
    return ft.Container(content=content, bgcolor=PANEL, border_radius=ft.BorderRadius.all(14),
                        border=ft.Border.all(1, LINE), padding=ft.Padding.all(padding), expand=expand, width=width)


def stat_card(title):
    """Returns (box, value text, small text) so the page can change the numbers later."""
    value = label("-", 24, TEXT, True)
    sub = label("", 11, MUTED)
    box = card(ft.Column([label(title, 10, MUTED, True), value, sub], spacing=4, tight=True), expand=True)
    return box, value, sub


def primary(text, on_click, icon=None):
    return ft.FilledButton(content=text, icon=icon, on_click=on_click,
                           style=ft.ButtonStyle(bgcolor=GREEN, color="#06210f"))


def danger(text, on_click, icon=None):
    return ft.FilledButton(content=text, icon=icon, on_click=on_click,
                           style=ft.ButtonStyle(bgcolor=RED, color="#2b0907"))


def plain(text, on_click, icon=None):
    return ft.OutlinedButton(content=text, icon=icon, on_click=on_click,
                             style=ft.ButtonStyle(color=TEXT, side=ft.BorderSide(1, LINE)))


def field(text="", value="", width=None, password=False, on_submit=None, expand=False, autofocus=False):
    return ft.TextField(label=text, value="" if value is None else str(value), width=width, password=password,
                        can_reveal_password=password, on_submit=on_submit, expand=expand, autofocus=autofocus,
                        border_color=LINE, filled=True, bgcolor=PANEL2, border_radius=ft.BorderRadius.all(10),
                        content_padding=ft.Padding.symmetric(vertical=10, horizontal=12))


def dropdown(text, options, value=None, width=None, on_select=None):
    """options: a list of strings or (key, text) pairs."""
    opts = []
    for o in options:
        k, t = (o if isinstance(o, (tuple, list)) else (o, o))
        opts.append(ft.DropdownOption(key=str(k), text=str(t)))
    return ft.Dropdown(label=text, options=opts, value=None if value is None else str(value), width=width,
```

```
                    on_select=on_select, border_color=LINE, filled=True, bgcolor=PANEL2,
                    border_radius=ft.BorderRadius.all(10))


# table
class Table:
    """Table made of rows you can click. Columns share the width by weight, so it always fills the window.

    set(rows) takes a list of (row_id, [text for each column], color or None)."""

    def __init__(self, headings, weights=None, on_change=None):
        self.weights = weights or [1] * len(headings)
        self.on_change = on_change
        self.selected = None
        self.data = []
        self.rows = {}
        self.page = None
        head = ft.Container(
            content=ft.Row([ft.Container(label(h, 11, MUTED, True), expand=w) for h, w in zip(headings, self.weights)],
                           spacing=10),
            bgcolor=PANEL2, padding=ft.Padding.symmetric(vertical=10, horizontal=14),
            border_radius=ft.BorderRadius.all(8))
        self.list = ft.ListView(controls=[], expand=True, spacing=0)
        self.empty = label("Nothing to show.", 12, MUTED)
        self.root = ft.Column([head, self.list], expand=True, spacing=4)

    def attach(self, page):
        self.page = page
        return self

    def set(self, rows):
        self.data = rows
        if self.selected not in [r[0] for r in rows]:
            self.selected = None
        self.rows = {r[0]: self._row(r) for r in rows}
        self.list.controls = list(self.rows.values()) or [self.empty]
        if self.page is not None:
            self.page.update()

    def _row(self, r):
        rid, cells, color = r
        texts = [ft.Container(ft.Text(str(c), size=12, color=color or TEXT, max_lines=1, overflow=ft.TextOverflow.ELLIP
                              expand=w) for c, w in zip(cells, self.weights)]
        return ft.Container(
            content=ft.Row(texts, spacing=10), padding=ft.Padding.symmetric(vertical=11, horizontal=14),
            bgcolor=GREEN_D if rid == self.selected else None, border=ft.Border.only(bottom=ft.BorderSide(1, LINE)),
            ink=True, on_click=lambda e, i=rid: self.select(i))

    def select(self, rid):
        self.selected = rid
        for i, row in self.rows.items():
            row.bgcolor = GREEN_D if i == rid else None
        if self.on_change:
            self.on_change(rid)
        if self.page is not None:
            self.page.update()
```

PHONE_Mobile/src/main.ts

```
import 'zone.js';
import { provideZoneChangeDetection } from '@angular/core';
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular';

import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';
import { authInterceptor } from './app/services/auth.interceptor';

bootstrapApplication(AppComponent, {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
});
```

PHONE_Mobile/src/app/config.ts

```
// Address of the PawPet server.
// - Browser on the same PC:      http://localhost:5000/api
// - Real phone / emulator:       http://<YOUR-PC-IP>:5000/api   (example: http://192.168.1.10:5000/api)
```

```
// Use the SERVER laptop's LAN IP (run ipconfig on it), NOT localhost, when testing on a phone.
export const API_URL = 'http://localhost:5000/api';
```

PHONE_Mobile/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { authGuard } from './guards/auth.guard';

export const routes: Routes = [
  { path: '', redirectTo: 'tabs/home', pathMatch: 'full' },
  { path: 'login', loadComponent: () => import('./pages/login/login.page').then(m => m.LoginPage) },
  { path: 'book', canActivate: [authGuard], loadComponent: () => import('./pages/book/book.page').then(m => m.BookPage)
  {
    path: 'tabs',
    canActivate: [authGuard],
    loadComponent: () => import('./tabs/tabs.page').then(m => m.TabsPage),
    children: [
      { path: 'home', loadComponent: () => import('./pages/home/home.page').then(m => m.HomePage) },
      { path: 'shop', loadComponent: () => import('./pages/shop/shop.page').then(m => m.ShopPage) },
      { path: 'appointments', loadComponent: () => import('./pages/appointments/appointments.page').then(m => m.Appoint
      { path: 'notifications', loadComponent: () => import('./pages/notifications/notifications.page').then(m => m.Noti
      { path: 'profile', loadComponent: () => import('./pages/profile/profile.page').then(m => m.ProfilePage) },
      { path: '', redirectTo: 'home', pathMatch: 'full' },
    ],
  },
  { path: '**', redirectTo: 'tabs/home' },
];
```

PHONE_Mobile/src/app/services/api.service.ts

```
import { Injectable, inject, signal } from '@angular/core';
import { HttpClient, HttpErrorResponse } from '@angular/common/http';
import { firstValueFrom, Observable, TimeoutError, timeout } from 'rxjs';
import { API_URL } from '../config';

export interface User { id: number; username: string; full_name: string; email: string; phone: string; role: string; mu
export interface Pet { id: number; owner_id: number; name: string; species: string; breed: string; age: number | null;
export interface Product { id: number; name: string; category: string; price: number; stock: number; reorder_level: num
export interface Service { id: number; name: string; description: string; price: number; duration: number; emoji: strin
export interface Vet { id: number; name: string; specialty: string; work_days: string; start_time: string; end_time: st
export interface Slot { time: string; available: boolean; }
export interface Appointment {
  id: number; appt_date: string; appt_time: string; status: string; notes: string;
  pet_name: string; species: string; service_name: string; service_price: number; service_emoji: string; vet_name: stri
}
export interface AppNotification { id: number; title: string; body: string; is_read: number; created_at: string; }

const REQUEST_TIMEOUT_MS = 15000;   // a stuck connection shows an error instead of spinning forever

@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  user = signal<User | null>(JSON.parse(localStorage.getItem('pp_user') || 'null'));
  unread = signal(0);

  get token(): string { return localStorage.getItem('pp_token') || ''; }
  isLoggedIn(): boolean { return !!this.token; }

  private async call<T>(req: Observable<T>): Promise<T> {
    try {
      return await firstValueFrom(req.pipe(timeout(REQUEST_TIMEOUT_MS)));
    } catch (e) {
      if (e instanceof TimeoutError || (e as Error)?.name === 'TimeoutError') {
        throw new Error('The server is taking too long to answer. Check your connection and try again.');
      }
      const err = e as HttpErrorResponse;
      if (err.status === 0) throw new Error('Cannot reach the PawPet server. Check your connection and the server addre
      if (err.status === 413) throw new Error('That is too much text. Please shorten it.');
      if (err.status === 429 && !err.error?.error) throw new Error('Too many attempts. Please wait a few minutes and tr
      throw new Error(err.error?.error || 'Something went wrong. Please try again.');
    }
  }

  // ---- auth
  async login(username: string, password: string) {
    const r = await this.call(this.http.post<{ token: string; user: User }>(`${API_URL}/login`, { username, password })
    localStorage.setItem('pp_token', r.token);
    localStorage.setItem('pp_user', JSON.stringify(r.user));
    this.user.set(r.user);
  }
  register(body: { full_name: string; username: string; password: string; email: string; phone: string }) {
    return this.call(this.http.post(`${API_URL}/register`, body));
  }
  async changePassword(oldPassword: string, newPassword: string) {
    await this.call(this.http.post(`${API_URL}/change-password`, { old_password: oldPassword, new_password: newPasswor
```

```
    const u = this.user();
    if (u) { u.must_change = false; localStorage.setItem('pp_user', JSON.stringify(u)); this.user.set({ ...u }); }
  }
  /** Forget everything and reload the app, so the next person never sees the previous person's data. */
  hardReset() {
    this.clearSession();
    window.location.replace(new URL('login', document.baseURI).toString());
  }
  clearSession() {
    localStorage.removeItem('pp_token');
    localStorage.removeItem('pp_user');
    this.user.set(null);
    this.unread.set(0);
  }
  async logout() {
    try { await this.call(this.http.post(`${API_URL}/logout`, {})); } catch { /* ignore */ }
    this.clearSession();
  }

  // ---- catalog
  products(q = '') { return this.call(this.http.get<Product[]>(`${API_URL}/products`, { params: q ? { q } : {} })); }
  services() { return this.call(this.http.get<Service[]>(`${API_URL}/services`)); }
  vets() { return this.call(this.http.get<Vet[]>(`${API_URL}/vets`)); }
  availability(vetId: number, serviceId: number, date: string) {
    return this.call(this.http.get<Slot[]>(`${API_URL}/availability`, { params: { vet_id: vetId, service_id: serviceId,
  }

  // ---- pets
  pets() { return this.call(this.http.get<Pet[]>(`${API_URL}/pets`)); }
  addPet(p: Partial<Pet>) { return this.call(this.http.post(`${API_URL}/pets`, p)); }
  deletePet(id: number) { return this.call(this.http.delete(`${API_URL}/pets/${id}`)); }

  // ---- appointments
  appointments() { return this.call(this.http.get<Appointment[]>(`${API_URL}/appointments`)); }
  book(body: { pet_id: number; service_id: number; vet_id: number; appt_date: string; appt_time: string; notes: string
    return this.call(this.http.post<{ id: number }>(`${API_URL}/appointments`, body));
  }
  cancel(id: number) { return this.call(this.http.post(`${API_URL}/appointments/${id}/cancel`, {})); }

  // ---- notifications
  async notifications() {
    const l = await this.call(this.http.get<AppNotification[]>(`${API_URL}/notifications`));
    this.unread.set(l.filter(n => !n.is_read).length);
    return l;
  }
  async markRead() {
    await this.call(this.http.post(`${API_URL}/notifications/read`, {}));
    this.unread.set(0);
  }
}

// ---- small helpers shared by pages
export function localDate(d: Date): string {
  return d.getFullYear() + '-' + String(d.getMonth() + 1).padStart(2, '0') + '-' + String(d.getDate()).padStart(2, '0')
}
export function t12(t: string): string {
  const [h, m] = t.split(':').map(Number);
  return ((h + 11) % 12 + 1) + ':' + String(m).padStart(2, '0') + ' ' + (h >= 12 ? 'PM' : 'AM');
}
export function peso(n: number): string {
  return '?' + Number(n || 0).toLocaleString('en-PH', { minimumFractionDigits: 2, maximumFractionDigits: 2 });
}
export function prettyDate(iso: string): string {
  const d = new Date(iso + 'T00:00:00');
  return d.toLocaleDateString('en-US', { weekday: 'short', month: 'short', day: 'numeric' });
}

export function passwordProblem(p: string): string {
  if (p.length < 8) return 'Password must be at least 8 characters.';
  if (p.length > 128) return 'Password is too long.';
  if (!/[A-Za-z]/.test(p) || !/\d/.test(p)) return 'Password needs at least one letter and one number.';
  if (['admin123', 'staff123', 'pass123'].includes(p)) return 'Choose a password that is not a common or default one.';
  return '';
}
```

PHONE_Mobile/src/app/services/auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { ApiService } from './api.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const api = inject(ApiService);
  const token = api.token;
  const authed = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
```

```
  return next(authed).pipe(
    catchError(err => {
      // an expired token on a protected call sends the user back to login
      if (err.status === 401 && token && !req.url.endsWith('/login')) {
        api.hardReset();
      }
      return throwError(() => err);
    })
  );
};
```

PHONE_Mobile/src/app/guards/auth.guard.ts

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { ApiService } from '../services/api.service';

export const authGuard: CanActivateFn = () => {
  const api = inject(ApiService);
  return api.isLoggedIn() ? true : inject(Router).parseUrl('/login');
};
```

PHONE_Mobile/src/app/tabs/tabs.page.ts

```
import { Component, inject } from '@angular/core';
import { IonTabs, IonTabBar, IonTabButton, IonIcon, IonLabel, IonBadge } from '@ionic/angular';
import { addIcons } from 'ionicons';
import { homeOutline, storefrontOutline, calendarOutline, notificationsOutline, personOutline } from 'ionicons/icons';
import { ApiService } from '../services/api.service';

@Component({
  selector: 'app-tabs',
  imports: [IonTabs, IonTabBar, IonTabButton, IonIcon, IonLabel, IonBadge],
  template: `
  <ion-tabs>
    <ion-tab-bar slot="bottom">
      <ion-tab-button tab="home"><ion-icon name="home-outline"></ion-icon><ion-label>Home</ion-label></ion-tab-button>
      <ion-tab-button tab="shop"><ion-icon name="storefront-outline"></ion-icon><ion-label>Shop</ion-label></ion-tab-bu
      <ion-tab-button tab="appointments"><ion-icon name="calendar-outline"></ion-icon><ion-label>Visits</ion-label></io
      <ion-tab-button tab="notifications">
        <ion-icon name="notifications-outline"></ion-icon><ion-label>Alerts</ion-label>
        @if (api.unread() > 0) { <ion-badge color="secondary">{{ api.unread() }}</ion-badge> }
      </ion-tab-button>
      <ion-tab-button tab="profile"><ion-icon name="person-outline"></ion-icon><ion-label>Profile</ion-label></ion-tab-
    </ion-tab-bar>
  </ion-tabs>`,
})
export class TabsPage {
  api = inject(ApiService);
  constructor() {
    addIcons({ homeOutline, storefrontOutline, calendarOutline, notificationsOutline, personOutline });
  }
}
```

PHONE_Mobile/src/app/pages/login/login.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { IonContent, IonButton, IonItem, IonInput, IonSegment, IonSegmentButton, IonLabel, IonSpinner } from '@ionic/an
import { ApiService, passwordProblem } from '../../services/api.service';

@Component({
  selector: 'app-login',
  imports: [FormsModule, IonContent, IonButton, IonItem, IonInput, IonSegment, IonSegmentButton, IonLabel, IonSpinner],
  template: `
  <ion-content>
    <div class="top">
      <div class="logo">?</div>
      <h1>PawPet</h1>
      <p>Care for your pets, one tap away.</p>
    </div>
    <div class="page">
      @if (mode !== 'change') {
        <ion-segment [value]="mode" (ionChange)="mode = $any($event.detail.value); error = ''">
          <ion-segment-button value="login"><ion-label>Sign in</ion-label></ion-segment-button>
          <ion-segment-button value="register"><ion-label>Create account</ion-label></ion-segment-button>
        </ion-segment>
      }

      @if (mode === 'change') {
        <div style="margin-top:18px">
          <h3 style="margin:0 0 6px">Choose a new password</h3>
          <p class="hint" style="text-align:left;margin:0 0 12px">Your password was set by the clinic. Please pick your
          <ion-item class="field"><ion-input label="New password" labelPlacement="floating" type="password" autocomplet
```

```
          <ion-item class="field"><ion-input label="Repeat new password" labelPlacement="floating" type="password" auto
          @if (error) { <div class="err">{{ error }}</div> }
          <ion-button class="pp" expand="block" (click)="submit()" [disabled]="busy">
            @if (busy) { <ion-spinner name="dots"></ion-spinner> } @else { Save password }
          </ion-button>
        </div>
      } @else {
      <div style="margin-top:18px">
        @if (mode === 'register') {
          <ion-item class="field"><ion-input label="Full name" labelPlacement="floating" maxlength="100" [(ngModel)]="f
          <ion-item class="field"><ion-input label="Phone" labelPlacement="floating" type="tel" maxlength="30" [(ngMode
          <ion-item class="field"><ion-input label="Email" labelPlacement="floating" type="email" maxlength="100" [(ngM
        }
        <ion-item class="field"><ion-input label="Username" labelPlacement="floating" autocomplete="username" maxlength
        <ion-item class="field"><ion-input label="Password" labelPlacement="floating" type="password" [attr.autocomplet
        @if (error) { <div class="err">{{ error }}</div> }
        <ion-button class="pp" expand="block" (click)="submit()" [disabled]="busy">
          @if (busy) { <ion-spinner name="dots"></ion-spinner> } @else { {{ mode === 'login' ? 'Sign in' : 'Create acco
        </ion-button>
        @if (mode === 'login') { <p class="hint">Forgot your password? Ask the clinic to reset it for you.</p> }
        @else { <p class="hint">Password: 8+ characters with letters and numbers.</p> }
      </div>
      }
    </div>
  </ion-content>`,
  styles: [`
    .top { background: linear-gradient(160deg, #0f3b24, #1f5a3a); color: #fff; text-align: center; padding: 56px 20px 4
    .logo { font-size: 48px; } h1 { margin: 6px 0 4px; font-size: 32px; } p { margin: 0; color: #bfe3cd; }
    .hint { text-align: center; color: var(--pp-muted); font-size: 13px; margin-top: 14px; }
  `],
})
export class LoginPage {
  private api = inject(ApiService);
  private router = inject(Router);
  private cdr = inject(ChangeDetectorRef);
  mode: 'login' | 'register' | 'change' = 'login';
  newPassword = ''; newPassword2 = '';
  username = ''; password = ''; full_name = ''; phone = ''; email = '';
  error = ''; busy = false;

  async submit() {
    this.error = '';
    if (this.mode === 'change') return this.saveNewPassword();
    if (!this.username.trim() || !this.password) { this.error = 'Enter your username and password.'; return; }
    if (this.mode === 'register') {
      if (!this.full_name.trim()) { this.error = 'Enter your full name.'; return; }
      if (!/^[A-Za-z0-9_.\-]{3,50}$/.test(this.username.trim())) { this.error = 'Username: 3-50 letters, numbers, dot,
      if (this.email.trim() && !/^[^@\s]+@[^@\s]+\.[^@\s]{2,}$/.test(this.email.trim())) { this.error = 'Enter a valid
      if (this.phone.trim() && !/^[0-9+()\- ]{0,30}$/.test(this.phone.trim())) { this.error = 'Phone can only have digi
      const pwErr = passwordProblem(this.password);
      if (pwErr) { this.error = pwErr; return; }
    }
    this.busy = true;
    this.cdr.detectChanges();
    try {
      if (this.mode === 'register') {
        await this.api.register({ full_name: this.full_name.trim(), username: this.username.trim(), password: this.pass
      }
      await this.api.login(this.username.trim(), this.password);
      this.oldPassword = this.password;
      this.password = '';
      if (this.api.user()?.must_change) { this.mode = 'change'; return; }
      await this.router.navigateByUrl('/tabs/home', { replaceUrl: true });
    } catch (e) {
      this.error = (e as Error).message || 'Something went wrong. Is the server running?';
      if (this.mode === 'register' && /already taken/i.test(this.error)) this.error += ' Try another one, or switch to
    } finally {
      this.busy = false;
      this.cdr.detectChanges();
    }
  }

  private oldPassword = '';

  private async saveNewPassword() {
    if (this.newPassword !== this.newPassword2) { this.error = 'The two passwords do not match.'; return; }
    const pwErr = passwordProblem(this.newPassword);
    if (pwErr) { this.error = pwErr; return; }
    this.busy = true;
    this.cdr.detectChanges();
    try {
      await this.api.changePassword(this.oldPassword, this.newPassword);
      this.oldPassword = this.newPassword = this.newPassword2 = '';
      await this.router.navigateByUrl('/tabs/home', { replaceUrl: true });
    } catch (e) {
      this.error = (e as Error).message;
```

```
    } finally {
      this.busy = false;
      this.cdr.detectChanges();
    }
  }
}
```

PHONE_Mobile/src/app/pages/home/home.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { Router, RouterLink } from '@angular/router';
import { IonContent, IonRefresher, IonRefresherContent, IonButton } from '@ionic/angular';
import { ApiService, Appointment, Pet, Product, localDate, t12, prettyDate, peso } from '../../services/api.service';

@Component({
  selector: 'app-home',
  imports: [RouterLink, IonContent, IonRefresher, IonRefresherContent, IonButton],
  template: `
  <ion-content>
    <ion-refresher slot="fixed" (ionRefresh)="refresh($event)"><ion-refresher-content></ion-refresher-content></ion-ref
    <div class="page">
      <div class="row">
        <div class="grow">
          <h1 class="hello">{{ greeting }},<br>{{ firstName }}</h1>
          <p class="sub">How can we help today?</p>
        </div>
        <div class="faces">
          @for (p of pets.slice(0, 3); track p.id) { <div class="avatar">{{ p.species === 'Cat' ? '?' : '?' }}</div> }
        </div>
      </div>

      @if (error) { <div class="err">{{ error }}</div> }

      @if (next) {
        <div class="hero-card">
          <div class="row">
            <div class="grow">
              <div>{{ next.pet_name }}'s visit<br>{{ next.service_name }} with {{ next.vet_name }}</div>
              <div class="when">{{ date(next.appt_date) }} · {{ time(next.appt_time) }}</div>
              <a class="link" routerLink="/tabs/appointments">View appointment ?</a>
            </div>
            <div class="datebox"><small>{{ month(next.appt_date) }}</small><b>{{ day(next.appt_date) }}</b></div>
          </div>
        </div>
      } @else {
        <div class="hero-card">
          <div>No upcoming visits</div>
          <div class="when">Book your pet's next checkup</div>
        </div>
      }
      <div class="row" style="margin-top:12px">
        <ion-button class="pp grow" routerLink="/book">? Book new</ion-button>
        <ion-button class="pp grow" fill="outline" routerLink="/tabs/appointments">History</ion-button>
      </div>

      <h3 class="sec">What do you need?</h3>
      <div class="row" style="align-items:stretch">
        <a class="tile grow" routerLink="/tabs/shop"><b>Shop</b><span>Food, meds & more ?</span><span class="big">??</s
        <a class="tile grow" (click)="goServices()"><b>Services</b><span>Checkups, grooming ?</span><span class="big">?
      </div>

      <h3 class="sec">Your pets</h3>
      @if (pets.length) {
        @for (p of pets; track p.id) {
          <div class="card row">
            <div class="avatar">{{ p.species === 'Cat' ? '?' : '?' }}</div>
            <div class="grow"><b>{{ p.name }}</b><div class="sub" style="margin:0">{{ p.breed || p.species }}@if (p.age
          </div>
        }
      } @else {
        <div class="card empty">Add your pet in the Profile tab so you can book visits.</div>
      }

      <h3 class="sec">Running low in the shop</h3>
      @for (p of lowStock; track p.id) {
        <div class="card row"><div class="emoji">{{ p.emoji }}</div><div class="grow"><b>{{ p.name }}</b><div class="su
          <span class="badge" [class.out]="p.stock === 0" [class.low]="p.stock > 0">{{ p.stock === 0 ? 'Out of stock' :
      } @empty { <div class="card empty">All items are well stocked.</div> }
    </div>
  </ion-content>`,
  styles: [`.faces { display: flex; gap: -8px; } .faces .avatar { margin-left: -10px; border: 2px solid #fff; } a.tile
})
export class HomePage {
  private api = inject(ApiService);
  private cdr = inject(ChangeDetectorRef);
  private router = inject(Router);
```

```
  next: Appointment | null = null;
  pets: Pet[] = [];
  lowStock: Product[] = [];
  error = '';

  get firstName() { return (this.api.user()?.full_name || '').split(' ')[0]; }
  get greeting() { const h = new Date().getHours(); return h < 12 ? 'Good morning' : h < 18 ? 'Good afternoon' : 'Good
  date = prettyDate; time = t12; money = peso;
  month(d: string) { return new Date(d + 'T00:00:00').toLocaleDateString('en-US', { month: 'short' }); }
  day(d: string) { return new Date(d + 'T00:00:00').getDate(); }
  goServices() { this.router.navigate(['/tabs/shop'], { queryParams: { tab: 'services' } }); }

  private entered = false;
  ngOnInit() { this.entered = true; return this.load(); }
  ionViewWillEnter() { if (this.entered) { this.entered = false; return; } return this.load(); }
  async refresh(ev: CustomEvent) { await this.load(); ev.detail.complete(); }

  private async load() {
    try { await this.loadInner(); } finally { this.cdr.detectChanges(); }
  }

  private async loadInner() {
    this.error = '';
    try {
      const [appts, pets, products] = await Promise.all([this.api.appointments(), this.api.pets(), this.api.products()]
      const today = localDate(new Date());
      this.next = appts.filter(a => ['pending', 'confirmed'].includes(a.status) && a.appt_date >= today)
        .sort((a, b) => (a.appt_date + a.appt_time).localeCompare(b.appt_date + b.appt_time))[0] || null;
      this.pets = pets;
      this.lowStock = products.filter(p => p.stock <= p.reorder_level).slice(0, 3);
      this.api.notifications().catch(() => undefined);
    } catch (e) { this.error = (e as Error).message; }
  }
}
```

PHONE_Mobile/src/app/pages/shop/shop.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { IonContent, IonSegment, IonSegmentButton, IonLabel, IonSearchbar, IonRefresher, IonRefresherContent, IonButtor
import { ApiService, Product, Service, peso } from '../../services/api.service';

@Component({
  selector: 'app-shop',
  imports: [FormsModule, IonContent, IonSegment, IonSegmentButton, IonLabel, IonSearchbar, IonRefresher, IonRefresherCo
  template: `
  <ion-content>
    <ion-refresher slot="fixed" (ionRefresh)="refresh($event)"><ion-refresher-content></ion-refresher-content></ion-ref
    <div class="page">
      <h1 class="hello">Shop & services</h1>
      <p class="sub">Check prices and live stock before you visit.</p>
      <ion-segment [value]="tab" (ionChange)="tab = $any($event.detail.value)">
        <ion-segment-button value="products"><ion-label>Products</ion-label></ion-segment-button>
        <ion-segment-button value="services"><ion-label>Services</ion-label></ion-segment-button>
      </ion-segment>
      @if (error) { <div class="err">{{ error }}</div> }

      @if (tab === 'products') {
        <ion-searchbar [(ngModel)]="search" placeholder="Search products" [debounce]="200"></ion-searchbar>
        <div style="margin-bottom:8px">
          @for (c of categories; track c) { <span class="chip" [class.on]="cat === c" (click)="cat = c">{{ c }}</span>
        </div>
        @for (p of filtered; track p.id) {
          <div class="card row">
            <div class="emoji">{{ p.emoji }}</div>
            <div class="grow"><b>{{ p.name }}</b><div class="sub" style="margin:2px 0 0">{{ p.category }} · {{ money(p.
            <span class="badge" [class.ok]="p.stock > p.reorder_level" [class.low]="p.stock > 0 && p.stock <= p.reorder
              {{ p.stock === 0 ? 'Out of stock' : p.stock <= p.reorder_level ? 'Only ' + p.stock + ' left' : 'In stock'
            </span>
          </div>
        } @empty { <div class="empty"><span class="big">?</span>No products found.</div> }
      } @else {
        @for (s of services; track s.id) {
          <div class="card">
            <div class="row">
              <div class="emoji">{{ s.emoji }}</div>
              <div class="grow"><b>{{ s.name }}</b><div class="sub" style="margin:2px 0 0">{{ s.description }}</div></d
            </div>
            <div class="row" style="margin-top:10px">
              <div class="grow"><b>{{ money(s.price) }}</b> <span class="sub">· {{ s.duration }} min</span></div>
              <ion-button size="small" (click)="book(s.id)">Book</ion-button>
            </div>
          </div>
        } @empty { <div class="empty">No services available.</div> }
      }
```

```
    </div>
  </ion-content>`,
})
export class ShopPage {
  private api = inject(ApiService);
  private cdr = inject(ChangeDetectorRef);
  private router = inject(Router);
  private route = inject(ActivatedRoute);
  tab: 'products' | 'services' = 'products';
  products: Product[] = [];
  services: Service[] = [];
  search = ''; cat = 'All'; error = '';
  money = peso;

  get categories() { return ['All', ...Array.from(new Set(this.products.map(p => p.category)))]; }
  get filtered() {
    const s = this.search.trim().toLowerCase();
    return this.products.filter(p => (this.cat === 'All' || p.category === this.cat) && (!s || p.name.toLowerCase().inc
  }

  private entered = false;
  ngOnInit() { this.entered = true; return this.enter(); }
  ionViewWillEnter() { if (this.entered) { this.entered = false; return; } return this.enter(); }
  private enter() {
    if (this.route.snapshot.queryParamMap.get('tab') === 'services') this.tab = 'services';
    return this.load();
  }
  async refresh(ev: CustomEvent) { await this.load(); ev.detail.complete(); }
  book(id: number) { this.router.navigate(['/book'], { queryParams: { service: id } }); }

  private async load() {
    try { await this.loadInner(); } finally { this.cdr.detectChanges(); }
  }

  private async loadInner() {
    this.error = '';
    try { [this.products, this.services] = await Promise.all([this.api.products(), this.api.services()]); }
    catch (e) { this.error = (e as Error).message; }
  }
}
```

PHONE_Mobile/src/app/pages/book/book.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { IonHeader, IonToolbar, IonTitle, IonButtons, IonBackButton, IonContent, IonButton, IonItem, IonInput, IonSpinn
import { ApiService, Pet, Service, Vet, Slot, localDate, t12, peso } from '../../services/api.service';

@Component({
  selector: 'app-book',
  imports: [FormsModule, RouterLink, IonHeader, IonToolbar, IonTitle, IonButtons, IonBackButton, IonContent, IonButton,
  template: `
  <ion-header class="ion-no-border">
    <ion-toolbar><ion-buttons slot="start"><ion-back-button defaultHref="/tabs/home" text=""></ion-back-button></ion-bu
  </ion-header>
  <ion-content>
    <div class="page">
      @if (loading) { <div class="empty"><ion-spinner></ion-spinner></div> }
      @if (error) { <div class="err">{{ error }}</div> }

      @if (!loading && pets.length === 0) {
        <div class="card empty"><span class="big">?</span>Add your pet first so we know who is coming.
          <div style="margin-top:12px"><ion-button routerLink="/tabs/profile">Add a pet</ion-button></div></div>
      } @else if (!loading) {
        <h3 class="sec" style="margin-top:4px">1. Your pet</h3>
        @for (p of pets; track p.id) {
          <div class="pick row" [class.sel]="petId === p.id" (click)="petId = p.id">
            <div class="avatar">{{ p.species === 'Cat' ? '?' : '?' }}</div>
            <div class="grow"><b>{{ p.name }}</b><div class="sub" style="margin:0">{{ p.breed || p.species }}@if (p.age
          </div>
        }

        <h3 class="sec">2. Service</h3>
        @for (s of services; track s.id) {
          <div class="pick row" [class.sel]="serviceId === s.id" (click)="serviceId = s.id; loadSlots()">
            <div class="emoji">{{ s.emoji }}</div>
            <div class="grow"><b>{{ s.name }}</b><div class="sub" style="margin:0">{{ s.duration }} min</div></div>
            <b>{{ money(s.price) }}</b>
          </div>
        }

        <h3 class="sec">3. Veterinarian</h3>
        @for (v of vets; track v.id) {
          <div class="pick" [class.sel]="vetId === v.id" (click)="vetId = v.id; loadSlots()">
            <b>{{ v.name }}</b><div class="sub" style="margin:0">{{ v.specialty }} · {{ t(v.start_time) }}?{{ t(v.end_t
```

```
          </div>
        }

        <h3 class="sec">4. Date</h3>
        <div class="dates">
          @for (d of days; track d.iso) {
            <div class="d" [class.sel]="date === d.iso" (click)="date = d.iso; loadSlots()">{{ d.dow }}<b>{{ d.day }}</
          }
        </div>

        <h3 class="sec">5. Time</h3>
        @if (slotsLoading) { <ion-spinner name="dots"></ion-spinner> }
        @else if (!ready) { <div class="sub">Choose a service, veterinarian and date to see available times.</div> }
        @else if (slots.length === 0) { <div class="sub">This veterinarian is not working on that day. Try another date
        @else {
          <div class="slot-grid">
            @for (s of slots; track s.time) {
              <div class="slot" [class.sel]="time === s.time" [class.dis]="!s.available" (click)="s.available && (time
            }
          </div>
        }

        <ion-item class="field" style="margin-top:18px"><ion-input label="Notes for the vet (optional)" labelPlacement=
        <ion-button class="pp" expand="block" color="success" (click)="submit()" [disabled]="busy">
          @if (busy) { <ion-spinner name="dots"></ion-spinner> } @else { Book Now }
        </ion-button>
        <p class="sub" style="text-align:center">You will get a confirmation and a reminder in Alerts.</p>
      }
    </div>
  </ion-content>`,
})
export class BookPage {
  private api = inject(ApiService);
  private router = inject(Router);
  private route = inject(ActivatedRoute);
  private toasts = inject(ToastController);
  private cdr = inject(ChangeDetectorRef);

  pets: Pet[] = []; services: Service[] = []; vets: Vet[] = []; slots: Slot[] = [];
  petId = 0; serviceId = 0; vetId = 0; date = ''; time = ''; notes = '';
  loading = true; busy = false; slotsLoading = false; error = '';
  days: { iso: string; dow: string; day: number; today: boolean }[] = [];
  t = t12; money = peso;

  get ready() { return !!(this.serviceId && this.vetId && this.date); }

  private entered = false;
  ngOnInit() { this.entered = true; return this.enter(); }
  ionViewWillEnter() { if (this.entered) { this.entered = false; return; } return this.enter(); }

  private async enter() {
    this.error = ''; this.loading = true; this.time = '';
    try {
      const base = new Date();
      this.days = Array.from({ length: 14 }, (_, i) => {
        const d = new Date(base.getFullYear(), base.getMonth(), base.getDate() + i);
        return { iso: localDate(d), dow: d.toLocaleDateString('en-US', { weekday: 'short' }), day: d.getDate(), today:
      });
      this.date = this.date || this.days[0].iso;
      [this.pets, this.services, this.vets] = await Promise.all([this.api.pets(), this.api.services(), this.api.vets()]
      const pre = Number(this.route.snapshot.queryParamMap.get('service'));
      if (pre && this.services.some(s => s.id === pre)) this.serviceId = pre;
      if (!this.petId || !this.pets.some(p => p.id === this.petId)) this.petId = this.pets[0]?.id || 0;
      if (!this.vetId && this.vets.length) this.vetId = this.vets[0].id;
      if (!this.serviceId && this.services.length) this.serviceId = this.services[0].id;
      await this.loadSlots();
    } catch (e) {
      this.error = (e as Error).message || 'Could not load the booking page.';
      console.error('Book page error:', e);
    } finally {
      this.loading = false;
      this.cdr.detectChanges();
    }
  }

  private slotSeq = 0;
  async loadSlots() {
    this.time = '';
    if (!this.ready) return;
    const seq = ++this.slotSeq;           // only the newest request may update the screen (slow networks)
    this.slotsLoading = true;
    this.cdr.detectChanges();
    try {
      const slots = await this.api.availability(this.vetId, this.serviceId, this.date);
      if (seq !== this.slotSeq) return;
      this.slots = slots; this.error = '';
```

```
    } catch (e) {
      if (seq !== this.slotSeq) return;
      this.error = (e as Error).message; this.slots = [];
    }
    this.slotsLoading = false;
    this.cdr.detectChanges();
  }

  async submit() {
    this.error = '';
    if (!this.petId || !this.serviceId || !this.vetId || !this.date) { this.error = 'Please choose a pet, service, vete
    if (!this.time) { this.error = 'Please pick a time.'; return; }
    if (this.busy) return;              // ignore a second tap while the first is still being sent
    this.busy = true;
    this.cdr.detectChanges();
    try {
      await this.api.book({ pet_id: this.petId, service_id: this.serviceId, vet_id: this.vetId, appt_date: this.date, a
      (await this.toasts.create({ message: 'Appointment booked! ?', duration: 2200, color: 'success' })).present();
      this.notes = '';
      await this.router.navigateByUrl('/tabs/appointments', { replaceUrl: true });
    } catch (e) {
      this.error = (e as Error).message;
      // the error banner is at the top of a long page, so also show it where the person is looking
      (await this.toasts.create({ message: this.error, duration: 3500, color: 'danger', position: 'top' })).present();
      await this.loadSlots();  // the slot may have just been taken, refresh the list
    } finally { this.busy = false; this.cdr.detectChanges(); }
  }
}
```

PHONE_Mobile/src/app/pages/appointments/appointments.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { RouterLink } from '@angular/router';
import { IonContent, IonSegment, IonSegmentButton, IonLabel, IonRefresher, IonRefresherContent, IonButton, AlertControl
import { ApiService, Appointment, localDate, t12, prettyDate, peso } from '../../services/api.service';

@Component({
  selector: 'app-appointments',
  imports: [RouterLink, IonContent, IonSegment, IonSegmentButton, IonLabel, IonRefresher, IonRefresherContent, IonButto
  template: `
  <ion-content>
    <ion-refresher slot="fixed" (ionRefresh)="refresh($event)"><ion-refresher-content></ion-refresher-content></ion-ref
    <div class="page">
      <h1 class="hello">My visits</h1>
      <p class="sub">Upcoming appointments and history.</p>
      <ion-segment [value]="tab" (ionChange)="tab = $any($event.detail.value)">
        <ion-segment-button value="upcoming"><ion-label>Upcoming ({{ upcoming.length }})</ion-label></ion-segment-butto
        <ion-segment-button value="history"><ion-label>History</ion-label></ion-segment-button>
      </ion-segment>
      @if (error) { <div class="err">{{ error }}</div> }
      <div style="margin-top:14px">
        @for (a of list; track a.id) {
          <div class="card">
            <div class="row">
              <div class="emoji">{{ a.service_emoji }}</div>
              <div class="grow"><b>{{ a.pet_name }} · {{ a.service_name }}</b>
                <div class="sub" style="margin:2px 0 0">{{ date(a.appt_date) }} at {{ time(a.appt_time) }}<br>{{ a.vet_
              <span class="badge" [class]="a.status">{{ a.status.replace('_', ' ') }}</span>
            </div>
            @if (a.status === 'pending' || a.status === 'confirmed') {
              <ion-button size="small" fill="outline" color="danger" style="margin-top:10px" (click)="cancel(a)">Cancel
            }
          </div>
        } @empty {
          <div class="empty"><span class="big">?</span>
            {{ tab === 'upcoming' ? 'No upcoming visits.' : 'No past visits yet.' }}
            @if (tab === 'upcoming') { <div style="margin-top:12px"><ion-button routerLink="/book">Book an appointment<
          </div>
        }
      </div>
    </div>
  </ion-content>`,
})
export class AppointmentsPage {
  private api = inject(ApiService);
  private cdr = inject(ChangeDetectorRef);
  private alerts = inject(AlertController);
  private toasts = inject(ToastController);
  tab: 'upcoming' | 'history' = 'upcoming';
  all: Appointment[] = [];
  error = '';
  date = prettyDate; time = t12; money = peso;

  private isUpcoming(a: Appointment) { return ['pending', 'confirmed'].includes(a.status) && a.appt_date >= localDate(n
  get upcoming() { return this.all.filter(a => this.isUpcoming(a)).sort((a, b) => (a.appt_date + a.appt_time).localeCom
  get history() { return this.all.filter(a => !this.isUpcoming(a)).sort((a, b) => (b.appt_date + b.appt_time).localeCom
```

```
  get list() { return this.tab === 'upcoming' ? this.upcoming : this.history; }

  private entered = false;
  ngOnInit() { this.entered = true; return this.load(); }
  ionViewWillEnter() { if (this.entered) { this.entered = false; return; } return this.load(); }
  async refresh(ev: CustomEvent) { await this.load(); ev.detail.complete(); }

  private async load() {
    try { await this.loadInner(); } finally { this.cdr.detectChanges(); }
  }

  private async loadInner() {
    this.error = '';
    try { this.all = await this.api.appointments(); } catch (e) { this.error = (e as Error).message; }
  }

  async cancel(a: Appointment) {
    const alert = await this.alerts.create({
      header: 'Cancel appointment?', message: `${a.pet_name} · ${a.service_name} on ${prettyDate(a.appt_date)}`,
      buttons: [{ text: 'Keep it', role: 'cancel' }, {
        text: 'Cancel visit', role: 'destructive',
        handler: async () => {
          try { await this.api.cancel(a.id); await this.load(); (await this.toasts.create({ message: 'Appointment cance
          catch (e) { (await this.toasts.create({ message: (e as Error).message, duration: 2500, color: 'danger' })).pr
        },
      }],
    });
    await alert.present();
  }
}
```

PHONE_Mobile/src/app/pages/notifications/notifications.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { IonContent, IonRefresher, IonRefresherContent } from '@ionic/angular';
import { ApiService, AppNotification } from '../../services/api.service';

@Component({
  selector: 'app-notifications',
  imports: [IonContent, IonRefresher, IonRefresherContent],
  template: `
  <ion-content>
    <ion-refresher slot="fixed" (ionRefresh)="refresh($event)"><ion-refresher-content></ion-refresher-content></ion-ref
    <div class="page">
      <h1 class="hello">Alerts</h1>
      <p class="sub">Confirmations and reminders from the clinic.</p>
      @if (error) { <div class="err">{{ error }}</div> }
      @for (n of items; track n.id) {
        <div class="card" [style.border-color]="n.is_read ? '' : '#ff8a4c'">
          <div class="row"><div class="avatar">?</div>
            <div class="grow"><b>{{ n.title }}</b><div class="sub" style="margin:2px 0 0">{{ n.body }}</div>
              <small style="color:var(--pp-muted)">{{ n.created_at.slice(0, 16) }}</small></div></div>
        </div>
      } @empty { <div class="empty"><span class="big">?</span>No notifications yet.</div> }
    </div>
  </ion-content>`,
})
export class NotificationsPage {
  private api = inject(ApiService);
  private cdr = inject(ChangeDetectorRef);
  items: AppNotification[] = [];
  error = '';

  private entered = false;
  ngOnInit() { this.entered = true; return this.load(); }
  ionViewWillEnter() { if (this.entered) { this.entered = false; return; } return this.load(); }
  async refresh(ev: CustomEvent) { await this.load(); ev.detail.complete(); }

  private async load() {
    try { await this.loadInner(); } finally { this.cdr.detectChanges(); }
  }

  private async loadInner() {
    this.error = '';
    try {
      this.items = await this.api.notifications();
      if (this.items.some(n => !n.is_read)) {
        // keep the orange "unread" highlight for this visit, clear the badge on the server
        await this.api.markRead();
      }
    } catch (e) { this.error = (e as Error).message; }
  }
}
```

PHONE_Mobile/src/app/pages/profile/profile.page.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { IonContent, IonButton, IonItem, IonInput, IonSelect, IonSelectOption, AlertController, ToastController } from
import { ApiService, Pet, passwordProblem } from '../../services/api.service';

@Component({
  selector: 'app-profile',
  imports: [FormsModule, IonContent, IonButton, IonItem, IonInput, IonSelect, IonSelectOption],
  template: `
  <ion-content>
    <div class="page">
      <h1 class="hello">Profile</h1>
      <div class="card row">
        <div class="avatar" style="background:var(--pp-green);color:#fff;font-weight:700">{{ initial }}</div>
        <div class="grow"><b>{{ api.user()?.full_name }}</b><div class="sub" style="margin:0">{{ api.user()?.email || a
          <div class="sub" style="margin:0">{{ api.user()?.phone }}</div></div>
      </div>

      <h3 class="sec">My pets</h3>
      @if (error) { <div class="err">{{ error }}</div> }
      @for (p of pets; track p.id) {
        <div class="card row">
          <div class="avatar">{{ p.species === 'Cat' ? '?' : '?' }}</div>
          <div class="grow"><b>{{ p.name }}</b>
            <div class="sub" style="margin:0">{{ p.breed || p.species }}@if (p.age) {, {{ p.age }} yr}@if (p.weight) {,
            @if (p.notes) { <small style="color:#a86a0a">{{ p.notes }}</small> }</div>
          <ion-button size="small" fill="clear" color="danger" (click)="remove(p)">Remove</ion-button>
        </div>
      } @empty { <div class="card empty">No pets yet. Add one below.</div> }

      <h3 class="sec">Add a pet</h3>
      <ion-item class="field"><ion-input label="Pet name" labelPlacement="floating" maxlength="60" [(ngModel)]="form.na
      <ion-item class="field"><ion-select label="Species" labelPlacement="floating" [(ngModel)]="form.species">
        @for (s of speciesList; track s) { <ion-select-option [value]="s">{{ s }}</ion-select-option> }</ion-select></i
      <ion-item class="field"><ion-input label="Breed" labelPlacement="floating" maxlength="60" [(ngModel)]="form.breed
      <div class="row">
        <ion-item class="field grow"><ion-input label="Age (years)" labelPlacement="floating" type="number" min="0" [(n
        <ion-item class="field grow"><ion-input label="Weight (kg)" labelPlacement="floating" type="number" min="0" [(n
      </div>
      <ion-item class="field"><ion-input label="Notes (allergies, medication)" labelPlacement="floating" maxlength="255
      <ion-button class="pp" expand="block" (click)="add()">Save pet</ion-button>

      <h3 class="sec">Change password</h3>
      @if (pwError) { <div class="err">{{ pwError }}</div> }
      <ion-item class="field"><ion-input label="Current password" labelPlacement="floating" type="password" autocomplet
      <ion-item class="field"><ion-input label="New password (8+, letters and numbers)" labelPlacement="floating" type=
      <ion-item class="field"><ion-input label="Repeat new password" labelPlacement="floating" type="password" autocomp
      <ion-button class="pp" expand="block" fill="outline" (click)="changePw()">Update password</ion-button>

      <ion-button class="pp" expand="block" fill="outline" color="danger" style="margin-top:28px" (click)="logout()">Lo
    </div>
  </ion-content>`,
})
export class ProfilePage {
  api = inject(ApiService);
  private cdr = inject(ChangeDetectorRef);
  private router = inject(Router);
  private alerts = inject(AlertController);
  private toasts = inject(ToastController);
  pets: Pet[] = [];
  error = ''; pwError = '';
  pw = { old: '', new1: '', new2: '' };
  speciesList = ['Dog', 'Cat', 'Bird', 'Rabbit', 'Hamster', 'Other'];
  form: { name: string; species: string; breed: string; age: string | number; weight: string | number; notes: string }
    { name: '', species: 'Dog', breed: '', age: '', weight: '', notes: '' };

  get initial() { return (this.api.user()?.full_name || '?')[0]; }

  private entered = false;
  ngOnInit() { this.entered = true; return this.load(); }
  ionViewWillEnter() { if (this.entered) { this.entered = false; return; } return this.load(); }
  private async load() {
    try { await this.loadInner(); } finally { this.cdr.detectChanges(); }
  }

  private async loadInner() {
    try { this.pets = await this.api.pets(); } catch (e) { this.error = (e as Error).message; }
  }

  async add() {
    this.error = '';
    if (!this.form.name.trim()) { this.error = 'Enter your pet\'s name.'; return; }
    const age = this.form.age === '' ? null : Number(this.form.age);
    const weight = this.form.weight === '' ? null : Number(this.form.weight);
    if ((age !== null && (isNaN(age) || age < 0)) || (weight !== null && (isNaN(weight) || weight < 0))) { this.error =
```

```
    try {
      await this.api.addPet({ name: this.form.name.trim(), species: this.form.species, breed: this.form.breed, age, wei
      this.form = { name: '', species: 'Dog', breed: '', age: '', weight: '', notes: '' };
      (await this.toasts.create({ message: 'Pet saved ?', duration: 1800, color: 'success' })).present();
      await this.load();
    } catch (e) { this.error = (e as Error).message; }
  }

  async remove(p: Pet) {
    const alert = await this.alerts.create({
      header: `Remove ${p.name}?`, buttons: [{ text: 'Keep', role: 'cancel' }, {
        text: 'Remove', role: 'destructive',
        handler: async () => {
          try { await this.api.deletePet(p.id); await this.load(); }
          catch (e) { this.error = (e as Error).message; }
        },
      }],
    });
    await alert.present();
  }

  async changePw() {
    this.pwError = '';
    if (!this.pw.old) { this.pwError = 'Enter your current password.'; this.cdr.detectChanges(); return; }
    if (this.pw.new1 !== this.pw.new2) { this.pwError = 'The two new passwords do not match.'; this.cdr.detectChanges()
    const problem = passwordProblem(this.pw.new1);
    if (problem) { this.pwError = problem; this.cdr.detectChanges(); return; }
    try {
      await this.api.changePassword(this.pw.old, this.pw.new1);
      this.pw = { old: '', new1: '', new2: '' };
      (await this.toasts.create({ message: 'Password updated', duration: 2000, color: 'success' })).present();
    } catch (e) { this.pwError = (e as Error).message; }
    this.cdr.detectChanges();
  }

  async logout() {
    await this.api.logout();
    this.api.hardReset();          // reload so the next person never sees this person's data
  }
}
```